

АННОТАЦИЯ

В настоящей выпускной квалификационной работе рассматривается разработка программно-аппаратного комплекса поддержки мобильного приложения визуализации результатов работы модификаций алгоритма биогеографической оптимизации, предназначенного для запуска алгоритмов оптимизации на сервере и наглядного представления хода и результатов их работы на мобильном устройстве. Актуальность исследования обусловлена потребностью в удобных средствах изучения и сравнения метаэвристических алгоритмов оптимизации.

В рамках данной работы проведён обзор задачи глобальной оптимизации и метаэвристических методов её решения, алгоритма биогеографической оптимизации и его модификаций, метода роя частиц, способов визуализации результатов оптимизации, а также существующих программ-аналогов. Разработан прототип комплекса, включающий серверное программное обеспечение на языке программирования Python с использованием библиотек NumPy и FastAPI, реализующее вычислительное ядро с семейством из семи методов оптимизации, а также мобильное приложение на языке программирования Dart с использованием фреймворка Flutter, обеспечивающее настройку экспериментов, визуализацию результатов и локальное хранение их истории. Обмен данными между сервером и мобильным приложением реализован поверх протокола WebSocket в формате JSON.

Объем дипломной работы составляет 118 страниц. Работа содержит 4 раздела, 17 рисунков, 24 листинга, 3 таблицы, 30 использованных источников и 2 приложения.

СОДЕРЖАНИЕ

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ	4
ВВЕДЕНИЕ	5
1 Исследовательская часть	7
1.1 Задача глобальной оптимизации и метаэвристики	7
1.2 Алгоритм биогеографической оптимизации	10
1.3 Модификации модели миграции	12
1.4 Модификации базового алгоритма	13
1.5 Метод роя частиц и сравнение с биогеографической оптимизацией	15
1.6 Методы визуализации результатов оптимизации	16
1.7 Обзор программ-аналогов	18
2 Конструкторская часть	20
2.1 Функциональные требования к программному комплексу . . .	20
2.2 Архитектура программного комплекса	21
2.3 Проектирование протокола обмена данными	22
2.4 Модель хранения данных	24
2.5 Проектирование пользовательского интерфейса	25
3 Технологическая часть	27
3.1 Обоснование выбора технологий	27
3.2 Реализация вычислительного ядра сервера	28
3.3 Реализация очереди задач и обработчика	29
3.4 Реализация алгоритмов оптимизации	31
3.5 Реализация мобильного клиента	34
3.5.1 Клиент протокола	34
3.5.2 Модели данных и конверт сообщения	36
3.5.3 Сборка конфигурации задачи	36
3.5.4 Управление состоянием	37
3.5.5 Локальное хранение истории	37

3.6	Пользовательский интерфейс приложения	38
3.6.1	Подключение к серверу	38
3.6.2	Настройка численного эксперимента	39
3.6.3	Выполнение и наблюдение за прогрессом	40
3.6.4	Просмотр результата	40
3.6.5	Трёхмерная визуализация	42
3.6.6	История и сравнение запусков	42
3.7	Реализация визуализации результатов	43
3.7.1	График сходимости	43
3.7.2	Контурная карта	44
3.7.3	Трёхмерная поверхность	44
3.7.4	Анимация процесса оптимизации	45
4	Аналитическая часть	46
4.1	Тестирование программного обеспечения	46
4.1.1	Автоматическое тестирование серверной части	46
4.1.2	Функциональное тестирование клиента	47
4.2	Вычислительные эксперименты и анализ результатов	48
4.2.1	Методика экспериментов	48
4.2.2	Сравнение методов на функции с множеством мини- мумов	48
4.2.3	Сравнение методов на разных функциях	50
4.2.4	Выводы по результатам	50
	ЗАКЛЮЧЕНИЕ	51
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	52
	ПРИЛОЖЕНИЕ А	55
	ПРИЛОЖЕНИЕ Б	59

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

В настоящей ВКР применяют следующие сокращения и обозначения.

- BBO — алгоритм биогеографической оптимизации (Biogeography-Based Optimization)
- GIF — формат обмена графическими данными (Graphics Interchange Format)
- HSI — индекс пригодности среды обитания (Habitat Suitability Index)
- HTTP — протокол передачи гипертекста (Hypertext Transfer Protocol)
- JSON — текстовый формат обмена данными (JavaScript Object Notation)
- PSO — метод роя частиц (Particle Swarm Optimization)
- TCP — протокол управления передачей (Transmission Control Protocol)
- WS — протокол полнодуплексной связи поверх TCP (WebSocket)

ВВЕДЕНИЕ

Многие практические задачи сводятся к поиску глобального минимума целевой функции. Часто у такой функции есть и множество локальных минимумов. Градиентные методы здесь малоэффективны: они сходятся к ближайшему локальному минимуму. Поэтому применяют метаэвристики — алгоритмы, работающие сразу с целой популяцией решений [1]. Один из них — алгоритм биогеографической оптимизации.

Поведение метаэвристик трудно предсказать аналитически, поэтому их оценивают по результатам запусков. Здесь помогает визуализация. По ней видно, как убывает лучшее значение, как распределяется популяция и не наступает ли преждевременная сходимость. Удобно, когда такой инструмент работает прямо на смартфоне и не требует настройки окружения. Этим обусловлена актуальность работы.

Объект исследования — модификации алгоритма биогеографической оптимизации и метод роя частиц, рассматриваемый для сравнения. Программа рассчитана на студентов, изучающих методы оптимизации, и на исследователей, которым нужно быстро сравнить метаэвристики на тестовых задачах.

Цель работы — разработка программно-аппаратного комплекса поддержки мобильного приложения визуализации результатов работы модификаций эволюционного алгоритма оптимизации.

Вычисления должны выполняться на сервере. Он поддерживает семейство методов: классический ВВО, его варианты с синусоидальной и смешанной миграцией, их модификации с адаптивной мутацией, рестартом и локальным поиском, а также метод роя частиц. Алгоритмы запускаются как на встроенных тестовых функциях, так и на функции, заданной пользователем.

Для связи смартфона с сервером нужен протокол обмена. Он поддерживает постановку задачи, передачу прогресса в реальном времени, выдачу результата, отмену и работу очереди при нескольких клиентах.

Пользователь работает с мобильным приложением. Через него настраивается эксперимент, отправляется задача и проводится наблюдение за вычислением. Главное в приложении — показать, как работает алгоритм. Для этого требуется график сходимости в реальном времени, контурная карта, трёхмерная поверхность и анимация популяции.

Комплекс нужен для сравнения методов, поэтому история запусков должна сохраняться. Сохранённые результаты сопоставляются на совмещённых графиках и в таблице. Наконец, комплекс применяется по назначению. На нём проводятся вычислительные эксперименты: методы сравниваются на функциях с различными рельефами и размерностью.

1 Исследовательская часть

1.1 Задача глобальной оптимизации и метаэвристики

Задача глобальной оптимизации состоит в поиске точки, в которой целевая функция принимает наименьшее значение:

$$f(x) \rightarrow \min, \quad x \in D \subset \mathbb{R}^d, \quad (1)$$

где f — целевая функция, D — допустимая область поиска, d — размерность пространства решений. Задача максимизации сводится к минимизации заменой f на $-f$, поэтому далее везде речь идёт о минимизации.

Сложность задачи определяется рельефом целевой функции. Если у функции единственный минимум, найти его несложно. Гораздо труднее функции с множеством локальных минимумов: наряду с глобальным минимумом у них есть много локальных, и отличить один от другого по поведению функции в окрестности точки невозможно. Классические градиентные методы движутся в сторону убывания функции и поэтому сходятся к ближайшему минимуму, не различая, локальный он или глобальный. Для таких задач они малопригодны. К тому же градиентные методы требуют, чтобы функция была дифференцируемой, а это условие выполняется не всегда.

Альтернативой служат метаэвристические алгоритмы. Это итеративные методы поиска, идеи которых заимствованы из природных процессов: эволюции, поведения стай, миграции видов. Их главная особенность — работа не с одной точкой, а с целой популяцией решений. За счёт этого алгоритм одновременно просматривает несколько областей пространства поиска, и вероятность застрять в локальном минимуме снижается. Разнообразие популяции поддерживается случайными механизмами: мутацией, обменом информацией между решениями, отбором. Метаэвристики не требуют дифференцируемости функции и не используют производные, поэтому подходят для широкого круга задач — в том числе для таких, где функция задана лишь алгоритмом её вычисления. Платят за это отсутствием гарантий: найденное решение может оказаться не глобальным минимумом, а лишь близким к нему. К этому классу относятся, в частности, генетические алгоритмы [2], метод роя частиц и рассматриваемый в работе алгоритм биогеографической оптимизации.

Чтобы сравнивать алгоритмы между собой, используют наборы тестовых функций с заранее известными минимумами [3]. Функции подбирают так, чтобы их рельеф был разным и проверял разные стороны поведения метода. В работе используются четыре такие функции.

Функция сферы — самая простая, с единственным минимумом:

$$f_{\text{sph}}(x) = \sum_{i=1}^d x_i^2. \quad (2)$$

Её глобальный минимум равен нулю и достигается в начале координат. На ней удобно оценивать скорость сходимости алгоритма, поскольку локальных ловушек нет. Поверхность и контурная карта функции сферы показаны на рисунке 1.

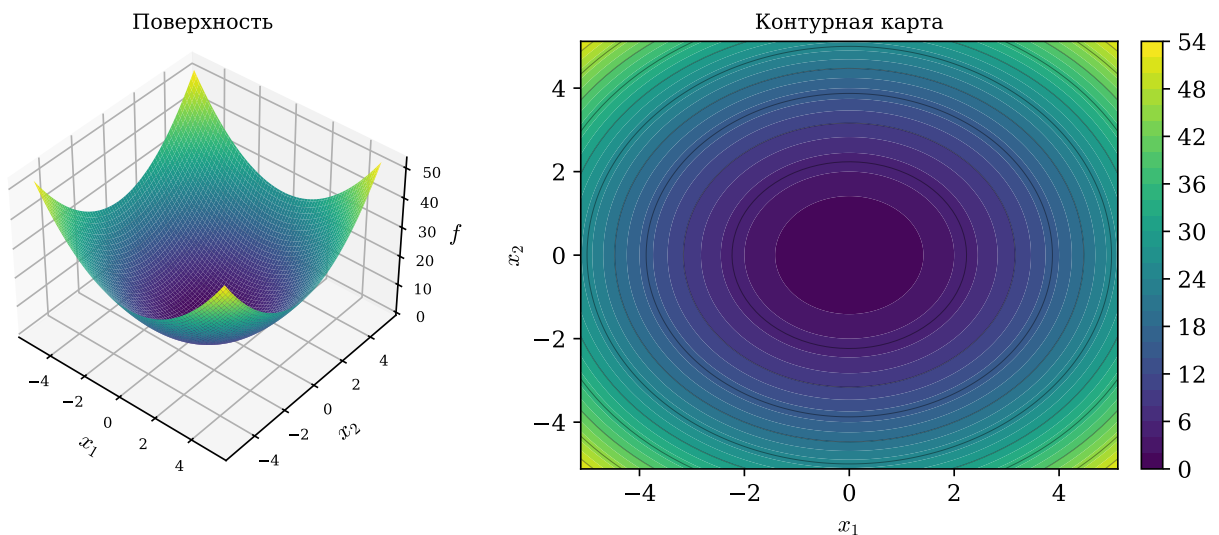


Рисунок 1 – Функция сферы: поверхность и контурная карта

У функции Растригина минимумов очень много:

$$f_{\text{ras}}(x) = Ad + \sum_{i=1}^d (x_i^2 - A \cos(2\pi x_i)), \quad A = 10. \quad (3)$$

Косинусное слагаемое создаёт большое число локальных минимумов, расположенных правильной решёткой. Глобальный минимум, равный нулю, находится в начале координат. Эта функция проверяет, способен ли алгоритм не застревать в локальных минимумах. Её рельеф приведён на рисунке 2.

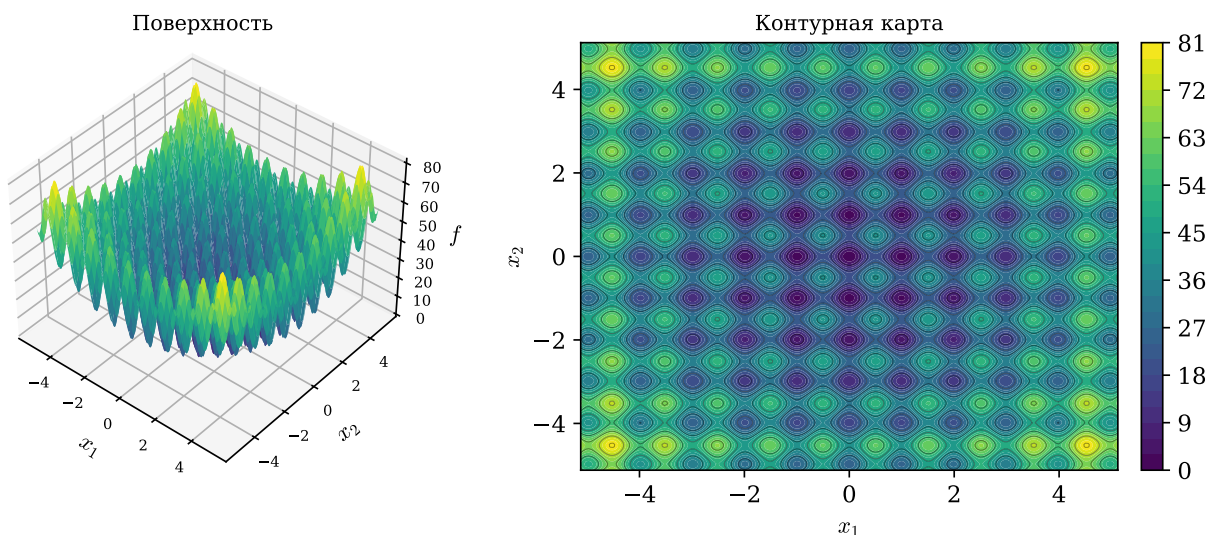


Рисунок 2 – Функция Растригина: поверхность и контурная карта

У функции Экли один глобальный минимум, окружённый почти плоской областью с мелкими неровностями:

$$f_{\text{ack}}(x) = -a \exp \left(-b \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \right) - \exp \left(\frac{1}{d} \sum_{i=1}^d \cos(c x_i) \right) + a + e, \quad (4)$$

где $a = 20$, $b = 0,2$, $c = 2\pi$. Минимум, равный нулю, достигается в начале координат. Вдали от минимума функция почти плоская и плохо подсказывает направление спуска, поэтому она проверяет, насколько точно метод выходит к минимуму. Поверхность и контурная карта функции Экли показаны на рисунке 3.

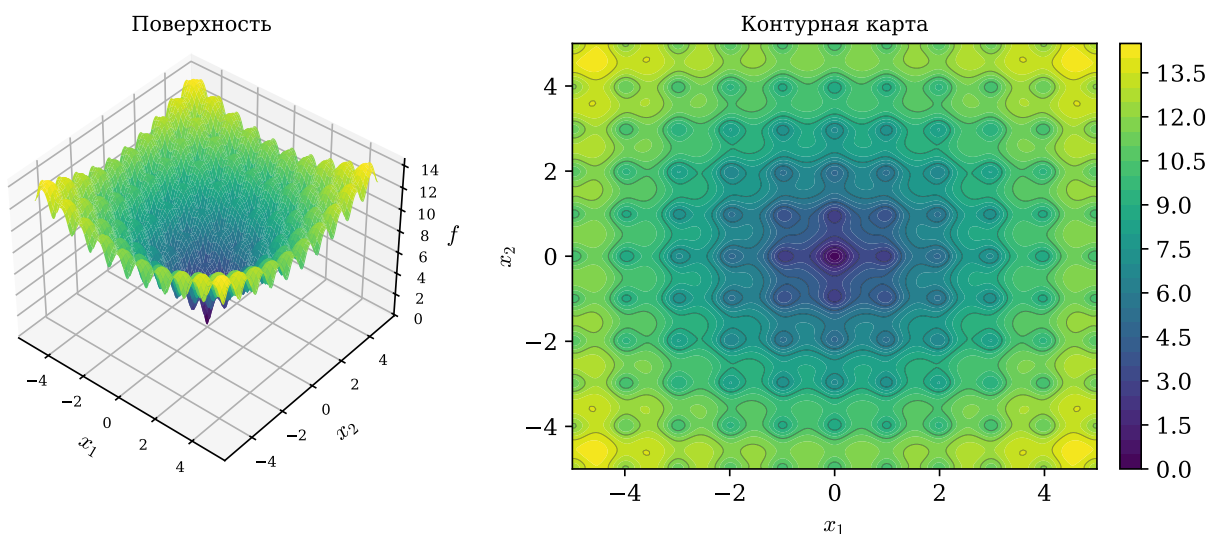


Рисунок 3 – Функция Экли: поверхность и контурная карта

Функция Букина N.6 определена только для двух переменных:

$$f_{\text{buk}}(x_1, x_2) = 100\sqrt{|x_2 - 0,01 x_1^2|} + 0,01 |x_1 + 10|. \quad (5)$$

Её минимум лежит вдоль узкого изогнутого оврага: малый шаг в сторону от него резко увеличивает значение функции. Эта функция проверяет, насколько хорошо алгоритм движется вдоль сложных по форме областей пространства решений. Её рельеф показан на рисунке 4.

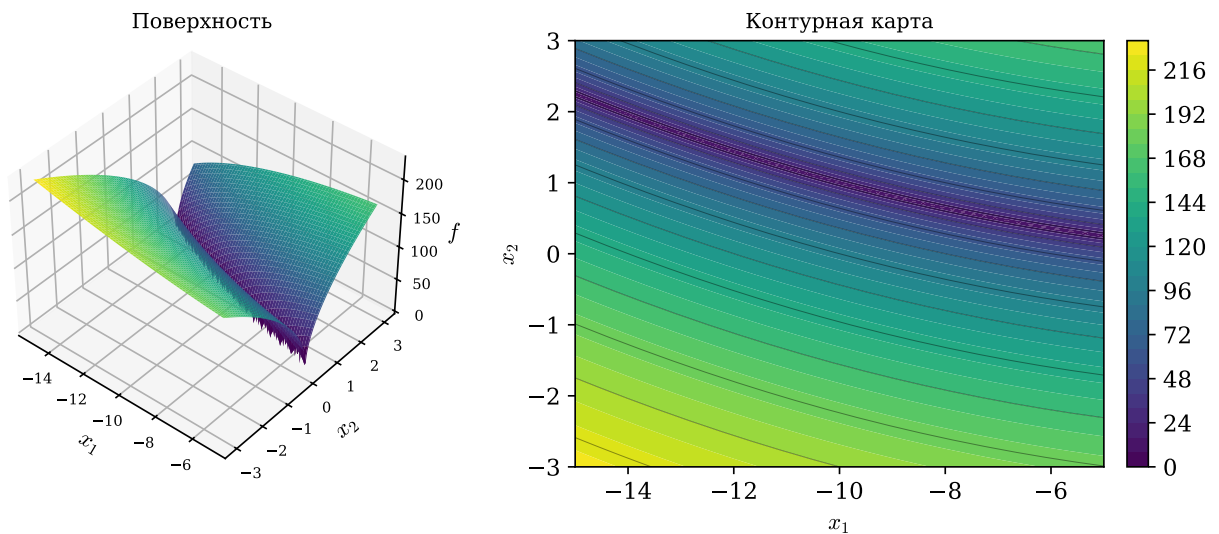


Рисунок 4 – Функция Букина N.6: поверхность и контурная карта

Такой набор охватывает задачи разного характера — от простой функции с единственным минимумом до функции с множеством локальных минимумов и до функции с узким оврагом, — что позволяет всесторонне оценить поведение метода.

1.2 Алгоритм биогеографической оптимизации

Алгоритм биогеографической оптимизации предложил Д. Саймон в 2008 году [4]. В его основе лежит модель биогеографии — науки о распределении живых организмов по островам. В терминах оптимизации каждое решение-кандидат считается отдельным островом, а качество решения — индексом пригодности среды обитания. Хорошему решению отвечает остров с высоким HSI, плохому — с низким. Хорошие острова охотно отдают свои признаки соседям, а плохие принимают чужие; так в популяции происходит обмен информацией.

У алгоритма два основных механизма — миграция и мутация. Пусть популяция состоит из N решений $X_1, \dots, X_N$, отсортированных по возрастанию значения целевой функции. Тогда ранг $r = 1$ получает худшее решение, а ранг $r = N$ — лучшее. Каждому решению сопоставляются две величины: скорость эмиграции λ_r , то есть готовность отдавать свои признаки, и скорость иммиграции μ_r — готовность принимать чужие. В линейной модели Саймона они задаются так:

$$\lambda_r = \frac{r}{N+1}, \quad \mu_r = 1 - \lambda_r, \quad r = 1, \dots, N. \quad (6)$$

Смысл формул простой: чем выше ранг решения, тем больше его скорость эмиграции и меньше скорость иммиграции. Лучшие решения становятся донорами признаков, худшие активнее принимают чужие. Вид линейных кривых миграции показан на левом графике рисунка 5.

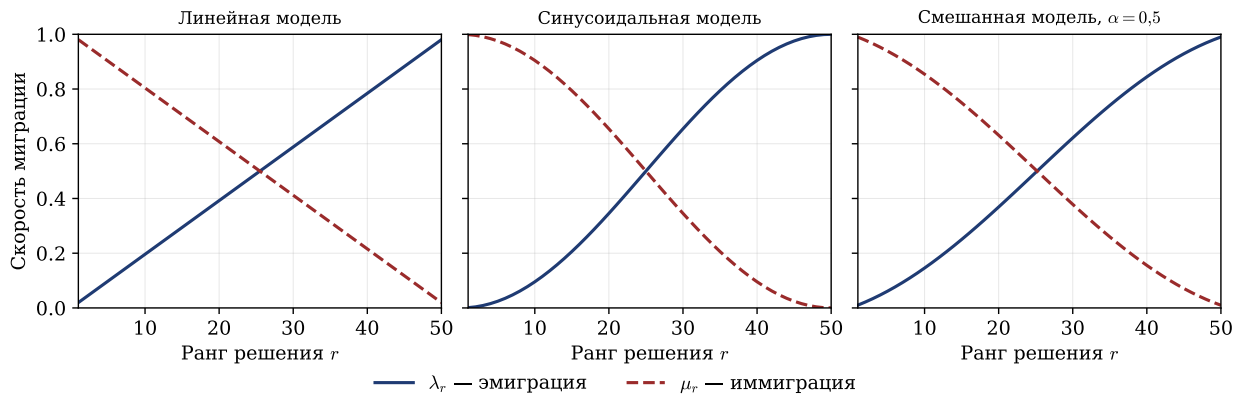


Рисунок 5 – Скорости эмиграции и иммиграции для линейной, синусоидальной и смешанной моделей миграции

Сам обмен идёт по схеме частичной иммиграции. Для каждого решения-получателя X_k , не входящего в число элитных, перебираются все его компоненты. Компонента с номером s заменяется с вероятностью μ_k : если замена происходит, на её место подставляется компонента s решения-донора. Донор выбирается случайно, причём вероятность стать донором пропорциональна скорости эмиграции λ_r , поэтому чаще донорами оказываются хорошие решения. Компоненты обновляются независимо друг от друга, и новое решение собирается из фрагментов нескольких разных доноров.

Мутация добавляет в популяцию случайность. Каждая компонента решения с вероятностью p_{mut} заменяется новым значением, равномерно выбран-

ным в допустимых границах. Это поддерживает разнообразие популяции и мешает ей преждевременно сойтись к одному минимуму.

Чтобы лучшие найденные решения не испортились при миграции и мутации, применяют элитарность: e лучших решений поколения переносятся в следующее без изменений. Один из распространённых способов сделать это таков: новые решения, полученные после миграции и мутации, оцениваются целевой функцией, после чего e худших из них заменяются сохранёнными элитными решениями.

Таким образом, одно поколение классического ВВО состоит из следующих шагов: вычислить целевую функцию для всей популяции и отсортировать решения по качеству; по рангу каждого решения найти λ_r и μ_r ; для всех неэлитных решений выполнить частичную иммиграцию, заменяя отдельные компоненты значениями доноров; применить мутацию; перенести элитные решения в новую популяцию; обновить лучшее найденное значение и историю сходимости. Эти шаги повторяются заданное число поколений.

1.3 Модификации модели миграции

В классическом ВВО скорости миграции линейно зависят от ранга решения. Форма кривой миграции определяет, насколько интенсивно лучшие решения передают свои признаки худшим и как обмен информацией распределён по популяции. Линейная модель распределяет эту интенсивность равномерно, и в ряде задач это приводит к преждевременной сходимости: решения быстро становятся похожими, а поиск теряет разнообразие [5]. Изменить поведение алгоритма можно, не трогая остальные его части, — достаточно заменить способ вычисления скоростей λ_r и μ_r .

Первый вариант — синусоидальная модель миграции. В ней кривые задаются так:

$$\lambda_r^{\sin} = \frac{1 - \cos\left(\pi \frac{r}{N}\right)}{2}, \quad \mu_r^{\sin} = 1 - \lambda_r^{\sin}, \quad r = 1, \dots, N. \quad (7)$$

Синусоидальная кривая нелинейна: на краях ранговой шкалы она пологая, а в середине меняется резко. Это видно на среднем графике рисунка 5. У группы лучших решений скорости эмиграции близки и велики, у группы худших так же близки и велики скорости иммиграции, а вблизи середины роль решения

быстро меняется с получателя на донора. В результате синусоидальная модель сильнее, чем линейная, разделяет популяцию на доноров и получателей признаков.

Второй вариант — смешанная модель. Её скорости строятся как взвешенная сумма линейной и синусоидальной кривых:

$$\lambda_r^{\text{mix}} = \alpha \lambda_r^{\text{lin}} + (1 - \alpha) \lambda_r^{\text{sin}}, \quad \mu_r^{\text{mix}} = 1 - \lambda_r^{\text{mix}}, \quad r = 1, \dots, N, \quad (8)$$

где $\alpha \in [0, 1]$ — параметр смешения, задающий долю линейной кривой. При $\alpha = 1$ модель совпадает с линейной, при $\alpha = 0$ — с синусоидальной, а промежуточные значения дают форму кривой между ними. Это позволяет настраивать баланс между широким исследованием пространства и уточнением уже найденных решений. Смешанная кривая при $\alpha = 0,5$ показана на правом графике рисунка 5.

Все три модели — линейная, синусоидальная и смешанная — используют одну и ту же схему иммиграции и мутации. Различаются они только формулами для λ_r и μ_r . Благодаря этому модель миграции остаётся самостоятельным и легко заменяемым компонентом алгоритма: переход от одного варианта к другому сводится к замене способа расчёта скоростей, а вся остальная схема поиска не меняется.

1.4 Модификации базового алгоритма

Кроме изменения формы кривых миграции, поиск можно улучшить, дополнив базовый алгоритм механизмами адаптации. Все три механизма, описанные ниже, не зависят от модели миграции и применимы к любому варианту ВВО.

Первый механизм — адаптивная мутация. В базовом алгоритме вероятность мутации p_{mut} постоянна на протяжении всей работы. Однако в начале поиска высокая мутация полезна, поскольку поддерживает разнообразие популяции и помогает охватить широкую область пространства решений, а ближе к концу она лишь вносит шум и мешает уточнять найденные минимумы. Поэтому вероятность мутации делают убывающей: она линейно снижается от начального значения P_{start} до конечного P_{end} по мере смены поколений. В

поколении t её значение равно

$$p_{\text{mut}}^{(t)} = P_{\text{start}} + \frac{t}{T-1}(P_{\text{end}} - P_{\text{start}}), \quad t = 0, 1, \dots, T-1, \quad (9)$$

где T — общее число поколений. Ранние поколения работают с высокой мутацией, поздние — с пониженной.

Второй механизм — детектор стагнации и частичный рестарт. Стагнацией называют ситуацию, когда лучшее найденное значение целевой функции не улучшается несколько поколений подряд. Базовый ВВО не умеет из неё выходить. Детектор стагнации подсчитывает поколения без улучшений; как только счётчик достигает порога W , выполняется частичный рестарт: доля ρ худших решений удаляется и заменяется новыми случайными решениями из допустимой области. Одновременно текущая вероятность мутации временно увеличивается в k раз, но не выше P_{start} . После рестарта счётчик обнуляется, а линейное убывание мутации продолжается [6]. Такой механизм помогает выбраться из локального минимума, не теряя при этом уже найденные хорошие решения.

Третий механизм — локальный поиск. Популяционные алгоритмы хорошо исследуют пространство в целом, но часто недостаточно точны при уточнении минимума. Чтобы это исправить, через заданное число поколений выбирается несколько лучших решений, и каждое из них уточняется серией шагов. На каждом шаге к решению добавляется случайное гауссовское возмущение с нулевым средним и масштабом, пропорциональным ширине допустимой области. Масштаб возмущения убывает с ростом номера поколения: в начале шаги крупнее, в конце мельче. Если возмущённая точка оказывается лучше, она принимается, иначе отвергается. Локальный поиск применяют только к лучшим решениям, чтобы не тратить вычисления на заведомо неперспективные.

Три механизма дополняют друг друга: адаптивная мутация управляет уровнем случайности на всём протяжении поиска, детектор стагнации обеспечивает выход из локальных ловушек, а локальный поиск повышает точность итогового результата. Они независимы друг от друга, поэтому могут применяться как вместе, так и по отдельности.

1.5 Метод роя частиц и сравнение с биогеографической оптимизацией

Метод роя частиц предложили Дж. Кеннеди и Р. Эберхарт в 1995 году [7]. Его идея навеяна поведением стаи птиц или косяка рыб при поиске пищи. Популяция здесь — это рой частиц, и каждая частица соответствует одному решению. Состояние частицы i задаётся положением $x_i \in \mathbb{R}^d$ и скоростью $v_i \in \mathbb{R}^d$.

На каждой итерации t скорость и положение частицы пересчитываются по формулам

$$v_i^{(t+1)} = \omega v_i^{(t)} + c_1 r_1 \left(p_{i,\text{best}}^{(t)} - x_i^{(t)} \right) + c_2 r_2 \left(g_{\text{best}}^{(t)} - x_i^{(t)} \right), \quad (10)$$

$$x_i^{(t+1)} = x_i^{(t)} + v_i^{(t+1)}, \quad (11)$$

где $p_{i,\text{best}}$ — лучшее положение, в котором частица i побывала за всё время; g_{best} — лучшее положение среди всех частиц роя; ω — инерционный коэффициент; c_1 и c_2 — когнитивная и социальная составляющие; $r_1, r_2 \sim U(0,1)$ — случайные числа [8].

Инерционный коэффициент ω задаёт баланс между исследованием пространства и уточнением найденного. При большом ω частицы сохраняют направление движения и шире обследуют область поиска, при малом — сильнее притягиваются к лучшим найденным точкам. На практике ω часто линейно убывает в ходе работы — от значения около 0,9 в начале до 0,4 в конце. Чтобы частицы не разлетались слишком далеко за одну итерацию, скорость по каждой координате обычно ограничивают по модулю некоторой величиной v_{max} , а вышедшее за границы положение возвращают в допустимую область [9].

Несмотря на внешнее сходство — оба алгоритма работают с популяцией решений и обмениваются информацией внутри неё — механизмы этого обмена устроены по-разному.

В PSO каждая частица учитывает два ориентира: своё лучшее прошлое положение и глобально лучшую точку роя. На задачах с простым рельефом это даёт быструю сходимость — рой быстро стягивается к минимуму. Но на задачах с множеством локальных минимумов тот же механизм оборачивается слабостью: если глобально лучшая точка попала в окрестность локального минимума, рой собирается там, и выбраться оттуда без перезапуска трудно.

В BBO обмен идёт покомпонентно: каждая компонента решения может быть независимо заимствована у любого донора. Это создаёт более разнообразный поток информации и мешает популяции полностью унифицироваться. Вдобавок элитарность защищает лучшие решения, а мутация постоянно вносит случайные возмущения. В сумме BBO устойчивее на сложных задачах высокой размерности, хотя на простых функциях сходится несколько медленнее [10].

Поэтому PSO и BBO занимают разные ниши. PSO удобен там, где рельеф простой и важна скорость сходимости. BBO и его модификации выигрывают на задачах с множеством локальных минимумов, где важно сохранять разнообразие популяции на протяжении всего поиска. Это согласуется с общим положением о том, что эффективность метода оптимизации зависит от свойств целевой функции и ни один метод не превосходит остальные на всех задачах [11]. В работе PSO используется как метод сравнения, относительно которого оценивается поведение вариантов BBO.

1.6 Методы визуализации результатов оптимизации

Поведение метаэвристик трудно оценить по одним лишь числам, поэтому результаты их работы принято представлять наглядно. Визуализация решает несколько задач: она позволяет наблюдать за ходом поиска и замечать нежелательные явления — преждевременную сходимость, стагнацию, потерю разнообразия популяции, — а также сравнивать алгоритмы между собой и понимать, как метод исследует пространство решений. Сложились несколько типов визуального представления, каждый из которых решает свою задачу.

Наиболее универсальное средство — график сходимости. По оси абсцисс на нём откладывается номер поколения, по оси ординат — лучшее значение целевой функции, найденное к этому поколению:

$$f_{\text{best}}^{(t)} = \min_{0 \leq \tau \leq t} f\left(x_{\text{best}}^{(\tau)}\right). \quad (12)$$

По построению такой график монотонно невозрастающий: лучшее найденное значение не может ухудшиться. По форме кривой судят о характере сходимости. Резкий начальный спад с выходом на плато говорит о быстрой сходимости к некоторому значению, которое может оказаться локальным минимумом. Плавный равномерный спад свидетельствует о планомерном ис-

следовании пространства. Ступенчатая кривая характерна для алгоритмов с рестартом: после каждого рестарта метод находит лучшее решение, и кривая делает скачок вниз. Когда значения целевой функции убывают на несколько порядков, по оси ординат применяют логарифмическую шкалу — иначе весь интересный участок кривой сжимается у нуля и становится нечитаемым. Для сравнения методов кривые их сходимости совмещают на одних осях, различая цветом и типом линии.

График сходимости показывает поведение алгоритма в терминах значений функции, но не говорит, где именно в пространстве решений находится популяция. Это показывает визуализация пространства поиска. При размерности $d = 2$ строится контурная карта: по осям откладываются координаты x_1 и x_2 , а значение $f(x_1, x_2)$ кодируется цветом, причём тёмные тона отвечают малым значениям функции, то есть перспективным областям [12]. Допустимая область разбивается на равномерную сетку, для каждой узловой точки вычисляется значение функции, и полученная матрица отображается как изображение; поверх него наносятся маркеры особей популяции. Если сетка содержит $M \times M$ точек, для построения карты требуется M^2 вычислений функции, поэтому карту рассчитывают один раз до запуска, а в ходе работы обновляют только маркеры. При $d > 2$ прямое отображение невозможно: тогда используют первые две координаты особей, а остальные фиксируют, строя сечение функции. Такой подход даёт лишь приближённое представление, но позволяет судить о расположении популяции и в многомерных задачах.

Контурная карта кодирует значение функции цветом, но не передаёт объёмной структуры рельефа: глубину минимумов и крутизну склонов по оттенку оценить трудно. Эту структуру передаёт трёхмерный поверхностный график, где значение $f(x_1, x_2)$ отображается высотой точки. Поверхность строится по той же сетке, что и контурная карта, а соседние узлы образуют четырёхугольные грани. Чтобы показать трёхмерную сцену на плоском экране, применяют проекцию, а грани рисуют по алгоритму художника — от дальних к ближним, так что ближние перекрывают дальние без использования буфера глубины.

Главное преимущество трёхмерного графика — возможность вращения. Точка обзора задаётся двумя углами: поворотом вокруг вертикальной оси φ_z и вокруг горизонтальной оси φ_x . При изменении углов координаты

каждой вершины (x, y, z) пересчитываются последовательным поворотом в двух плоскостях:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \varphi_z & -\sin \varphi_z \\ \sin \varphi_z & \cos \varphi_z \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}, \quad (13)$$

$$\begin{pmatrix} y'' \\ z'' \end{pmatrix} = \begin{pmatrix} \cos \varphi_x & -\sin \varphi_x \\ \sin \varphi_x & \cos \varphi_x \end{pmatrix} \begin{pmatrix} y' \\ z \end{pmatrix}. \quad (14)$$

Экранными координатами вершины служат x' и y'' с учётом масштаба и смещения к центру холста. Вращение позволяет рассмотреть поверхность под любым углом и заметить узкие овраги целевой функции, плохо различимые на плоской карте. Поверх поверхности можно нанести траекторию лучшего решения — последовательность точек $(x_1^{(t)}, x_2^{(t)}, f_{\text{best}}^{(t)})$, соединённых линией, которая показывает, как метод продвигался по рельефу.

Контурная карта и поверхность дают снимок одного состояния, но для понимания поиска важна динамика — как популяция смещалась от поколения к поколению. Зафиксировать её позволяет анимация в формате GIF, которая хранит последовательность кадров, сменяющих друг друга с заданной частотой [13]. Каждый кадр — это контурная карта с маркерами особей на соответствующем поколении; накопленные кадры собираются в один файл. GIF удобен тем, что не требует внешних плееров, просто собирается из готовых изображений и легко сохраняется. Его ограничение — не более 256 цветов в кадре и заметный размер файла при большом числе кадров, но для задач визуализации оптимизации это не критично.

1.7 Обзор программ-аналогов

Задача наблюдения за работой алгоритмов оптимизации не нова, и для неё уже создан ряд программных средств.

Инструмент STNWeb — это веб-приложение для анализа поведения метаэвристик. Он строит так называемые сети траекторий поиска: ориентированные графы, по которым видно, как алгоритм перемещается по пространству решений. STNWeb позволяет сравнивать несколько прогонов нескольких алгоритмов на одной задаче и помогает понять, почему тот или иной метод работает хуже [14]. Сильная сторона инструмента — глубокий анализ траекторий и удобное сравнение алгоритмов. Однако STNWeb ориентирован на

исследователей, требует подготовки входных данных в специальном формате и не показывает ход поиска в реальном времени.

Проект OptimViz решает другую задачу — он наглядно показывает, как различные методы оптимизации шаг за шагом движутся по поверхности тестовой функции, и сохраняет результат в виде анимаций [15]. Это удобно для обучения и демонстраций. Слабые стороны: инструмент написан на языке MATLAB и требует платных дополнительных пакетов, а сами анимации создаются заранее, без возможности интерактивно менять параметры запуска.

Веб-демонстрация Э. Дюпона позволяет прямо в браузере выбрать точку старта на контурной карте двумерной функции и посмотреть, как к минимуму сходятся разные алгоритмы [16]. Она интерактивна и наглядна, но ограничена двумя измерениями и фиксированным набором функций, а сами алгоритмы относятся к градиентным методам обучения нейронных сетей, а не к метаэвристикам.

Дашборд moPLOT строит детальные изображения рельефа тестовых функций, в том числе для задач многокритериальной оптимизации [17]. Он даёт качественную визуализацию ландшафта, но предназначен в первую очередь для анализа самих функций, а не для наблюдения за работой конкретного алгоритма, и требует навыков работы со средой R.

Сравнение показывает, что существующие инструменты решают близкие задачи, но каждый со своими ограничениями. Одни анализируют только завершённые запуски, другие работают лишь с двумя измерениями или с заранее подготовленными анимациями, третьи требуют специальной среды и рассчитаны на опытных исследователей. Кроме того, все рассмотренные средства предназначены для настольных компьютеров или браузера и не приспособлены для работы на мобильном устройстве.

Разрабатываемая система занимает свободную нишу. Она сочетает наблюдение за ходом поиска в реальном времени, визуализацию пространства решений в виде контурной карты и трёхмерной поверхности, анимацию динамики популяции и сравнение нескольких запусков, причём всё это доступно на мобильном устройстве и не требует от пользователя настройки вычислительного окружения. Основное внимание уделено семейству алгоритмов биогеографической оптимизации и их модификациям, что делает систему удобным средством для изучения именно этих методов.

2 Конструкторская часть

2.1 Функциональные требования к программному комплексу

Сначала нужно определить, что комплекс должен делать. Требования делятся на функциональные и нефункциональные.

Основной сценарий — проведение вычислительного эксперимента. Пользователь выбирает метод оптимизации, задаёт целевую функцию и параметры задачи и запускает вычисление. Целевой функцией может быть встроенная тестовая функция или произвольное выражение, введённое пользователем. Для задачи задаются размерность и границы области поиска, для метода — его параметры: размер популяции, число поколений, вероятность мутации.

Во время вычисления система показывает ход поиска, не дожидаясь его конца. Видны номер текущего поколения, лучшее найденное значение и график сходимости, который обновляется по мере поступления данных. Задачу можно отменить. Если она ещё не запущена, видно её место в очереди.

После завершения система представляет результат: лучшее решение, значение функции в нём, график сходимости и числовые показатели качества. Пространство поиска отображается визуализацией: контурной картой с нанесёнными решениями, трёхмерной поверхностью с возможностью вращения и анимацией популяции.

Визуализация задаёт ряд требований. График сходимости обновляется инкрементально: новые точки добавляются к уже построенной кривой, а не строятся заново. Так пользователь видит ход поиска во время вычислений, и снижается нагрузка на устройство. Экран смартфона мал, поэтому подписи осей и легенда должны быть читаемыми и не перекрывать график. Контурная карта и поверхность требуют многих вычислений функции, поэтому считаются один раз. При наблюдении и вращении меняются только маркеры и точка обзора, но не сам рельеф.

Результаты сохраняются, чтобы вернуться к ним позже. Система хранит историю запусков, позволяет просматривать её, отбирать отдельные запуски и сравнивать их — на совмещённом графике сходимости, в таблице показателей и на столбчатой диаграмме.

Нефункциональные требования задаёт назначение системы. Основное устройство — смартфон, поэтому приложение работает на мобильной плат-

форме и остаётся отзывчивым: интерфейс не подвисает при вычислениях и построении графиков. Сами вычисления длительны и ресурсоёмки, поэтому вынесены на сервер. Сервер обслуживает несколько клиентов и упорядочивает их задачи.

2.2 Архитектура программного комплекса

Возможности комплекса распределяются между двумя частями. Одна обращена к пользователю и работает на смартфоне, другая выполняет вычисления. Совмещать их в одном приложении нецелесообразно: оптимизация на больших популяциях и высоких размерностях нагружает процессор, а смартфон ограничен по мощности и заряду. Поэтому выбрана клиент-серверная архитектура: вычисления вынесены на сервер, а приложение отвечает за управление и отображение результатов. Общая структура показана на рисунке 6.

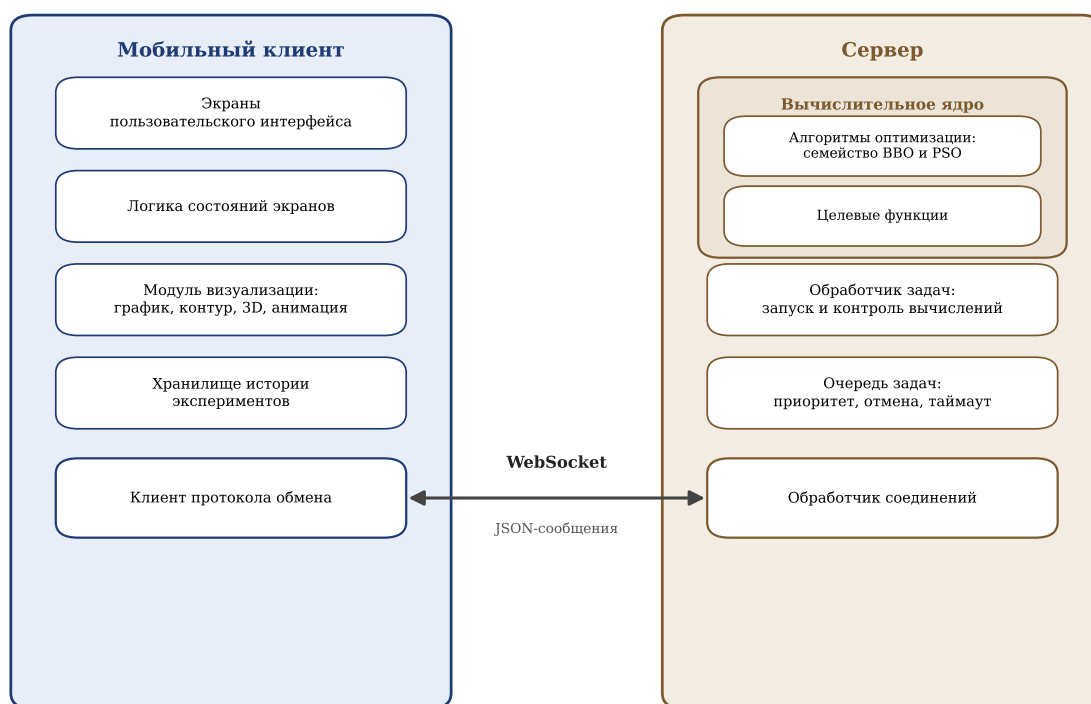


Рисунок 6 – Архитектура программного комплекса

Мобильный клиент — это приложение, с которым работает пользователь. Оно состоит из нескольких слоёв. Экраны образуют интерфейс: настройку эксперимента, наблюдение за прогрессом, просмотр результата и работу с историей. За экранами стоит слой логики состояний. Он хранит состояние каждого экрана и реагирует на действия пользователя и данные с сервера.

Отдельный модуль отвечает за визуализацию: график сходимости, контурную карту, поверхность и анимацию. История хранится в локальной базе данных на устройстве, поэтому с прошлыми результатами можно работать и без подключения. Связь с сервером обеспечивает клиент протокола.

Сервер выполняет вычисления и не имеет интерфейса. Запросы клиентов принимает обработчик соединений: он разбирает сообщения и направляет их дальше. Клиентов может быть несколько, а вычисления длительны, поэтому задачи помещаются в очередь. Она упорядочивает их и позволяет отменять ещё не выполненные. Выполнением занимается обработчик задач: он берёт задачу из очереди, запускает алгоритм и шлёт клиенту данные о прогрессе. Сами алгоритмы — семейство ВВО и метод роя частиц — вместе с целевыми функциями образуют вычислительное ядро. Какой алгоритм и функцию использовать, обработчик определяет по запросу.

Клиент и сервер связаны протоколом WebSocket. В отличие от обычного запроса и ответа по HTTP, WebSocket держит постоянное двустороннее соединение, по которому сервер сам присылает данные [18]. Это и нужно для прогресса: во время вычисления сервер шлёт сведения о каждом поколении, а приложение обновляет график в реальном времени. Сообщения передаются в формате JSON, его структура описана далее.

2.3 Проектирование протокола обмена данными

Клиент и сервер обмениваются сообщениями по постоянному соединению WebSocket. Чтобы обмен был упорядоченным и расширяемым, нужен прикладной протокол. Он задаёт, какие сообщения бывают, как они устроены и в каком порядке идут. Все сообщения передаются в формате JSON [19]. Этот формат удобочитаем, поддерживается без дополнительных библиотек и разбирается стандартными средствами обоих языков.

Каждое сообщение состоит из двух частей — заголовка и полезной нагрузки. Заголовок содержит служебные поля, общие для всех сообщений: версию протокола, тип, уникальный идентификатор, метку времени и поля для связи сообщений и идентификации задачи. Полезная нагрузка зависит от типа и содержит его данные: например, параметры задачи или сведения о поколении. Так служебную часть можно обрабатывать единообразно.

Все сообщения делятся на несколько групп по назначению. Служебные сообщения обслуживают само соединение:

- `hello` — рукопожатие при подключении и обмен сведениями о возможностях сторон;
- `ping` и `pong` — проверка того, что соединение живо;
- `error` — сообщение об ошибке протокола, проверки данных или выполнения.

Сообщения управления задачами обеспечивают её постановку и сопровождение:

- `job.submit` — запрос на запуск задачи с указанием метода, целевой функции и параметров;
- `job.accepted` — подтверждение приёма задачи и присвоение ей идентификатора;
- `cancel` — запрос на отмену задачи;
- `job.status.get` и `job.status` — запрос текущего состояния задачи и ответ на него.

Сообщения о ходе и итогах вычисления идут от сервера к клиенту:

- `job.started` — уведомление о начале выполнения задачи;
- `job.progress` — данные об очередном поколении: его номер и лучшее найденное значение;
- `job.result` — итоговый результат с лучшим решением и показателями качества;
- `job.finished` — уведомление о завершении задачи с указанием итогового состояния.

Порядок смены состояний задачи и отправки этих сообщений образует её жизненный цикл, показанный на рисунке 7. Поступившая задача попадает в очередь. Когда обработчик берёт её в работу, она переходит в выполнение, и клиенту уходит уведомление о начале. Во время выполнения сервер периодически шлёт прогресс. Завершиться задача может по-разному: успешно, с ошибкой, по отмене или по превышению времени. Отменить её можно и пока она стоит в очереди.

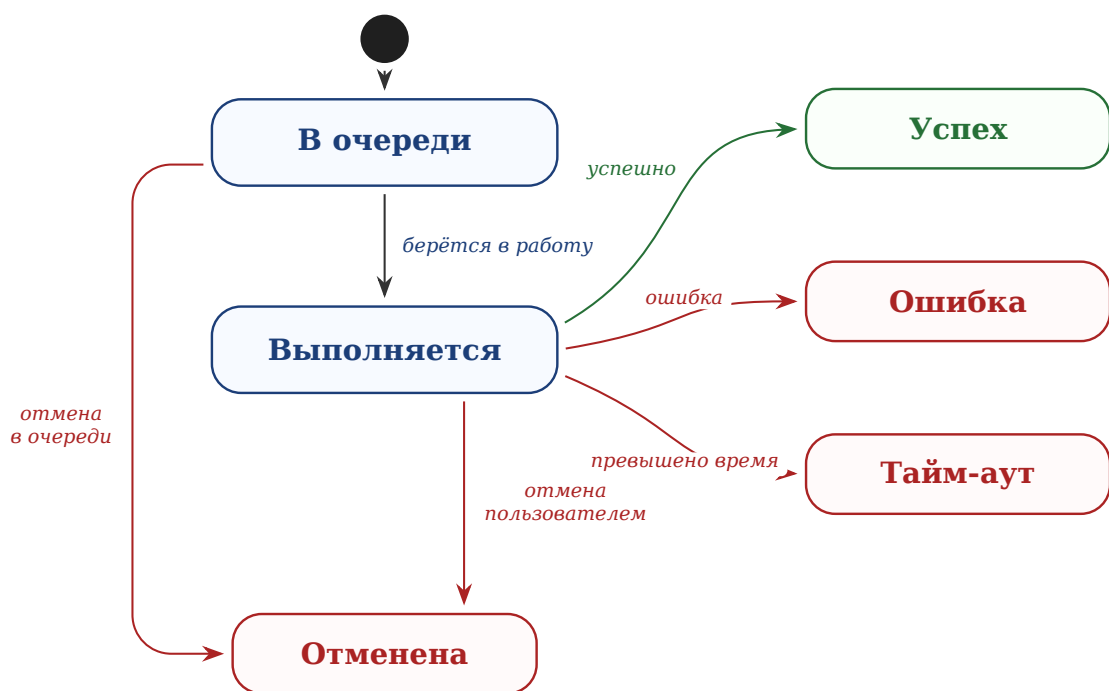


Рисунок 7 – Жизненный цикл задачи на сервере

Протокол предусматривает и передачу больших данных. Если сообщение превышает допустимый размер, его нагрузку можно разбить на части — чанки — и собрать на клиенте по идентификатору потока. Заголовок допускает и сжатие нагрузки. Эти средства заложены на будущее: при текущих объёмах они не нужны, и сообщения идут целиком в несжатом виде.

2.4 Модель хранения данных

Система должна сохранять историю экспериментов, чтобы пользователь возвращался к прошлым запускам и сравнивал их. Отсюда вытекает несколько задач хранилища. Во-первых, по каждому запуску нужно сохранять достаточно сведений, чтобы опознать его в списке и понять, что вычислялось: каким методом, на какой задаче и с каким итогом. Во-вторых, нужно хранить сам результат — не только итоговое число, но и данные для графиков и визуализации. В-третьих, записи должны быть однородными, чтобы их можно было перебирать, отбирать и сопоставлять.

Чтобы учесть эти требования, нужно понять, какие данные и в каких отношениях хранить. Для этого построена концептуальная модель данных в виде диаграммы «сущность-связь» на рисунке 8. Она описывает предметную область на уровне сущностей и связей, без привязки к способу хранения.

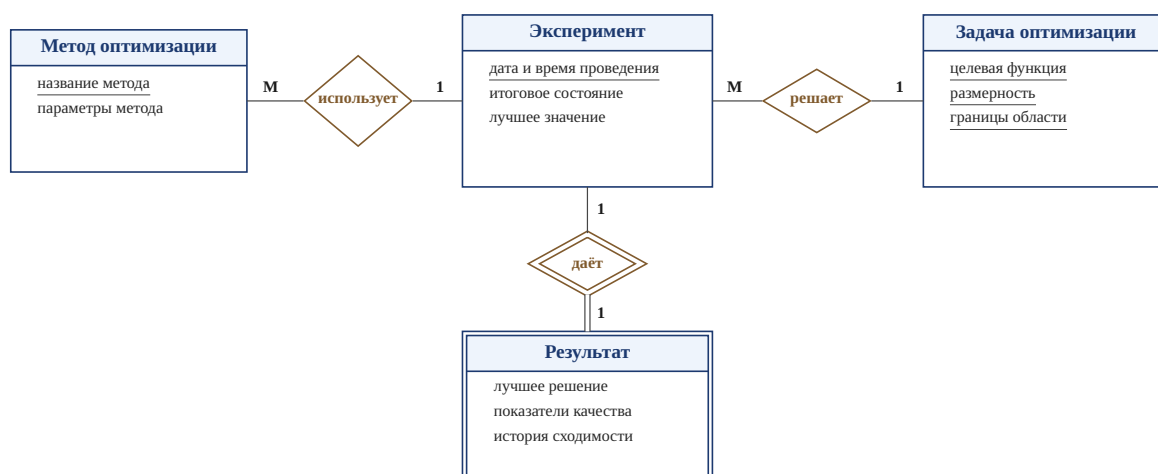


Рисунок 8 – Модель «сущность-связь» истории экспериментов

Центральная сущность — эксперимент. Он соответствует одному завершённому запуску и описывается датой и временем проведения, итоговым состоянием и лучшим найденным значением. Ключом служит дата и время: запуск определяется моментом начала.

С экспериментом связаны три сущности. Метод оптимизации хранит название метода и значения его параметров; ключом служит название. Один метод используется во многих экспериментах, поэтому связь «использует» имеет тип «многие к одному». Задача оптимизации состоит из целевой функции, размерности и границ области поиска. Эти три атрибута образуют составной ключ: одна функция в разных размерностях задаёт разные задачи. Связь «решает» тоже имеет тип «многие к одному».

Результат хранит итог запуска: лучшее решение, числовые показатели и историю сходимости. Он не имеет собственного ключа и не существует отдельно от эксперимента, поэтому моделируется как слабая сущность. Связь «даёт» идентифицирующая, тип «один к одному»: каждому эксперименту соответствует один результат. В итоге эксперимент объединяет вокруг себя метод, задачу и результат, а история — это набор таких однородных записей на устройстве пользователя.

2.5 Проектирование пользовательского интерфейса

Интерфейс строится вокруг основного сценария: настроить эксперимент, запустить его, понаблюдать за вычислением и изучить результат. Каждый шаг вынесен на отдельный экран, чтобы не перегружать пользователя.

Нужны также экраны истории и сравнения. Переходы между экранами показаны на рисунке 9.

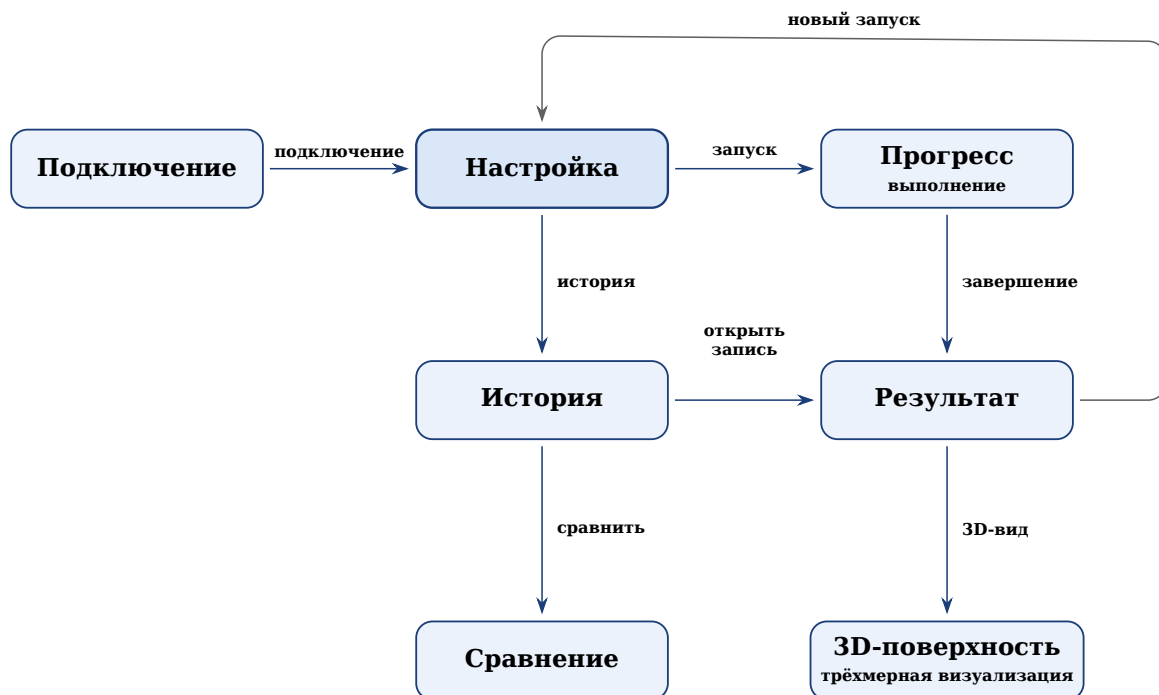


Рисунок 9 – Схема навигации между экранами приложения

Работа начинается с экрана подключения: задаётся адрес сервера и устанавливается соединение. Затем открывается экран настройки — центральный экран приложения. Здесь выбирается метод оптимизации, задаются целевая функция, размерность, границы области поиска и параметры метода. Отсюда же можно перейти к истории.

После запуска открывается экран прогресса. На нём видны номер текущего поколения, лучшее значение и график сходимости в реальном времени. По завершении приложение переходит на экран результата. Он показывает лучшее решение, числовые показатели, график сходимости за весь запуск и контурную карту пространства поиска. С него можно открыть трёхмерную поверхность, вернуться к настройке или перейти к истории. Завершённый эксперимент сохраняется в историю.

Экран истории содержит список прошлых экспериментов. Любую запись можно открыть заново, а отметив несколько — перейти к сравнению. На экране сравнения запуски выводятся вместе: на совмещённом графике, в таблице и на столбчатой диаграмме. Такое разделение даёт простой путь от постановки задачи до анализа результатов.

3 Технологическая часть

3.1 Обоснование выбора технологий

Архитектура, описанная в конструкторской части, не привязана к конкретным технологиям. В этом подразделе обосновывается выбор средств, которыми комплекс реализован: транспортного протокола, серверного и клиентского языков, модели параллельной обработки и формата обмена данными.

Транспортный протокол должен удовлетворять трём условиям. Сервер обязан слать данные клиенту сам, после каждого поколения, не дожидаясь запроса. Соединение должно оставаться открытым всё время работы задачи. Накладные расходы на сообщение должны быть малыми: прогресс идёт потоком, по одному сообщению на поколение. Не все решения отвечают этим условиям. Опрос сервера по HTTP прост, но неэффективен: большинство запросов возвращают пустой ответ, а интервал опроса задаёт задержку данных [20]. Однонаправленная потоковая передача событий снимает проблему опроса, но не даёт обратной связи: клиент не может по тому же соединению отправить задачу или отмену. Всем условиям отвечает WebSocket. Он держит постоянное двустороннее соединение поверх TCP, обе стороны шлют сообщения в любой момент, а заголовок кадра занимает лишь несколько байт [21]. Поэтому транспорт — WebSocket.

Серверная часть написана на Python. Этот язык стал стандартным в научных вычислениях во многом благодаря библиотеке NumPy [22] с её быстрыми операциями над массивами. Действия над всей популяцией — вычисление функции, сортировка, отбор доноров — выражаются через массивы NumPy и идут на уровне скомпилированного кода. Это намного быстрее циклов на чистом Python.

Сервер должен обслуживать несколько клиентов сразу, не останавливаясь на время чужих вычислений. Для этого применяется асинхронная модель `asyncio`: много соединений живут в одном потоке, и операция, ждущая данных по сети, уступает управление другим [23]. Но алгоритм оптимизации нагружен и не имеет точек ожидания. Запущенный прямо в цикле событий, он заблокировал бы остальные соединения. Поэтому алгоритм идёт в отдельном потоке из пула, а прогресс возвращается в цикл событий через потокобезопас-

ную очередь. Веб-часть построена на FastAPI: он поддерживает WebSocket и работает поверх той же модели [24].

Сообщения сериализуются в JSON. Формат поддерживается без дополнительных библиотек и в Python, и в Dart [25, 26], а текст удобно читать при отладке. Двоичные форматы компактнее, но выигрыш здесь невелик: сообщения о прогрессе содержат немного чисел, и их сериализация ничтожна на фоне вычислений [27]. Для оценки: финальная популяция 50×10 — это 500 чисел, около 5–7 КБ в JSON. По умолчанию в потоке идут лишь номер поколения и лучшее значение. Объёмные данные — популяция, пригодность, скорости — передаются только по запросу клиента.

Клиентская часть написана на Dart с фреймворком Flutter. Dart имеет строгую статическую типизацию и встроенную поддержку асинхронности [28]. Особенно полезна абстракция потока событий: сообщения по WebSocket естественно ложатся в такую последовательность, на которую приложение подписывается. Flutter — кроссплатформенный фреймворк, компилирующий приложение в нативный код со своим движком рендеринга [29]. Интерфейс описывается деревом виджетов, и при новых данных перестраивается только изменившаяся часть. Это и позволяет плавно обновлять график в реальном времени. Контурная карта и поверхность рисуются через низкоуровневый холст с доступом к графическим примитивам.

3.2 Реализация вычислительного ядра сервера

Вычислительное ядро отвечает за запуск алгоритмов. Оно состоит из трёх частей: реестра методов, набора целевых функций и связующего модуля запуска. Так новые алгоритмы и функции добавляются, не меняя остальной код.

Реестр методов — единая точка, где перечислены все алгоритмы. Каждому методу сопоставлена пара из класса алгоритма и класса конфигурации, а ключом служит имя метода из запроса клиента. Содержимое реестра показано в листинге 1.

Чтобы создать алгоритм, фабрика находит пару по имени метода, собирает конфигурацию из присланных параметров с размерностью и границами и возвращает готовый экземпляр. Если имени в реестре нет, фабрика сообщает об ошибке. Поэтому добавление алгоритма сводится к записи в реестр.

Листинг 1: Реестр методов оптимизации

```
1 METHODS = {  
2     "bbo.classic": (ClassicBBO, ClassicBBOConfig),  
3     "bbo.classic.modified": (ClassicBBO_Modified, BBOConfigModified),  
4     "bbo.sinusoidal": (BBO_Sinusoidal, SinBBOConfig),  
5     "bbo.sinusoidal.modified": (BBO_Sinusoidal_Modified,  
6         BBOConfigSinModified),  
7     "bbo.mixed": (BBO_Mixed, MixedBBOConfig),  
8     "bbo.mixed.modified": (BBO_Mixed_Modified, BBOConfigMixedModified),  
9     "pso": (PSO, PSOConfig),  
10 }
```

Целевая функция создаётся отдельно. Видов два. Встроенные функции — сфера, Растригина, Экли и Букина N.6 — выбираются по имени. Кроме того, пользователь может задать функцию выражением от координат. Выполнять выражение напрямую небезопасно: в нём может быть обращение к файлам или иной вредный код. Поэтому выражение не исполняется как код, а разбирается в синтаксическое дерево и проверяется. Разрешены только арифметика и функции из ограниченного набора: корень, экспонента, логарифм, тригонометрические и подобные. Любое постороннее имя или обращение к атрибуту отклоняет выражение. Только проверенное выражение компилируется в функцию. Ей доступны лишь координаты и разрешённые функции, а встроенные средства языка закрыты.

Связующий модуль выполняет один запуск от начала до конца. Он разбирает параметры задачи, строит границы, создаёт функцию и алгоритм и запускает оптимизацию. После каждого поколения алгоритм вызывает функцию обратного вызова. Через неё модуль отдаёт прогресс наружу и проверяет, не пришла ли отмена; если пришла — прерывает работу. По завершении результат готовится к передаче: массивы переводятся в списки, добавляются время счёта и имя метода.

3.3 Реализация очереди задач и обработчика

Сервер обслуживает несколько клиентов, а каждый запуск занимает заметное время. Поэтому задачи не выполняются сразу, а проходят через очередь. За постановку и извлечение отвечает очередь задач, за выполнение — обработчик.

Очередь построена на приоритетной очереди из `asyncio`. Её элемент — тройка из приоритета, порядкового номера постановки и идентификатора

задачи. Приоритетов три: высокий, обычный и низкий. При равных приоритетах задачи идут в порядке поступления — для этого и нужен порядковый номер. Сам объект задачи лежит в отдельном реестре, а в очереди — только идентификатор.

При постановке очередь решает ещё две задачи. Первая — защита от повторов. Клиент даёт каждому запросу уникальный идентификатор; при повторном приходе очередь находит совпадение и возвращает уже созданную задачу. Вторая — защита от переполнения: число ожидающих задач ограничено, и при достижении предела постановка отклоняется. По идентификатору также можно узнать позицию в очереди и время ожидания и отменить задачу — ожидающую или выполняемую.

Выполнением занимается обработчик. При запуске сервера создаётся пул обработчиков, их число задаётся в настройках. Каждый работает в цикле: берёт задачу, переводит её в выполнение, уведомляет клиента и запускает оптимизацию. Так сервер считает несколько задач сразу. Алгоритм нагружен и не имеет точек ожидания, поэтому в асинхронном цикле его запускать нельзя — он заблокировал бы соединения. Поэтому алгоритм идёт в отдельном потоке из пула, а цикл тем временем обслуживает других клиентов.

На выполнение задаётся ограничение по времени. Запуск обёрнут в ожидание с таймаутом: если срок вышел, вычисление прерывается, а задача переходит в состояние превышения времени. Так же обрабатываются успех, ошибка и отмена: каждому исходу соответствует свой переход и своё сообщение клиенту.

Отдельно стоит передача прогресса. Алгоритм идёт в рабочем потоке, а отправка — в асинхронном цикле, поэтому напрямую данные между ними не передать. Прогресс идёт через потокобезопасную очередь: после каждого поколения рабочий поток кладёт в неё данные, а отдельная задача в цикле их извлекает и шлёт клиенту. Поддерживаются режимы: сообщать о каждом поколении, о каждом k -м или не сообщать — так снижается плотность потока. По окончании в очередь кладётся признак конца, и отправка корректно останавливается.

Связь очереди с клиентами обеспечивает обработчик соединений. Для каждого клиента создаётся свой обработчик, ведущий диалог по протоколу. Сначала он принимает приветствие и сообщает о возможностях сервера:

доступные методы, виды целевых функций, режимы прогресса и предельный размер сообщения. Если первым пришло не приветствие, соединение отклоняется. После рукопожатия обработчик читает сообщения и направляет каждое по типу: запуск ставит задачу в очередь, отмена снимает её, запрос состояния возвращает сведения, проверка связи подтверждается ответом. Сообщения неизвестного типа или с ошибкой в данных не прерывают работу — в ответ идёт сообщение об ошибке. Когда клиент отключается, обработчик не отменяет его задачи сразу, а запускает отложенную отмену со сроком в 2 минуты. Если клиент успеет переподключиться и запросить состояние, задача привязывается к новому соединению, и отмена снимается. Иначе задача отменяется, чтобы не тратить ресурсы впустую.

Перед постановкой параметры задачи проверяются. Проверка охватывает структуру сообщения и значения: имя метода должно быть известным, размерность и размер популяции — положительными, границы — согласованными. Некорректный запрос отклоняется с понятным сообщением ещё до запуска вычислений.

3.4 Реализация алгоритмов оптимизации

Все семь методов реализованы по общей схеме. Каждый алгоритм — отдельный класс, получающий при создании конфигурацию, целевую функцию и функцию обратного вызова. Основная работа идёт в методе `optimize`: он инициализирует популяцию, проводит заданное число поколений и возвращает лучшее решение, его значение, историю сходимости и финальную популяцию. Различаются методы только тем, что происходит внутри одного поколения.

Параметры метода описаны классом конфигурации. В нём заданы значения по умолчанию и проверки: размер популяции не меньше двух, вероятность мутации в отрезке от нуля до единицы, число элитных решений меньше размера популяции. Незаданные границы заполняются по умолчанию. Проверки идут при создании конфигурации, поэтому неверные параметры отсекаются до запуска. Случайность всех методов исходит из одного генератора с заданным `seed` — это делает запуски воспроизводимыми.

Рассмотрим классический ВВО. Одно поколение показано в листинге 2. Сначала популяция сортируется по значению функции, затем по рангам счи-

таются скорости эмиграции и иммиграции. Для каждого неэлитного решения идёт покомпонентная иммиграция: компонента заменяется значением донора с вероятностью, равной скорости иммиграции, а донор выбирается с вероятностью, пропорциональной скорости эмиграции. После миграции применяется мутация, а в конце поколения лучшие решения возвращаются вместо худших потомков.

Листинг 2: Одно поколение классического ВВО

```

1 order = np.argsort(fitness)
2 X, fitness = X[order], fitness[order]
3 best_f, best_x = float(fitness[0]), X[0].copy()
4
5 lambdas, mus = self._rank_based_migration_rates(self.cfg.pop_size)
6 donor_probs = lambdas[:, -1] / lambdas[:, -1].sum()
7 donor_cdf = np.cumsum(donor_probs)
8
9 Z = X.copy()
10 for k in range(self.cfg.elite_count, self.cfg.pop_size):
11     imm_flags = self.rng.random(self.cfg.dims) < mus[:, -1][k]
12     for s in np.where(imm_flags)[0]:
13         j = self._roulette_select(donor_cdf)
14         Z[k, s] = X[j, s]
15     mut_flags = self.rng.random(self.cfg.dims) < self.cfg.mutation_rate
16     Z[k, mut_flags] = self.low[mut_flags] + self.rng.random(mut_flags.
17         sum()) * self.span[mut_flags]
18
19 fit_Z = self._evaluate_population(self._clip(Z))
20 worst = np.argsort(fit_Z)[-self.cfg.elite_count:]
21 Z[worst], fit_Z[worst] = X[:self.cfg.elite_count], fitness[:self.cfg.
    elite_count]
22 X, fitness = Z, fit_Z

```

Варианты с синусоидальной и смешанной миграцией построены по тому же циклу. Отличие — только в способе вычисления скоростей λ_r и μ_r . Остальное совпадает с классическим.

Модифицированные версии устроены сложнее. К базовому циклу добавлены три механизма: адаптивная мутация, частичный рестарт и локальный поиск. Структура поколения показана в листинге 3. Сначала вычисляется текущая вероятность мутации, линейно убывающая от начального значения к конечному. Затем проверяется счётчик стагнации: если лучшее решение долго не улучшалось, идёт частичный рестарт, а вероятность мутации временно повышается. После этого — миграция, мутация, элитарность и при необхо-

димости локальный поиск. В конце счётчик стагнации обновляется по тому, улучшилось ли решение.

Листинг 3: Одно поколение модифицированного ВВО

```
1 mutation_rate = self._current_mutation_rate(gen)
2 if stagnation >= self.cfg.stagnation_generations:
3     X, fit = self._restart_partially(X, fit)
4     mutation_rate = min(self.cfg.mutation_start,
5                          mutation_rate * self.cfg.mutation_boost_factor)
6     stagnation = 0
7
8 X_new = self._migration_step(X, fit)
9 X_new = self._mutation_step(X_new, mutation_rate)
10 fit_new = self._evaluate(X_new)
11 X, fit = self._apply_elitism(X, fit, X_new, fit_new)
12
13 if self.cfg.enable_local_search and (gen + 1) % self.cfg.
14     local_search_interval == 0:
15     X, fit = self._local_search(X, fit, gen)
16
17 cur_best = float(fit[np.argmin(fit)])
18 if cur_best < best_f:
19     best_f, stagnation = cur_best, 0
20 else:
21     stagnation += 1
```

Частичный рестарт — это замена доли худших решений новыми случайными. Число заменяемых задаётся долей от размера популяции, элитные решения защищены. Локальный поиск уточняет несколько лучших решений: к каждому добавляется случайное гауссовское возмущение, масштаб которого пропорционален ширине области и убывает с номером поколения. Лучшая точка принимается, иначе отвергается. Так точность растёт вблизи найденных минимумов, не затрагивая остальную популяцию.

Метод роя частиц реализован матрично: вся популяция обновляется группой операций над массивами, без цикла по частицам. Одна итерация показана в листинге 4. Скорости пересчитываются сразу: к инерционной части добавляются когнитивная, направленная к личному лучшему, и социальная, направленная к глобально лучшему. Затем скорости ограничиваются по модулю, положения сдвигаются и отсекаются по границам, после чего обновляются личные и глобально лучшее решения.

Листинг 4: Одна итерация метода роя частиц

```
1 w = self._inertia_weight(t)
2 r1, r2 = self.rng.random((N, n)), self.rng.random((N, n))
3
4 cognitive = self.cfg.c1 * r1 * (pbest_pos - X)
5 social     = self.cfg.c2 * r2 * (gbest_pos - X)
6 V = w * V + cognitive + social
7 V = np.clip(V, -self.vmax, self.vmax)
8
9 X = np.clip(X + V, self.low, self.high)
10 fit = self._evaluate(X)
11
12 improved = fit < pbest_fit
13 pbest_pos[improved], pbest_fit[improved] = X[improved], fit[improved]
14 g_idx = int(np.argmin(pbest_fit))
15 if pbest_fit[g_idx] < gbest_fit:
16     gbest_fit, gbest_pos = float(pbest_fit[g_idx]), pbest_pos[g_idx].
        copy()
```

Использование массивов NumPy здесь принципиально. Оценка функции для всей популяции, сортировка, отбор и пересчёт скоростей идут как операции над матрицами, а не как циклы на чистом Python. Для PSO это убирает цикл по частицам полностью. В ВВО покомпонентная иммиграция всё же требует перебора, ведь донор для каждой компоненты выбирается отдельно, но оценка и мутация остаются векторными. Вид функции учитывается: сначала она вызывается для всей матрицы решений, и лишь если так не вышло — например, для поточечного пользовательского выражения, — идёт поэлементный расчёт. После каждого поколения методы вызывают функцию обратного вызова с номером поколения и лучшим значением — эти данные и уходят клиенту.

3.5 Реализация мобильного клиента

Мобильный клиент построен по слоям. Нижний отвечает за связь по протоколу, средний хранит состояние экранов и обрабатывает события, верхний отображает интерфейс. Дополнительно выделены слой моделей данных и слой хранения истории. Так одну часть можно менять, не трогая остальные.

3.5.1 Клиент протокола

Связь с сервером заключена в отдельном классе. Он скрывает детали WebSocket и даёт приложению единый поток событий. Сообщения сервера

разбираются в типизированные события и публикуются в широковещательный поток, на который подписываются нужные части приложения. Приём и маршрутизация показаны в листинге 5: строка разбирается в конверт, из него берутся тип и нагрузка, и по типу создаётся событие.

Листинг 5: Приём и маршрутизация сообщений на клиенте

```
1 void _onMessage(String raw) {
2   final envelope = parseEnvelope(raw);
3   final type = getType(envelope);
4   final payload = getPayload(envelope);
5
6   switch (type) {
7     case MessageTypes.jobAccepted:
8       _eventController.add(WsJobAccepted(JobAccepted.fromPayload(
9         payload)));
10    case MessageTypes.jobStarted:
11      _eventController.add(WsJobStarted(JobStarted.fromPayload(payload)
12        ));
13    case MessageTypes.jobProgress:
14      _eventController.add(WsJobProgress(JobProgress.fromPayload(
15        payload)));
16    case MessageTypes.jobResult:
17      _eventController.add(WsJobResult(JobResult.fromPayload(payload)))
18      ;
19    case MessageTypes.jobFinished:
20      _eventController.add(WsJobFinished(JobFinished.fromPayload(
21        payload)));
22    case MessageTypes.error:
23      _eventController.add(WsError(ProtocolError.fromPayload(payload)))
24      ;
25  }
26 }
```

Клиент устойчив к обрывам связи. При неожиданном закрытии он восстанавливает соединение сам, и задержка между попытками растёт от одной секунды до тридцати, чтобы не нагружать сеть. Пока связь есть, клиент периодически шлёт проверку. При подключении он отправляет приветствие с запросом потока прогресса. Намеренное отключение помечается особым признаком — тогда восстановление не запускается. Кроме приёма событий клиент даёт методы для отправки задачи, запроса состояния и отмены.

3.5.2 Модели данных и конверт сообщения

Сообщения разбираются и собираются в отдельном слое моделей. Каждому типу сообщения соответствует свой класс с методом разбора нагрузки, поэтому остальной код работает с типизированными объектами, а не со словарями. Исходящие сообщения строит общая функция конверта. Она добавляет служебные поля заголовка: версию протокола, тип, идентификатор сообщения, метку времени и поля для связи сообщений и задачи. Функция показана в листинге 6.

Листинг 6: Построение конверта исходящего сообщения

```
1 Map<String, dynamic> buildEnvelope({
2   required String type,
3   required Map<String, dynamic> payload,
4   String? clientReqId,
5   String? jobId,
6 }) {
7   return {
8     'header': {
9       'v': 1,
10      'type': type,
11      'msg_id': _uuid.v4(),
12      'ts': DateTime.now().toUtc().toIso8601String(),
13      'trace': {'client_req_id': clientReqId, 'job_id': jobId},
14    },
15    'payload': payload,
16  };
17 }
```

3.5.3 Сборка конфигурации задачи

Параметры эксперимента с экрана настройки собираются из нескольких частей: целевой функции, постановки задачи, метода, режима потоковой передачи и состава выходных данных. Описание функции хранит её вид — встроенная или заданная выражением — и соответствующие поля. Постановка задаёт размерность и границы области, единые для всех координат или свои для каждой. Параметры метода зависят от алгоритма: у ВВО это размер популяции и вероятность мутации, у роя частиц — коэффициенты. Каждая часть умеет переводить себя в представление для передачи, а перед отправкой части объединяются в единый запрос.

3.5.4 Управление состоянием

Средний слой построен на однонаправленном потоке данных. Каждому экрану соответствует объект состояния. Он подписан на поток событий клиента, обрабатывает их и порождает новое неизменяемое состояние, на которое реагирует интерфейс. Таких объектов несколько: подключение, сборка конфигурации, ход задачи и работа с историей. Показателем объекта экрана прогресса. При создании он отправляет задачу и подписывается на события, а получая прогресс, инкрементально накапливает историю лучших значений, как в листинге 7. Новые значения дописываются в конец, и график достраивается в реальном времени, не перестраиваясь.

Листинг 7: Накопление истории прогресса

```
1 case WsJobProgress(:final data):
2   final newHistory = List<double>.from(state.historyBestF);
3   if (data.progress.historyBestFTail != null) {
4     for (final v in data.progress.historyBestFTail!) {
5       if (newHistory.isEmpty || newHistory.last != v) newHistory.add(v)
6     }
7   } else {
8     newHistory.add(data.progress.bestF);
9   }
10  emit(state.copyWith(
11    phase: JobPhase.running,
12    iteration: data.progress.iteration,
13    bestF: data.progress.bestF,
14    historyBestF: newHistory,
15  ));
```

Этот же объект следит за фазой задачи: ожидание в очереди, начало, ход вычисления и завершение. При сообщении о завершении фаза переходит в одно из конечных состояний — успех, ошибка, отмена или превышение времени, — и интерфейс меняет вид. Отдельно копится история лучших решений в пространстве: по ней строится траектория на трёхмерной визуализации.

3.5.5 Локальное хранение истории

История завершённых экспериментов хранится на устройстве в локальной базе данных. Результат запуска сериализуется в текст и сохраняется отдельной записью с метаданными: методом, функцией, размерностью, итоговым значением и временем. При открытии истории записи читаются обратно

в типизированные объекты, а метаданные позволяют отбирать и фильтровать запуски. Хранение на устройстве делает историю доступной и без подключения к серверу.

3.6 Пользовательский интерфейс приложения

В этом подразделе показано готовое приложение. Интерфейс выдержан в едином стиле: общая цветовая гамма, крупные элементы под сенсорный экран и привычные компоненты — выпадающие списки, переключатели и ползунки. Экраны идут в том порядке, в каком их проходит пользователь.

3.6.1 Подключение к серверу

С этого экрана начинается работа. Пользователь вводит адрес и порт сервера и нажимает кнопку подключения; под ней показано состояние соединения. Адрес последнего сервера сохраняется, поэтому вводить его заново не нужно. Экран показан на рисунке 10.

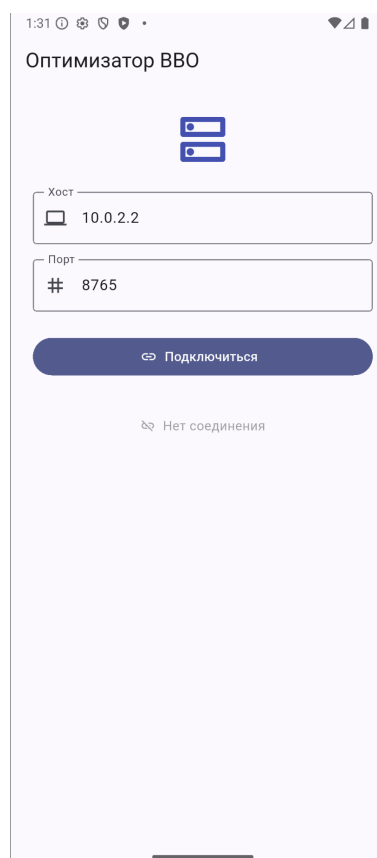


Рисунок 10 – Экран подключения к серверу

3.6.2 Настройка численного эксперимента

После подключения открывается основной экран настройки. Он разбит на блоки: целевая функция, постановка задачи, метод, режим потоковой передачи и состав выходных данных. В блоке функции выбирается встроенная функция или вводится выражение, а переключатель задаёт минимизацию или максимизацию. В блоке задачи ползунком задаётся размерность, ниже — границы области. В блоке метода выбирается алгоритм и его параметры: размер популяции, число поколений, вероятность мутации и другие. Настроек много, поэтому экран прокручивается; внизу — кнопка запуска. Вид экрана показан на рисунке 11.

Настройка задачи

Целевая функция

Тип: Встроенная

Функция: sphere

Минимизация: ☒

Пространство поиска

Размерность: 10

Тип границ: Одинаковые

Мин.: -5.12

Макс.: 5.12

Метод оптимизации

Метод: BBO Classic

Размер популяции: 50

Макс. поколений: 200

Частота мутации: 0.01

Элитных особей:

а)

Настройка задачи

Элитных особей: 2

Запрет самомиграции: ☒

Seed (необязательно):

Потоковая передача

Режим: Каждое поколение

Включить best_x: ☒

Включить best_f: ☒

Включить популяцию: ☐

Включить fitness: ☐

Включить скорости: ☐

Выходные данные

Вернуть финальную популяцию: ☒

Вернуть финальный fitness: ☒

Вернуть финальные скорости: ☐

▶ Запустить оптимизацию

б)

Рисунок 11 – Экран настройки эксперимента:

а) – выбор функции и задачи;

б) – параметры метода и потоковой передачи

3.6.3 Выполнение и наблюдение за прогрессом

После запуска приложение переходит на экран выполнения. Вверху — индикатор прогресса с номером поколения и долей выполнения, ниже — лучшее значение и время. Основную часть занимает график сходимости, который достраивается в реальном времени. Внизу — кнопка отмены. Экран показан на рисунке 12.

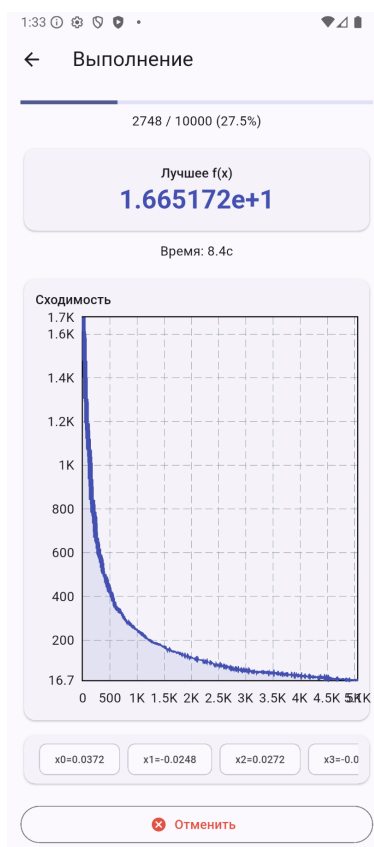
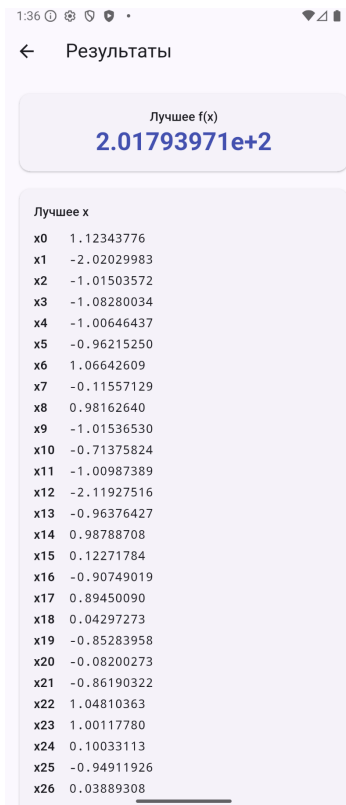


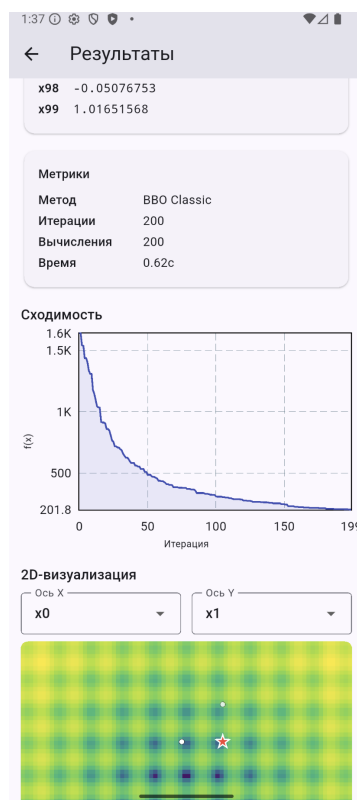
Рисунок 12 – Экран выполнения с графиком сходимости в реальном времени

3.6.4 Просмотр результата

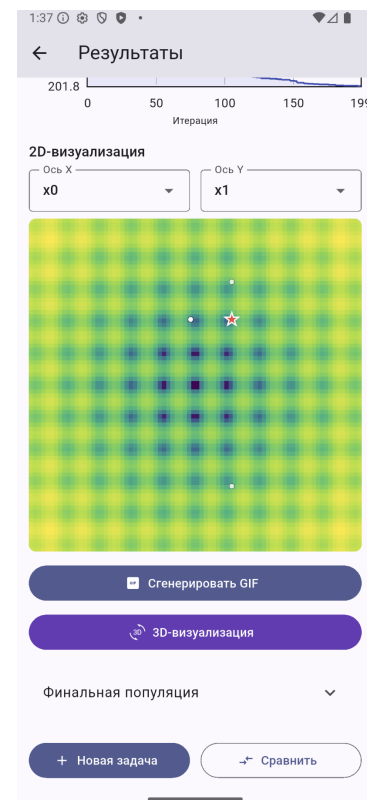
По завершении вычисления открывается экран результата. Вверху выводится лучшее найденное значение целевой функции, ниже — координаты лучшего решения и сводка показателей: метод, число поколений и вычислений функции, затраченное время. Далее расположен график сходимости за весь запуск, а под ним — визуализация пространства поиска в виде контурной карты, на которой отмечены лучшее решение и ближайшие к нему особи. Здесь же находятся кнопки построения анимации и перехода к трёхмерной визуализации. Вид экрана показан на рисунке 13.



а)



б)



в)

Рисунок 13 – Экран результата:
а) – лучшее решение;
б) – показатели и график сходимости;
в) – контурная карта пространства поиска

По нажатию кнопки строится анимация: контурная карта, на которой популяция смещается от поколения к поколению. Готовая анимация показывается прямо на экране, как на рисунке 14.

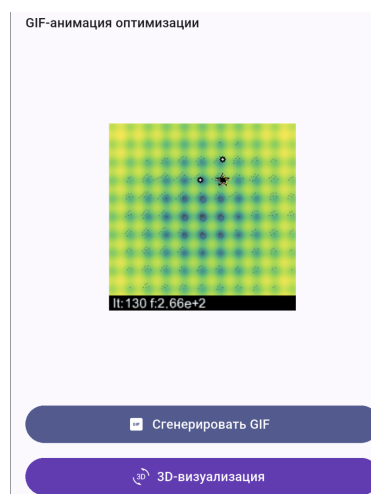


Рисунок 14 – Анимация процесса оптимизации на экране результата

3.6.5 Трёхмерная визуализация

Отдельный экран показывает функцию трёхмерной поверхностью. Высота точки — это значение функции, на поверхности отмечены лучшее решение и путь его улучшения. Поверхность вращается движением пальца, а выпадающие списки сверху задают, какие две координаты отложить по осям. Экран показан на рисунке 15.

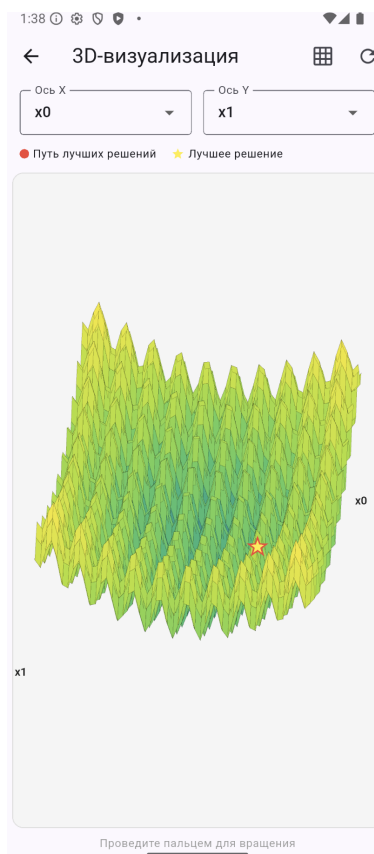
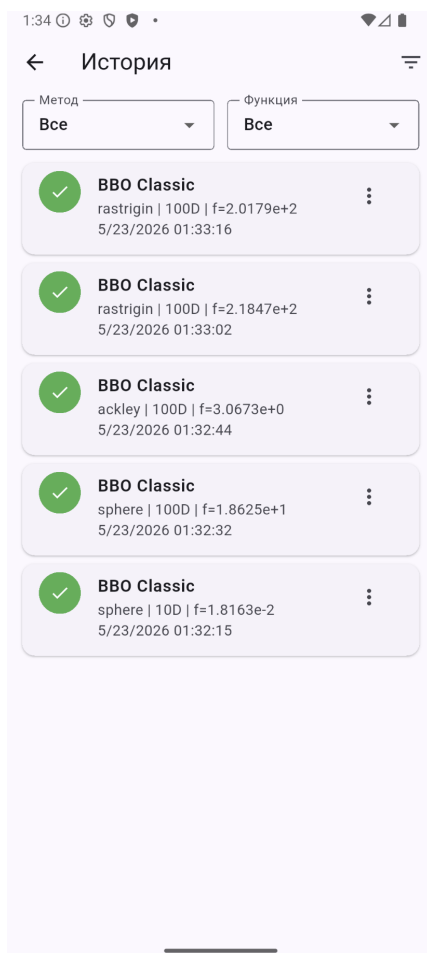


Рисунок 15 – Экран трёхмерной визуализации целевой функции

3.6.6 История и сравнение запусков

Все завершённые эксперименты попадают в историю. На её экране они показаны списком, где для каждого запуска указаны метод, целевая функция, размерность, итоговое значение и время проведения; список можно фильтровать по методу и функции. Отметив несколько запусков, пользователь переходит к их сравнению. На экране сравнения выводятся совмещённый график сходимости выбранных запусков, таблица их показателей и столбчатая диаграмма лучших значений. Оба экрана показаны на рисунке 16.



а)



б)

Рисунок 16 – Экраны истории и сравнения запусков:

а) – экран истории экспериментов;

б) – экран сравнения запусков

3.7 Реализация визуализации результатов

Визуализация выполнена на клиенте. График сходимости строится готовым средством, а контурная карта, поверхность и кадры анимации рисуются вручную через низкоуровневый холст. Ниже рассмотрен каждый вид.

3.7.1 График сходимости

График сходимости показывает, как убывает лучшее значение от поколения к поколению. Он строится готовым средством, которому передаётся список накопленных значений. На экране выполнения список пополняется по мере прихода прогресса, и график достраивается в реальном времени. Если значения убывают на несколько порядков, по оси ординат берётся логарифмическая шкала, иначе нижняя часть кривой сжимается у нуля. На экранах

результата и сравнения тот же график строится по полной истории, а при сравнении на одни оси ложится несколько кривых разного цвета.

3.7.2 Контурная карта

Контурная карта строится вручную. Область поиска разбивается на равномерную сетку, в каждой ячейке вычисляется значение функции, которое через логарифмическую нормировку переводится в цвет, и ячейка закрашивается прямоугольником. Нормировка делает различимыми и глубокие минимумы, и пологие участки. Поверх наносятся положения особей и отдельно лучшее решение. Основная часть рисования — в листинге 8.

Листинг 8: Построение контурной карты

```
1 for (int j = 0; j < resolution; j++) {
2   for (int i = 0; i < resolution; i++) {
3     final val = grid.values[j][i];
4     final t = range > 0
5       ? (math.log(val - minVal + 1) / math.log(range + 1)).clamp(0.0,
6         1.0)
7       : 0.0;
8     paint.color = ColorMaps.viridis(t);
9     canvas.drawRect(
10      Rect.fromLTWH(i * cellW, j * cellH, cellW + 0.5, cellH + 0.5),
11      paint,
12    );
13  }
```

Карта требует вычислить функцию во всех узлах, поэтому считается один раз при открытии экрана, а при перерисовке берутся готовые значения. При размерности больше двух по осям откладываются две выбранные координаты, остальные фиксируются — это даёт сечение функции в выбранной плоскости.

3.7.3 Трёхмерная поверхность

Трёхмерная поверхность строится по той же сетке. Узлы образуют четырёхугольные грани, которые проектируются на экран по формулам поворота: координаты узла нормируются к единому диапазону и поворачиваются вокруг вертикальной и горизонтальной осей, после чего две координаты дают точку на экране. Проекция показана в листинге 9.

Листинг 9: Проекция точки поверхности на экран

```

1 Offset project(double px, double py, double pz) {
2     final nx = 2 * (px - xMin) / (xMax - xMin) - 1;
3     final ny = 2 * (py - yMin) / (yMax - yMin) - 1;
4     final nz = 2 * (pz - fMin) / fRange - 1;
5
6     final rx = nx * cosZ - ny * sinZ;           // rotate around Z
7     final ry = nx * sinZ + ny * cosZ;
8     final ry2 = ry * cosX - nz * sinX;         // rotate around X
9
10    return Offset(cx + rx * scale, cy - ry2 * scale);
11 }

```

Чтобы ближние грани перекрывали дальние, применяется алгоритм художника: грани сортируются по глубине от дальних к ближним и рисуются в этом порядке. Цвет грани задаётся средним значением функции в её узлах по той же шкале, что и карта. Поверх наносится путь лучшего решения — последовательность точек за время поиска; чтобы не рисовать тысячи точек, путь прореживается. Углы поворота меняются жестом, и при каждом изменении поверхность проектируется заново.

3.7.4 Анимация процесса оптимизации

Анимация показывает, как популяция смещалась от поколения к поколению. Она собирается кадром в формат GIF. Сначала строится фоновый кадр с контурной картой целевой функции, затем для каждого кадра поверх фона наносятся положения особей и лучшее решение, а также служебная строка с номером поколения и текущим лучшим значением. Число кадров фиксировано, поэтому из всей истории берётся равномерная выборка поколений. Готовые кадры собираются в один файл с заданной частотой смены, причём последний кадр показывается дольше остальных, чтобы итоговое расположение популяции было видно.

Готовая анимация показывается прямо на экране результата и может быть сохранена как обычный файл изображения. Поскольку формат GIF не требует внешних средств воспроизведения и собирается из готовых кадров, он удобен для этой задачи.

4 Аналитическая часть

4.1 Тестирование программного обеспечения

Правильность работы комплекса проверялась двумя способами. Серверная часть с вычислительным ядром и логикой обработки задач покрыта автоматическими тестами. Клиентская часть, в основе которой интерфейс, проверялась вручную по сценариям. Логiku сервера удобно проверять программно, а работу интерфейса — наблюдением на устройстве.

4.1.1 Автоматическое тестирование серверной части

Серверная часть протестирована библиотекой `pytest` [30]. Написано 73 теста по группам модулей; они охватывают и отдельные функции, и их взаимодействие:

1. Тесты очереди задач. Проверялись постановка с учётом приоритетов, извлечение в правильном порядке, отмена ожидающих и выполняемых задач, защита от повторов и переполнения. 14 тестов.
2. Тесты целевых функций. Проверялись вычисление встроенных функций в известных точках, разбор и вычисление пользовательских выражений, отклонение небезопасных и некорректных. 15 тестов.
3. Тесты протокола обмена. Проверялись построение и разбор конверта, сохранение полей заголовка и обработка всех типов сообщений. 11 тестов.
4. Тесты реестра методов. Проверялись регистрация всех семи методов, создание алгоритма по имени и ошибка при неизвестном методе. 4 теста.
5. Тесты модуля запуска. Проверялся полный прогон: запуск алгоритма, передача прогресса, прерывание по отмене и подготовка результата к передаче. 7 тестов.
6. Тесты валидации. Проверялось, что корректные параметры проходят, а некорректные — неизвестный метод, неположительная размерность, несогласованные границы — отклоняются. 14 тестов.
7. Тесты соединения. Проверялась работа обработчика отдельно и в связке с очередью: рукопожатие, маршрутизация, постановка задачи и ответы. 8 тестов.

Для примера в листинге 10 приведён тест, проверяющий, что в реестре зарегистрированы все семь методов.

Листинг 10: Пример теста: проверка состава реестра методов

```
1 def test_all_methods_registered(self):
2     expected = {
3         "bbo.classic", "bbo.classic.modified",
4         "bbo.sinusoidal", "bbo.sinusoidal.modified",
5         "bbo.mixed", "bbo.mixed.modified",
6         "pso",
7     }
8     assert set(METHODS.keys()) == expected
```

Все 73 теста проходят успешно. Это подтверждает, что вычислительное ядро, очередь, протокол и логика обработки задач работают как задумано, а изменения в коде не ломают уже проверенное поведение.

4.1.2 Функциональное тестирование клиента

Клиентская часть проверялась вручную: на устройстве последовательно выполнялись основные сценарии использования, и поведение приложения сравнивалось с ожидаемым. Тестирование охватывало все экраны и основные действия пользователя.

1. Подключение к серверу. Приложение устанавливает соединение по адресу и порту, показывает его состояние и восстанавливает при кратком обрыве. При неверном адресе сообщает об ошибке и не зависает.
2. Настройка и запуск задачи. На экране настройки можно выбрать метод и функцию, задать размерность, границы и параметры, ввести своё выражение. После запуска открывается экран выполнения, а недопустимые значения не дают начать вычисление.
3. Наблюдение за прогрессом. Во время вычисления видны номер поколения и лучшее значение, график достраивается в реальном времени. Кнопка отмены прерывает вычисление, и приложение корректно обрабатывает это.
4. Просмотр результата. После завершения видны лучшее решение, показатели и график; строятся контурная карта, поверхность и анимация. Поверхность вращается жестом, а выбор координат меняет сечение.
5. Работа с историей. Эксперименты сохраняются, список фильтруется по методу и функции, любую запись можно открыть заново. История сохраняется между запусками.

6. Сравнение запусков. Несколько выбранных запусков выводятся вместе на графике, в таблице и на диаграмме, и сравнение остаётся читаемым при разных методах и функциях.

Все сценарии отработали как ожидалось. Вместе с автоматическими тестами сервера это позволяет считать комплекс работоспособным и готовым к экспериментам.

4.2 Вычислительные эксперименты и анализ результатов

Главная цель экспериментов — сравнить методы между собой и оценить влияние модификаций базового алгоритма. Для этого все семь методов запускались на наборе тестовых функций при разных размерностях.

4.2.1 Методика экспериментов

Использовались четыре функции из подраздела 1.1: сфера, Растригина, Экли и Букина N.6. Сфера имеет единственный минимум и проверяет скорость сходимости. Растригина и Экли содержат множество локальных минимумов и проверяют, не застревает ли метод. Букина N.6 определена только для двух переменных. Сфера, Растригина и Экли запускались в размерностях 2, 10, 30 и 50; вместе с двумерной Букина N.6 это тринадцать задач.

На каждой задаче метод запускался несколько раз с разными seed, а результаты усреднялись. Параметры подбирались предварительно. Качество оценивалось средним и медианным лучшими значениями, их разбросом, наилучшим значением по всем запускам и средним временем счёта. Минимум всех функций равен нулю, поэтому меньшее значение означает более точный результат.

4.2.2 Сравнение методов на функции с множеством минимумов

Наиболее показательна функция Растригина: у неё много локальных минимумов, и разница между методами хорошо видна. Средние лучшие значения при разных размерностях приведены в таблице 1.

Таблица 1 – Среднее лучшее значение на функции Растригина

d	ВВО кл.	ВВО кл. мод.	ВВО син. мод.	ВВО см. мод.	PSO
2	0,159	0,019	0,007	0,010	0
10	1,47	0,67	0,65	0,91	5,72
30	14,40	8,08	8,47	8,27	42,50
50	36,0	30,1	28,5	25,6	95,0

При размерности 2 лучший результат у PSO — он находит минимум точно. С ростом размерности картина меняется: начиная с десяти переменных все варианты ВВО заметно опережают PSO, и отрыв растёт. При пятидесяти переменных лучшие варианты ВВО дают около 25–30, а PSO — около 95. Причина в устройстве методов: рой частиц быстро стягивается к одной точке и застревает в одном из минимумов, тогда как покомпонентный обмен в ВВО дольше хранит разнообразие популяции. Та же закономерность показана на рисунке 17.

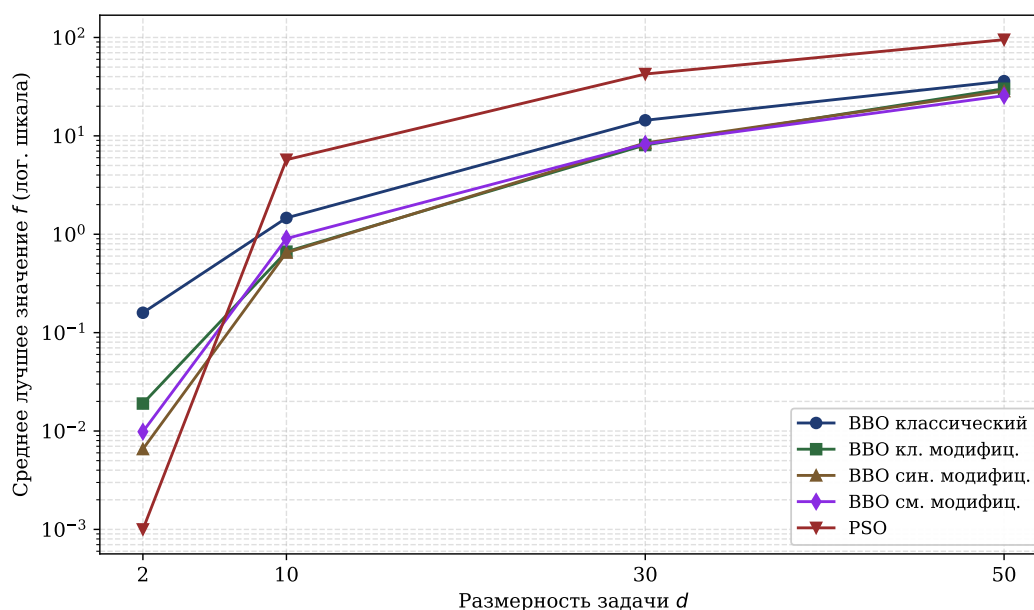


Рисунок 17 – Зависимость среднего лучшего значения на функции Растригина от размерности задачи

Сравнение базовых и модифицированных вариантов показывает, что модификации почти всегда улучшают результат. На Растригина любой модифицированный вариант точнее классического во всех размерностях, а в среднем по всем тринадцати задачам модификация не хуже базового вари-

анта в двенадцати-тринадцати случаях из тринадцати. Значит, адаптивная мутация, рестарт и локальный поиск действительно помогают.

4.2.3 Сравнение методов на разных функциях

Для общей картины в таблице 2 приведены средние лучшие значения на трёх функциях при размерности 30.

Таблица 2 – Среднее лучшее значение на разных функциях при $d = 30$

Метод	Сфера	Растригина	Экли
ВВО классический	0,23	14,40	3,69
ВВО кл. модифиц.	0,18	8,08	3,35
ВВО син. модифиц.	0,13	8,47	2,67
ВВО см. модифиц.	0,15	8,27	3,32
PSO	0	42,50	0,29

Таблица подтверждает, что у методов разные сильные стороны. На сфере и Экли с их более гладким рельефом PSO заметно точнее: на сфере — на несколько порядков. А на Растригина PSO, наоборот, уступает ВВО в несколько раз. Среди вариантов ВВО различия между моделями миграции невелики — значителен вклад модификаций, а не моделей миграции.

За точность приходится платить временем. На Растригина при размерности 30 один запуск классического ВВО занимает около 0,65 секунды, а модифицированного — около 5 секунд: основное время уходит на локальный поиск. PSO самый быстрый — около 0,02 секунды, ведь его итерация — это простые операции над массивами. Значит, выбор метода — это выбор между скоростью и точностью на сложном рельефе.

4.2.4 Выводы по результатам

PSO предпочтителен на задачах с простым рельефом и невысокой размерностью, где важна скорость. На задачах с множеством локальных минимумов и высокой размерностью лучше работают варианты ВВО, причём влияют механизмы адаптации: адаптивная мутация, рестарт и локальный поиск стабильно улучшают результат. Различия между моделями миграции невелики. Эти результаты согласуются с теоретическими представлениями о поведении методов.

ЗАКЛЮЧЕНИЕ

В ходе работы разработан программный комплекс для исследования и визуализации модификаций алгоритма биогеографической оптимизации.

Реализовано серверное вычислительное ядро из семи методов: классический ВВО, его варианты с синусоидальной и смешанной миграцией, три их модификации с адаптивной мутацией, рестартом и локальным поиском, а также метод роя частиц. Алгоритмы запускаются на четырёх встроенных функциях и на функции, заданной выражением; выражение перед вычислением проверяется на безопасность.

Разработан протокол поверх WebSocket. Он поддерживает постановку задачи, передачу прогресса в реальном времени, выдачу результата и отмену, а очередь с приоритетами и пул обработчиков позволяют обслуживать несколько клиентов сразу.

Создано мобильное приложение: через него настраивается эксперимент, отправляется задача и проводится наблюдение за вычислением. Оно построено по слоям, а связь с сервером и хранение истории вынесены в отдельные слои.

Реализованы средства визуализации: график сходимости в реальном времени, контурная карта и трёхмерная поверхность с вращением, а также анимация популяции. Это позволяет увидеть как итог, так и сам процесс поиска.

Организовано хранение истории на устройстве и сравнение запусков. Сохранённые результаты сопоставляются на совмещённом графике, в таблице и на диаграмме — в этом и состоит назначение комплекса.

Также были проведены эксперименты: семь методов сравнивались на тринадцати задачах разной структуры и размерности. Рой частиц точнее на простом рельефе и низкой размерности, а на функциях с множеством минимумов и высокой размерностью выигрывают варианты ВВО. Механизмы адаптации почти всегда улучшают базовый алгоритм ценой времени, а различия между моделями миграции невелики.

Комплекс допускает дальнейшее развитие. Семейство методов расширяется другими метаэвристиками через реестр, без изменения остального кода. Набор функций можно дополнить, а заложенные в протокол разбиение сообщений и сжатие — задействовать при больших объёмах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Карпенко А. П.* Современные алгоритмы поисковой оптимизации. Алгоритмы, вдохновлённые природой : учебное пособие. — 3-е изд. — Москва : Изд-во МГТУ им. Н. Э. Баумана, 2021. — 446 с.
2. *Гладков Л. А., Курейчик В. В., Курейчик В. М.* Генетические алгоритмы. — Москва : Физматлит, 2010. — 368 с.
3. *Пантелеев А. В., Метлицкая Д. В., Алешина Е. А.* Методы глобальной оптимизации. Метаэвристические стратегии и алгоритмы. — Москва : Вузовская книга, 2013. — С. 244.
4. *Саймон Д.* Алгоритмы эволюционной оптимизации / пер. А. В. Логунов. — Москва : ДМК Пресс, 2020. — 1002 с.
5. *Карпенко А. П.* Эволюционные операторы популяционных алгоритмов глобальной оптимизации. Опыт систематизации // Математика и математическое моделирование. — 2018. — № 1. — С. 59—89.
6. *Родзин С. И., Скобцов Ю. А., Эль-Хатиб С. А.* Биоэвристики: теория, алгоритмы и приложения. — Чебоксары : Изд-во «Среда», 2019. — 224 с.
7. *Kennedy J., Eberhart R.* Particle swarm optimization // Proceedings of ICNN'95 — International Conference on Neural Networks. Т. 4. — 1995. — С. 1942—1948.
8. *Карпенко А. П., Щербакова Н. О., Буланов В. А.* Гибридный алгоритм глобальной оптимизации на основе алгоритмов искусственной иммунной системы и метода роя частиц // Наука и образование (электрон. журн. МГТУ им. Н. Э. Баумана). — 2014. — № 3. — С. 255—271.
9. *Казакова Е. М.* Краткий обзор методов оптимизации на основе роя частиц // Вестник КРАУНЦ. Физико-математические науки. — 2022. — Т. 19, № 2. — С. 150—174.
10. *Карпенко А. П., Синяговская О. А.* Глобальная оптимизация методом биогеографии // Наука и образование (электрон. журн. МГТУ им. Н. Э. Баумана). — 2013. — № 10. — С. 373—398.

11. К вопросу эффективности методов и алгоритмов решения оптимизационных задач с учётом специфики целевой функции / Е. Н. Остроух [и др.] // Вестник Донского государственного технического университета. — 2019. — Т. 19, № 1. — С. 81—85.
12. Новикова Е. С., Бекенева Я. А., Бестужев М. П. Методы визуального анализа для мониторинга данных от сенсорных сетей // Известия СПбГ-ЭТУ «ЛЭТИ». — 2020. — № 8/9. — С. 52—60.
13. CompuServe Incorporated. GIF89a Graphics Interchange Format Version 89a. — 1990. — 34 с.
14. Chacón Sartori C., Blum C., Ochoa G. STNWeb: A new visualization tool for analyzing optimization algorithms // Software Impacts. — 2023. — Т. 17. — С. 100558.
15. Acerbi L. OptimViz: Visualize optimization algorithms in MATLAB. — URL: <https://github.com/lacerbi/optimviz> (дата обр. 21.05.2026).
16. Dupont E. Interactive Visualization of Optimization Algorithms in Deep Learning. — URL: <https://emiliendupont.github.io/2018/01/24/optimization-visualization/> (дата обр. 21.05.2026).
17. Schäpermeier L., Grimme C., Kerschke P. To Boldly Show What No One Has Seen Before: A Dashboard for Visualizing Multi-objective Landscapes // Evolutionary Multi-Criterion Optimization. Lecture Notes in Computer Science. — 2021. — Т. 12654. — С. 632—644.
18. RFC 6455: The WebSocket Protocol : RFC Editor. — 2011. — URL: <https://www.rfc-editor.org/rfc/rfc6455> (дата обр. 22.05.2026).
19. RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format : RFC Editor. — 2017. — URL: <https://www.rfc-editor.org/rfc/rfc8259> (дата обр. 22.05.2026).
20. Олифер В. Г., Олифер Н. А. Компьютерные сети. Принципы, технологии, протоколы. — 5-е изд. — Санкт-Петербург : Питер, 2016. — 992 с.
21. Федюлов А. А. Проектирование архитектуры протокола клиент-серверного взаимодействия web-приложений на основе websocket // Программные системы и вычислительные методы. — 2025. — № 3. — С. 20—30.

22. NumPy documentation : NumPy documentation. — URL: <https://numpy.org/doc/stable/> (дата обр. 22.05.2026).
23. *Яворски М., Зуаде Т.* Python. Лучшие практики и инструменты. — Санкт-Петербург : Питер, 2024. — 592 с.
24. FastAPI : FastAPI documentation. — URL: <https://fastapi.tiangolo.com/> (дата обр. 22.05.2026).
25. json — JSON encoder and decoder : Python 3 documentation. — URL: <https://docs.python.org/3/library/json.html> (дата обр. 09.04.2026).
26. dart:convert : Dart documentation. — URL: <https://dart.dev/libraries/dart-convert> (дата обр. 09.04.2026).
27. Сравнительный анализ форматов сериализации и передачи данных JSON, XML, CBOR и GPB / В. Д. Шульман [и др.] // StudNet. — 2021. — Т. 4, № 7. — С. 1686—1696.
28. *Рябоконь О. С., Кукарцев В. В.* Новый язык структурного веб-программирования Dart // Актуальные проблемы авиации и космонавтики. — 2013. — Т. 1, № 9. — С. 436—437.
29. *Камышев А. О., Зотин А. Г.* Особенности использования фреймворка Flutter при разработке кроссплатформенных приложений // Актуальные проблемы авиации и космонавтики: сб. материалов VI Междунар. науч.-практ. конф. Т. 2. — Красноярск : СибГУ им. М. Ф. Решетнёва, 2020. — С. 138—140.
30. pytest: helps you write better programs : pytest documentation. — URL: <https://docs.pytest.org/en/stable/> (дата обр. 22.05.2026).

ПРИЛОЖЕНИЕ А

Руководство администратора

Руководство администратора описывает развёртывание и настройку программного комплекса. Комплекс состоит из серверной части на языке Python и мобильного приложения на Flutter. Сервер выполняет вычисления и принимает подключения по протоколу WebSocket, мобильное приложение устанавливается на устройство пользователя. Исходный код доступен в репозитории проекта по адресу <https://github.com/oFem1m/diploma>.

1. Требования к окружению. Для запуска сервера необходим интерпретатор Python версии 3.10 или выше, рекомендуется 3.12. Сборка мобильного приложения требует установленного Flutter SDK версии 3.9 или выше с входящим в его состав Dart SDK. Сервер и мобильное устройство должны находиться в одной сети, а порт сервера — быть доступен с устройства.

2. Установка серверной части. Серверная часть располагается в каталоге `server` репозитория. Зависимости перечислены в файле `requirements.txt`: `fastapi`, `uvicorn`, `websockets`, `pydantic` и `numpy`. Установка выполняется в отдельном виртуальном окружении:

```
1 cd server
2 python -m venv .venv
3 source .venv/bin/activate           # Linux, macOS
4 .venv\Scripts\activate              # Windows
5 pip install -r requirements.txt
```

3. Запуск сервера. Сервер запускается одной командой из каталога `server`:

```
1 python main.py
```

По умолчанию сервер слушает адрес `0.0.0.0` и порт `8765`, а соединения принимаются по конечной точке `ws://<host>:8765/ws/optimize`. Именно этот адрес и порт указываются на экране подключения мобильного приложения.

4. Настройка сервера. Параметры сервера задаются в файле `config.py`. Основные из них приведены в таблице 3.

Таблица 3 – Параметры конфигурации сервера

Параметр	По умолчанию	Назначение
host	0.0.0.0	адрес привязки
port	8765	порт сервера
max_queue_size	50	предельный размер очереди задач
max_workers	2	число параллельных обработчиков
default_timeout_ms	600000	таймаут задачи, мс
ping_interval_s	30	интервал проверки связи, с
reconnect_grace_s	120	срок ожидания переподключения, с

Отдельно в классе `Limits` заданы ограничения на параметры задач: предельное число поколений равно 10000, максимальный размер популяции — 500, наибольшая размерность — 100, предельный размер сообщения — 1 МБ. Эти значения защищают сервер от заведомо чрезмерных задач.

5. Установка мобильного приложения. Мобильное приложение располагается в каталоге `mobile` и собирается средствами Flutter. Ниже приведён порядок действий для платформы Android.

5.1. Подготовка окружения. Должен быть установлен Flutter SDK версии 3.9 или выше. Корректность установки проверяется командой, которая выводит сведения об установленных компонентах и обнаруженных устройствах:

```
1 flutter doctor
```

5.2. Загрузка зависимостей. В каталоге приложения загружаются пакеты, перечисленные в `pubspec.yaml`, — среди них `flutter_bloc`, `web_socket_channel`, `fl_chart`, `sqflite` и `image`:

```
1 cd mobile
2 flutter pub get
```

5.3. Запуск на устройстве при отладке. К компьютеру подключается устройство с включённой отладкой по USB либо запускается эмулятор. Список доступных устройств выводится командой `flutter devices`, после чего приложение собирается и устанавливается командой:

```
1 flutter run
```

5.4. Сборка установочного пакета. Для установки без среды разработки собирается пакет APK. Готовый файл создаётся в каталоге `build/app/outputs/flutter-apk/app-release.apk`:

```
1 flutter build apk --release
```

5.5. Установка пакета. Собранный пакет переносится на устройство и устанавливается. При наличии подключённого по USB устройства это можно сделать с компьютера:

```
1 adb install build/app/outputs/flutter-apk/app-release.apk
```

Приложению требуется только доступ в сеть — соответствующее разрешение уже объявлено в манифесте `AndroidManifest.xml`. При первом запуске на экране подключения указываются адрес и порт сервера, заданные при его настройке.

Руководство пользователя

Руководство описывает работу с мобильным приложением комплекса — от запуска сервера до анализа результатов. Предполагается, что серверная часть уже запущена и доступна по сети, а мобильное приложение установлено на устройство.

Перед началом работы необходимо запустить сервер на компьютере, доступном по сети. Сервер начинает принимать подключения по указанному адресу и порту. После этого работа ведётся через мобильное приложение в следующем порядке.

1. Подключение к серверу. При запуске приложение открывает экран подключения. В поля вводятся адрес и порт сервера, после чего нажимается кнопка подключения. Если соединение установлено, приложение переходит к экрану настройки; иначе под кнопкой выводится сообщение об ошибке, и адрес следует проверить. Внешний вид экрана приведён на рисунке 10.
2. Настройка эксперимента. На экране настройки выбирается целевая функция — встроенная или заданная собственным выражением, — задаются размерность и границы области поиска, выбирается метод оптимизации и его параметры. Для большинства параметров уже подставлены разумные значения по умолчанию, поэтому при первом знакомстве

достаточно выбрать метод и функцию. Экран настройки показан на рисунке 11.

3. Запуск и наблюдение за вычислением. После нажатия кнопки запуска приложение переходит к экрану выполнения, где отображаются номер текущего поколения, лучшее найденное значение и график сходимости, обновляемый в реальном времени. При необходимости вычисление можно прервать кнопкой отмены. Этот экран показан на рисунке 12.
4. Просмотр результата. По завершении вычисления открывается экран результата с лучшим решением, показателями качества, графиком сходимости и контурной картой пространства поиска. Здесь же доступны построение анимации и переход к трёхмерной визуализации, которую можно вращать движением пальца. Вид экрана результата приведён на рисунках 13–15.
5. Работа с историей и сравнение. Каждый завершённый эксперимент сохраняется в историю, доступную с экрана настройки. В истории запуски можно фильтровать, открывать повторно и отбирать несколько штук для сравнения. На экране сравнения выбранные запуски выводятся вместе — на совмещённом графике сходимости, в таблице показателей и на столбчатой диаграмме. Экраны истории и сравнения показаны на рисунке 16.

При потере соединения с сервером приложение пытается восстановить его автоматически. Если сервер недоступен длительное время, следует проверить, запущен ли он, и повторить подключение вручную с экрана подключения.

ПРИЛОЖЕНИЕ Б

Листинг 11: Точка входа серверного приложения и подключение WebSocket-маршрута (server/main.py)

```
1 from __future__ import annotations
2
3 import asyncio
4 import logging
5 import sys
6 import os
7 from contextlib import asynccontextmanager
8
9 sys.path.insert(0, os.path.dirname(__file__))
10
11 import uvicorn
12 from fastapi import FastAPI, WebSocket
13
14 from config import ServerConfig
15 from task_queue.job_queue import JobQueue
16 from task_queue.worker import Worker
17 from ws_handler import ConnectionHandler
18
19 logging.basicConfig(
20     level=logging.INFO,
21     format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
22 )
23 logger = logging.getLogger(__name__)
24
25 config = ServerConfig()
26 job_queue = JobQueue(max_size=config.max_queue_size)
27 workers: list[Worker] = []
28
29
30 @asynccontextmanager
31 async def lifespan(application: FastAPI):
32     worker_tasks = []
33     for i in range(config.max_workers):
34         w = Worker(worker_id=f"worker-{i+1}", job_queue=job_queue,
35                     config=config)
36         workers.append(w)
37         worker_tasks.append(asyncio.create_task(w.start()))
38     logger.info("started %d workers", len(workers))
39     yield
40     for w in workers:
41         w.stop()
42     for t in worker_tasks:
43         t.cancel()
```

```

43     logger.info("workers stopped")
44
45
46 app = FastAPI(title="Optimization Server", version="0.1.0", lifespan=
    lifespan)
47
48
49 @app.websocket("/ws/optimize")
50 async def websocket_endpoint(ws: WebSocket):
51     handler = ConnectionHandler(ws, job_queue, config)
52     await handler.handle()
53
54
55 @app.get("/health")
56 async def health():
57     return {
58         "status": "ok",
59         "workers": len(workers),
60         "queue_size": len(job_queue.get_queued_jobs()),
61     }
62
63
64 if __name__ == "__main__":
65     uvicorn.run(
66         "main:app",
67         host=config.host,
68         port=config.port,
69         log_level="info",
70         ws_ping_interval=None,
71     )

```

Листинг 12: Обработка WebSocket-соединения, команд клиента и передачи прогресса (server/ws_handler.py)

```

1 from __future__ import annotations
2
3 import asyncio
4 import json
5 import logging
6 from datetime import datetime, timezone
7
8 from fastapi import WebSocket, WebSocketDisconnect
9
10 from config import ServerConfig
11 from protocol.envelope import build_envelope, parse_envelope
12 from protocol.types import (
13     HelloClientPayload,
14     JobSubmitPayload,

```

```

15     CancelPayload,
16     JobStatusGetPayload,
17 )
18 from protocol.validation import validate_job_submit, SUPPORTED_METHODS,
    SUPPORTED_BUILTINS
19 from task_queue.job import Job
20 from task_queue.job_queue import JobQueue, QueueOverflowError
21
22 logger = logging.getLogger(__name__)
23
24
25 class ConnectionHandler:
26     def __init__(self, ws: WebSocket, job_queue: JobQueue, config:
        ServerConfig):
27         self.ws = ws
28         self.job_queue = job_queue
29         self.config = config
30         self._active_jobs: set[str] = set()
31         self._ping_task = None
32         self._send_lock = asyncio.Lock()
33
34     async def handle(self):
35         await self.ws.accept()
36         logger.info("client connected")
37
38         try:
39             if not await self._wait_hello():
40                 return
41             self._ping_task = asyncio.ensure_future(self._ping_loop())
42             await self._message_loop()
43         except WebSocketDisconnect:
44             logger.info("client disconnected")
45         except Exception:
46             logger.exception("connection error")
47         finally:
48             if self._ping_task:
49                 self._ping_task.cancel()
50             for job_id in self._active_jobs:
51                 self.job_queue.detach_ws_later(
52                     job_id,
53                     self.ws,
54                     self.config.reconnect_grace_s,
55                 )
56
57     async def _wait_hello(self):
58         raw = await self.ws.receive_text()
59         envelope = parse_envelope(raw)
60

```

```

61         if envelope.header.type != "hello":
62             await self._send_error(
63                 "BAD_REQUEST",
64                 "first message must be hello",
65                 reply_to=envelope.header.msg_id,
66             )
67             await self.ws.close()
68             return False
69
70         capabilities = {
71             "methods": list(SUPPORTED_METHODS),
72             "objective_kinds": ["builtin", "expression"],
73             "progress_modes": ["per_generation", "every_k", "none"],
74             "max_payload_bytes": self.config.limits.max_payload_bytes,
75         }
76
77         await self._send("hello", {
78             "server": {"name": "optimizer-server", "version": "0.1.0"},
79             "capabilities": capabilities,
80         }, reply_to=envelope.header.msg_id)
81         return True
82
83     async def _message_loop(self):
84         while True:
85             raw = await self.ws.receive_text()
86             try:
87                 envelope = parse_envelope(raw)
88             except Exception as e:
89                 await self._send_error("BAD_REQUEST", f"invalid message
90                                         : {e}")
91                 continue
92
93             msg_type = envelope.header.type
94             handlers = {
95                 "job.submit": self._handle_submit,
96                 "cancel": self._handle_cancel,
97                 "job.status.get": self._handle_status_get,
98                 "ping": self._handle_ping,
99                 "pong": self._handle_pong,
100             }
101
102             handler = handlers.get(msg_type)
103             if handler:
104                 await handler(envelope)
105             else:
106                 await self._send_error(
107                     "BAD_REQUEST",
108                     f"unknown message type: {msg_type}",

```

```

108         reply_to=envelope.header.msg_id,
109     )
110
111     async def _handle_submit(self, envelope):
112         try:
113             payload = JobSubmitPayload.model_validate(envelope.payload)
114         except Exception as e:
115             await self._send_error(
116                 "VALIDATION_ERROR",
117                 f"invalid payload: {e}",
118                 reply_to=envelope.header.msg_id,
119             )
120             return
121
122         errors = validate_job_submit(payload, self.config.limits)
123         if errors:
124             await self._send_error(
125                 "VALIDATION_ERROR",
126                 "; ".join(errors),
127                 reply_to=envelope.header.msg_id,
128             )
129             return
130
131         client_req_id = envelope.header.trace.client_req_id or envelope
132             .header.msg_id
133         priority = payload.job.priority.value if hasattr(payload.job,
134             priority, "value") else payload.job.priority
135
136         try:
137             job, is_new = await self.job_queue.submit(
138                 payload=payload.model_dump(),
139                 client_req_id=client_req_id,
140                 priority=priority,
141                 ws=self.ws,
142                 ws_send_lock=self._send_lock,
143             )
144         except QueueOverflowError:
145             await self._send_error(
146                 "QUEUE_OVERFLOW",
147                 "job queue is full, try again later",
148                 reply_to=envelope.header.msg_id,
149                 retryable=True,
150             )
151             return
152
153         self._active_jobs.add(job.job_id)

```

```

153         position, eta_ms = self.job_queue.get_queue_position(job.job_id
154         )
155         trace = {
156             "client_req_id": client_req_id,
157             "job_id": job.job_id,
158             "span_id": None,
159         }
160         await self._send("job.accepted", {
161             "job_id": job.job_id,
162             "state": "queued",
163             "queue": {"position": position, "eta_ms": eta_ms},
164         }, reply_to=envelope.header.msg_id, trace=trace)
165
166     async def _handle_cancel(self, envelope):
167         try:
168             payload = CancelPayload.model_validate(envelope.payload)
169         except Exception as e:
170             await self._send_error("BAD_REQUEST", str(e), reply_to=
171                 envelope.header.msg_id)
172             return
173
174         job = self.job_queue.get_job(payload.job_id)
175         if not job:
176             await self._send_error("JOB_NOT_FOUND", f"job {payload.
177                 job_id} not found",
178                 reply_to=envelope.header.msg_id)
179             return
180
181         was_queued = job.state.value == "queued"
182         success = self.job_queue.cancel(payload.job_id)
183         if not success:
184             await self._send_error("JOB_ALREADY_FINISHED",
185                 f"job {payload.job_id} already in
186                     state {job.state.value}",
187                 reply_to=envelope.header.msg_id)
188             return
189
190         if was_queued:
191             trace = {
192                 "client_req_id": job.client_req_id,
193                 "job_id": job.job_id,
194                 "span_id": None,
195             }
196             await self._send("job.finished", {
197                 "job_id": job.job_id,
198                 "final_state": "cancelled",
199                 "finished_at": datetime.now(timezone.utc).isoformat(
200                     timespec="milliseconds"),

```

```

196         }, reply_to=envelope.header.msg_id, trace=trace)
197
198     async def _handle_status_get(self, envelope):
199         try:
200             payload = JobStatusGetPayload.model_validate(envelope.
201                 payload)
202         except Exception as e:
203             await self._send_error("BAD_REQUEST", str(e), reply_to=
204                 envelope.header.msg_id)
205             return
206
207         job = self.job_queue.get_job(payload.job_id)
208         if not job:
209             await self._send_error("JOB_NOT_FOUND", f"job {payload.
210                 job_id} not found",
211                 reply_to=envelope.header.msg_id)
212             return
213
214         client_req_id = envelope.header.trace.client_req_id
215         if client_req_id and client_req_id != job.client_req_id:
216             await self._send_error("JOB_NOT_FOUND", f"job {payload.
217                 job_id} not found",
218                 reply_to=envelope.header.msg_id)
219             return
220
221         if not job.is_terminal:
222             self.job_queue.attach_ws(job.job_id, self.ws, self.
223                 _send_lock)
224             self._active_jobs.add(job.job_id)
225
226         status_payload = {
227             "job_id": job.job_id,
228             "state": job.state.value,
229             "queue": None,
230             "progress": job.last_progress,
231         }
232
233         if job.state.value == "queued":
234             pos, eta = self.job_queue.get_queue_position(job.job_id)
235             status_payload["queue"] = {"position": pos, "eta_ms": eta}
236
237         trace = {
238             "client_req_id": job.client_req_id,
239             "job_id": job.job_id,
240             "span_id": None,
241         }
242
243         await self._send("job.status", status_payload,
244             reply_to=envelope.header.msg_id, trace=trace)

```

```

239         if job.is_terminal:
240             await self._send_terminal_snapshot(job, reply_to=envelope.
                header.msg_id, trace=trace)
241
242     async def _handle_ping(self, envelope):
243         await self._send("pong", {}, reply_to=envelope.header.msg_id)
244
245     async def _handle_pong(self, envelope):
246         return
247
248     async def _send_terminal_snapshot(self, job: Job, *, reply_to: str
        = None, trace: dict = None):
249         if job.state.value == "succeeded" and job.result:
250             history = job.result.get("history_best_f", [])
251             await self._send("job.result", {
252                 "job_id": job.job_id,
253                 "result": self._build_result_payload(job),
254                 "metrics": {
255                     "method": job.result.get("method", "unknown"),
256                     "iterations": len(history),
257                     "evals": len(history),
258                     "wall_time_ms": job.result.get("wall_time_ms", 0),
259                     "seed": job.result.get("seed"),
260                 },
261                 }, reply_to=reply_to, trace=trace)
262         await self._send("job.finished", {
263             "job_id": job.job_id,
264             "final_state": job.state.value,
265             "finished_at": datetime.now(timezone.utc).isoformat(
                timespec="milliseconds"),
266         }, reply_to=reply_to, trace=trace)
267
268     def _build_result_payload(self, job: Job):
269         output = job.payload.get("output", {})
270         if hasattr(output, "model_dump"):
271             output = output.model_dump()
272
273         result = job.result
274         payload = {
275             "best_x": result["best_x"],
276             "best_f": result["best_f"],
277             "history_best_f": result.get("history_best_f", []),
278         }
279         if output.get("return_final_population", True):
280             if result.get("final_population") is not None:
281                 payload["final_population"] = result.get("
                    final_population")
282             if result.get("final_positions") is not None:

```

```

283         payload["final_positions"] = result.get("
                final_positions")
284     if output.get("return_final_fitness", True) and result.get("
                final_fitness") is not None:
285         payload["final_fitness"] = result.get("final_fitness")
286     if output.get("return_final_velocities", False) and result.get(
                "final_velocities") is not None:
287         payload["final_velocities"] = result.get("final_velocities"
                )
288     return payload
289
290     async def _ping_loop(self):
291         try:
292             while True:
293                 await asyncio.sleep(self.config.ping_interval_s)
294                 await self._send("ping", {})
295         except asyncio.CancelledError:
296             pass
297
298     async def _send(self, msg_type: str, payload: dict, *,
299                    reply_to: str = None, trace: dict = None):
300         msg = build_envelope(msg_type, payload, reply_to=reply_to,
301                             trace=trace)
302         try:
303             async with self._send_lock:
304                 await self.ws.send_text(json.dumps(msg, default=str))
305         except RuntimeError:
306             raise WebSocketDisconnect()
307
308     async def _send_error(self, code: str, message: str, *,
309                          reply_to: str = None, retryable: bool = False
310                          ):
311         await self._send("error", {
312             "code": code,
313             "message": message,
314             "details": None,
315             "retryable": retryable,
316         }, reply_to=reply_to)

```

Листинг 13: Фоновый исполнитель задач оптимизации и отправка промежуточных результатов (server/task_queue/worker.py)

```

1 from __future__ import annotations
2
3 import asyncio
4 import logging
5 import traceback
6 from concurrent.futures import ThreadPoolExecutor

```

```

7 from datetime import datetime, timezone
8 from typing import Any, Dict
9
10 from config import ServerConfig
11 from optimizer.runner import run_optimization, CancelledError
12 from protocol.envelope import build_envelope
13 from task_queue.job import Job, JobState
14 from task_queue.job_queue import JobQueue
15
16 logger = logging.getLogger(__name__)
17
18
19 class Worker:
20     def __init__(self, worker_id: str, job_queue: JobQueue, config:
        ServerConfig):
21         self.worker_id = worker_id
22         self.job_queue = job_queue
23         self.config = config
24         self._executor = ThreadPoolExecutor(max_workers=1,
            thread_name_prefix=f"opt-{worker_id}")
25         self._running = False
26
27     async def start(self):
28         self._running = True
29         logger.info("worker %s started", self.worker_id)
30         while self._running:
31             try:
32                 job = await self.job_queue.get_next()
33                 if job.is_terminal:
34                     continue
35                 await self._run_job(job)
36             except asyncio.CancelledError:
37                 break
38             except Exception:
39                 logger.exception("worker %s unexpected error", self.
                    worker_id)
40                 await asyncio.sleep(1)
41
42     def stop(self):
43         self._running = False
44         self._executor.shutdown(wait=False)
45
46     async def _run_job(self, job: Job):
47         job.mark_started()
48         await self._send(job, "job.started", {
49             "job_id": job.job_id,
50             "started_at": datetime.now(timezone.utc).isoformat(timespec
                ="milliseconds"),

```

```

51         "worker": {"id": self.worker_id, "host": "localhost"},
52     })
53
54     loop = asyncio.get_event_loop()
55
56     def on_progress(data: Dict):
57         try:
58             loop.call_soon_threadsafe(job.progress_queue.put_nowait
59                                     , data)
60         except Exception:
61             pass
62
63     progress_task = asyncio.ensure_future(self._stream_progress(job
64                                         ))
65
66     timeout_ms = job.payload.get("job", {})
67     if hasattr(timeout_ms, "timeout_ms"):
68         timeout_ms = timeout_ms.timeout_ms
69     elif isinstance(timeout_ms, dict):
70         timeout_ms = timeout_ms.get("timeout_ms")
71     timeout_s = (timeout_ms or self.config.default_timeout_ms) /
72                 1000.0
73
74     try:
75         result = await asyncio.wait_for(
76             loop.run_in_executor(
77                 self._executor,
78                 run_optimization,
79                 self._serialize_payload(job.payload),
80                 on_progress,
81                 job.cancel_event,
82             ),
83             timeout=timeout_s,
84         )
85
86     job.progress_queue.put_nowait(None)
87     await progress_task
88
89     if job.state == JobState.CANCELLED:
90         await self._send_finished(job, "cancelled")
91         return
92
93     job.mark_succeeded(result)
94
95     max_gen = result.get("history_best_f", [])
96     iterations = len(max_gen)
97
98     await self._send(job, "job.result", {

```

```

96         "job_id": job.job_id,
97         "result": self._build_result_payload(job, result),
98         "metrics": {
99             "method": result.get("method", "unknown"),
100             "iterations": iterations,
101             "evals": iterations,
102             "wall_time_ms": result.get("wall_time_ms", 0),
103             "seed": result.get("seed"),
104         },
105     })
106
107     await self._send_finished(job, "succeeded")
108
109     except asyncio.TimeoutError:
110         job.mark_timeout()
111         job.progress_queue.put_nowait(None)
112         await progress_task
113         await self._send_error(job, "TIMEOUT", "job exceeded
114             timeout")
115         await self._send_finished(job, "timeout")
116
117     except CancelledError:
118         job.mark_cancelled()
119         job.progress_queue.put_nowait(None)
120         await progress_task
121         await self._send_finished(job, "cancelled")
122
123     except Exception as e:
124         job.mark_failed(str(e))
125         job.progress_queue.put_nowait(None)
126         await progress_task
127         logger.exception("job %s failed", job.job_id)
128         await self._send_error(job, "EXECUTION_ERROR", str(e))
129         await self._send_finished(job, "failed")
130
131     async def _stream_progress(self, job: Job):
132         streaming = job.payload.get("streaming", {})
133         if hasattr(streaming, "model_dump"):
134             streaming = streaming.model_dump()
135
136         mode = streaming.get("mode", "per_generation")
137         every_k = streaming.get("every_k", 1)
138         include = streaming.get("include", {})
139
140         if mode == "none":
141             while True:
142                 data = await job.progress_queue.get()
143                 if data is None:

```

```

143         break
144     return
145
146     last_sent = -1
147     while True:
148         data = await job.progress_queue.get()
149         if data is None:
150             break
151
152         iteration = data.get("iteration", 0)
153         if mode == "every_k" and (iteration - last_sent) < every_k:
154             continue
155
156         last_sent = iteration
157         progress_payload = {
158             "job_id": job.job_id,
159             "progress": {
160                 "iteration": data["iteration"],
161                 "max_iterations": data["max_iterations"],
162                 "elapsed_ms": data.get("elapsed_ms", 0),
163             },
164             "snapshots": None,
165         }
166
167         if include.get("best_f", True):
168             progress_payload["progress"]["best_f"] = data["best_f"]
169             if data.get("history_best_f_tail"):
170                 progress_payload["progress"]["history_best_f_tail"]
171                     = data["history_best_f_tail"]
172         if include.get("best_x", True):
173             progress_payload["progress"]["best_x"] = data.get("
174                 best_x")
175         snapshots = {}
176         if include.get("population", False) and data.get("
177             population") is not None:
178             snapshots["population"] = self._to_jsonable(data.get("
179                 population"))
180         if include.get("fitness", False) and data.get("fitness") is
181             not None:
182             snapshots["fitness"] = self._to_jsonable(data.get("

```

```

183         job.last_progress = progress_payload["progress"]
184         await self._send(job, "job.progress", progress_payload)
185
186     def _build_result_payload(self, job: Job, result: Dict[str, Any])
187     -> Dict[str, Any]:
188         output = job.payload.get("output", {})
189         if hasattr(output, "model_dump"):
190             output = output.model_dump()
191
192         result_payload = {
193             "best_x": result["best_x"],
194             "best_f": result["best_f"],
195             "history_best_f": result.get("history_best_f", []),
196         }
197         if output.get("return_final_population", True):
198             if result.get("final_population") is not None:
199                 result_payload["final_population"] = result.get("
200                     final_population")
201             if result.get("final_positions") is not None:
202                 result_payload["final_positions"] = result.get("
203                     final_positions")
204             if output.get("return_final_fitness", True) and result.get("
205                 final_fitness") is not None:
206                 result_payload["final_fitness"] = result.get("final_fitness
207                     ")
208             if output.get("return_final_velocities", False) and result.get(
209                 "final_velocities") is not None:
210                 result_payload["final_velocities"] = result.get("
211                     final_velocities")
212         return result_payload
213
214     def _to_jsonable(self, value):
215         if hasattr(value, "tolist"):
216             return value.tolist()
217         return value
218
219     async def _send_finished(self, job: Job, final_state: str):
220         await self._send(job, "job.finished", {
221             "job_id": job.job_id,
222             "final_state": final_state,
223             "finished_at": datetime.now(timezone.utc).isoformat(
224                 timespec="milliseconds"),
225         })
226
227     async def _send_error(self, job: Job, code: str, message: str):
228         await self._send(job, "error", {
229             "code": code,
230             "message": message,

```

```

223         "details": None,
224         "retryable": False,
225     })
226
227     async def _send(self, job: Job, msg_type: str, payload: Dict[str,
228         Any]):
229         if not job.ws:
230             return
231         try:
232             trace = {
233                 "client_req_id": job.client_req_id,
234                 "job_id": job.job_id,
235                 "span_id": None,
236             }
237             msg = build_envelope(msg_type, payload, trace=trace)
238             import json
239             if job.ws_send_lock:
240                 async with job.ws_send_lock:
241                     await job.ws.send_text(json.dumps(msg, default=str))
242             else:
243                 await job.ws.send_text(json.dumps(msg, default=str))
244         except Exception:
245             logger.debug("failed to send %s to job %s", msg_type, job.
246                 job_id)
247
248     def _serialize_payload(self, payload: Dict[str, Any]) -> Dict[str,
249         Any]:
250         result = {}
251         for key, val in payload.items():
252             if hasattr(val, "model_dump"):
253                 result[key] = val.model_dump()
254             else:
255                 result[key] = val
256         return result

```

Листинг 14: Единый адаптер запуска методов оптимизации (server/optimizer/runner.py)

```

1 from __future__ import annotations
2
3 import threading
4 import time
5 from typing import Any, Callable, Dict, Optional
6
7 from optimizer.objectives import create_objective
8 from optimizer.registry import create_algorithm
9

```

```

10
11 class CancelledError(Exception):
12     pass
13
14
15 def run_optimization(
16     submit_payload: Dict[str, Any],
17     on_progress: Optional[Callable[[Dict], None]] = None,
18     cancel_event: Optional[threading.Event] = None,
19 ) -> Dict[str, Any]:
20     objective_cfg = submit_payload["objective"]
21     if hasattr(objective_cfg, "model_dump"):
22         objective_cfg = objective_cfg.model_dump()
23
24     problem = submit_payload["problem"]
25     if hasattr(problem, "model_dump"):
26         problem = problem.model_dump()
27
28     method = submit_payload["method"]
29     if hasattr(method, "model_dump"):
30         method = method.model_dump()
31
32     dims = problem["dims"]
33     bounds_raw = problem["bounds"]
34     if bounds_raw.get("kind") == "uniform":
35         bounds = [(bounds_raw["low"], bounds_raw["high"])] * dims
36     else:
37         bounds = [(b[0], b[1]) for b in bounds_raw["items"]]
38
39     objective_fn = create_objective(objective_cfg, dims)
40
41     start_time = time.monotonic()
42
43     def progress_wrapper(data: Dict):
44         if cancel_event and cancel_event.is_set():
45             raise CancelledError("job cancelled")
46         if on_progress:
47             data["elapsed_ms"] = int((time.monotonic() - start_time) *
48                                     1000)
49             on_progress(data)
50
51     algo = create_algorithm(
52         method_name=method["name"],
53         config_dict=method.get("config", {}),
54         dims=dims,
55         bounds=bounds,
56         objective=objective_fn,
57         on_progress=progress_wrapper,

```

```

57     )
58
59     result = algo.optimize()
60
61     wall_time_ms = int((time.monotonic() - start_time) * 1000)
62
63     for key in ("best_x", "final_population", "final_fitness", "
64                 final_positions", "final_velocities"):
65         if key in result:
66             import numpy as np
67             val = result[key]
68             if isinstance(val, np.ndarray):
69                 result[key] = val.tolist()
70
71     if isinstance(result.get("history_best_f"), list):
72         result["history_best_f"] = [float(x) for x in result["
73                                     history_best_f"]]
74
75     result["wall_time_ms"] = wall_time_ms
76     result["method"] = method["name"]
77     result["seed"] = method.get("config", {}).get("random_seed")
78
79     return result

```

Листинг 15: Реестр доступных методов оптимизации и фабрика оптимизаторов (server/optimizer/registry.py)

```

1  from __future__ import annotations
2
3  import sys
4  import os
5  from typing import Any, Callable, Dict, Optional, Tuple, Type
6
7  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
8
9  from BB0.BB0_classic.BB0_classic import ClassicBB0, BB0Config as
10     ClassicBB0Config
11  from BB0.BB0_classic.BB0_classic_modified import ClassicBB0_Modified,
12     BB0ConfigModified
13  from BB0.BB0_sinusoidal.BB0_sinusoidal import BB0_Sinusoidal, BB0Config
14     as SinBB0Config
15  from BB0.BB0_sinusoidal.BB0_sinusoidal_modified import
16     BB0_Sinusoidal_Modified, BB0ConfigSinModified
17  from BB0.BB0_mixed.BB0_mixed import BB0_Mixed, BB0Config as
18     MixedBB0Config
19  from BB0.BB0_mixed.BB0_mixed_modified import BB0_Mixed_Modified,
20     BB0ConfigMixedModified
21  from BB0.PSO.PSO import PSO, PSOConfig

```

```

16
17 METHODS: Dict[str, Tuple[Type, Type]] = {
18     "bbo.classic": (ClassicBBO, ClassicBBOConfig),
19     "bbo.classic.modified": (ClassicBBO_Modified, BBOConfigModified),
20     "bbo.sinusoidal": (BBO_Sinusoidal, SinBBOConfig),
21     "bbo.sinusoidal.modified": (BBO_Sinusoidal_Modified,
22                                 BBOConfigSinModified),
23     "bbo.mixed": (BBO_Mixed, MixedBBOConfig),
24     "bbo.mixed.modified": (BBO_Mixed_Modified, BBOConfigMixedModified),
25     "pso": (PSO, PSOConfig),
26 }
27
28 def _build_config(config_cls: Type, config_dict: Dict[str, Any], dims:
29     int, bounds: list) -> Any:
30     params = dict(config_dict)
31     params["dims"] = dims
32     params["bounds"] = bounds
33     return config_cls(**params)
34
35 def create_algorithm(
36     method_name: str,
37     config_dict: Dict[str, Any],
38     dims: int,
39     bounds: list,
40     objective: Callable,
41     on_progress: Optional[Callable] = None,
42 ):
43     if method_name not in METHODS:
44         raise ValueError(f"unknown method: {method_name}")
45
46     algo_cls, config_cls = METHODS[method_name]
47     cfg = _build_config(config_cls, config_dict, dims, bounds)
48     return algo_cls(cfg, objective, on_progress=on_progress)

```

Листинг 16: Модифицированный алгоритм ВВО с адаптивной мутацией, перезапуском и локальным поиском (server/BBO/BBO_classic/BBO_classic_modified.py)

```

1 """
2 Модифицированный классический ВВО для задач минимизации.
3
4 Особенности по сравнению с базовым ClassicBBO:
5 - адаптивная мутация: от mutation_start к mutation_end по поколениям;
6 - детектор стагнации: если долго нет улучшений, усиливается мутация
7 и часть худших особей регенерируется;
8 - локальный поиск для лучших решений.

```

```

9  """
10
11  from __future__ import annotations
12
13  from dataclasses import dataclass
14  from typing import Callable, Optional, Sequence, Tuple, Dict, Any, List
15
16  import numpy as np
17
18
19  @dataclass
20  class BB0ConfigModified:
21      pop_size: int = 50
22      dims: int = 10
23      bounds: Optional[Sequence[Tuple[float, float]]] = None
24      max_generations: int = 200
25
26      mutation_start: float = 0.05
27      mutation_end: float = 0.005
28
29      elite_count: int = 2
30      forbid_self_donation: bool = True
31
32      stagnation_generations: int = 25
33      restart_fraction: float = 0.3
34      mutation_boost_factor: float = 4.0
35
36      enable_local_search: bool = False
37      local_search_interval: int = 20
38      local_search_count: int = 2
39      local_search_steps: int = 20
40      local_search_step_start: float = 0.25
41      local_search_step_end: float = 0.02
42
43      random_seed: Optional[int] = None
44
45  def __post_init__(self):
46      assert self.pop_size >= 2
47      assert self.dims >= 1
48      if self.bounds is None:
49          self.bounds = [(-5.0, 5.0)] * self.dims
50      assert len(self.bounds) == self.dims
51      assert self.max_generations >= 1
52      assert 0.0 < self.mutation_end <= self.mutation_start <= 1.0
53      assert 0 <= self.elite_count < self.pop_size
54      assert self.stagnation_generations >= 1
55      assert 0.0 < self.restart_fraction < 1.0
56      assert self.mutation_boost_factor >= 1.0

```

```

57         if self.enable_local_search:
58             assert self.local_search_interval >= 1
59             assert self.local_search_count >= 1
60             assert self.local_search_steps >= 1
61             assert 0.0 < self.local_search_step_end <= self.
                local_search_step_start
62
63
64 class ClassicBB0_Modified:
65     """Классический ВВ0 с адаптивной мутацией и перезапусками."""
66
67     def __init__(self, config: BB0ConfigModified, objective: Callable[[
        np.ndarray], float],
68                 on_progress: Optional[Callable] = None):
69         self.cfg = config
70         self.objective = objective
71         self.on_progress = on_progress
72         self.rng = np.random.default_rng(config.random_seed)
73
74         self.low = np.array([a for a, _ in self.cfg.bounds], dtype=
            float)
75         self.high = np.array([b for _, b in self.cfg.bounds], dtype=
            float)
76         self.span = self.high - self.low
77
78     def optimize(self) -> Dict[str, Any]:
79         N, d = self.cfg.pop_size, self.cfg.dims
80
81         X = self._init_population(N, d)
82         fit = self._evaluate(X)
83
84         best_idx = int(np.argmin(fit))
85         best_x = X[best_idx].copy()
86         best_f = float(fit[best_idx])
87
88         history_best: List[float] = [best_f]
89         stagnation = 0
90
91         for gen in range(self.cfg.max_generations):
92             mutation_rate = self._current_mutation_rate(gen)
93             if stagnation >= self.cfg.stagnation_generations:
94                 X, fit = self._restart_partially(X, fit)
95                 mutation_rate = min(
96                     self.cfg.mutation_start,
97                     mutation_rate * self.cfg.mutation_boost_factor,
98                 )
99                 stagnation = 0
100

```

```

101         X_new = self._migration_step(X, fit)
102         X_new = self._mutation_step(X_new, mutation_rate)
103         fit_new = self._evaluate(X_new)
104
105         X, fit = self._apply_elitism(X, fit, X_new, fit_new)
106
107         if self.cfg.enable_local_search and (gen + 1) % self.cfg.
            local_search_interval == 0:
108             X, fit = self._local_search(X, fit, gen)
109
110         cur_idx = int(np.argmin(fit))
111         cur_best_f = float(fit[cur_idx])
112
113         if cur_best_f < best_f:
114             best_f = cur_best_f
115             best_x = X[cur_idx].copy()
116             stagnation = 0
117         else:
118             stagnation += 1
119
120         history_best.append(best_f)
121
122         if self.on_progress:
123             self.on_progress({
124                 "iteration": gen,
125                 "max_iterations": self.cfg.max_generations,
126                 "best_f": best_f,
127                 "best_x": best_x.tolist(),
128                 "history_best_f_tail": history_best[-10:],
129                 "population": X.copy(),
130                 "fitness": fit.copy(),
131             })
132
133         return {
134             "best_x": best_x,
135             "best_f": best_f,
136             "history_best_f": history_best,
137             "final_population": X,
138             "final_fitness": fit,
139         }
140
141     def _init_population(self, N: int, d: int) -> np.ndarray:
142         return self.low + self.rng.random((N, d)) * self.span
143
144     def _evaluate(self, X: np.ndarray) -> np.ndarray:
145         try:
146             vals = self.objective(X)
147             vals = np.asarray(vals, dtype=float)

```

```

148         if vals.shape == (X.shape[0],):
149             return vals
150     except Exception:
151         pass
152     return np.array([self.objective(ind) for ind in X], dtype=float
153                     )
154
155 def _current_mutation_rate(self, gen: int) -> float:
156     T = max(1, self.cfg.max_generations - 1)
157     alpha = min(1.0, max(0.0, gen / T))
158     return self.cfg.mutation_start + alpha * (self.cfg.mutation_end
159         - self.cfg.mutation_start)
160
161 def _current_local_step(self, gen: int) -> float:
162     T = max(1, self.cfg.max_generations - 1)
163     alpha = min(1.0, max(0.0, gen / T))
164     return self.cfg.local_search_step_start + alpha * (self.cfg.
165         local_search_step_end - self.cfg.local_search_step_start)
166
167 def _migration_step(self, X: np.ndarray, fit: np.ndarray) -> np.
168     ndarray:
169     N, d = X.shape
170     idx = np.argsort(fit)
171     ranks = np.empty_like(idx)
172     ranks[idx] = np.arange(N)
173
174     imm_prob = ranks / max(1, N - 1)
175     emig_weight = (N - 1 - ranks).astype(float)
176     emig_weight /= max(1e-12, emig_weight.sum())
177
178     X_new = X.copy()
179
180     for i in range(N):
181         p_i = imm_prob[i]
182         if p_i <= 0.0:
183             continue
184         for k in range(d):
185             if self.rng.random() < p_i:
186                 donor = i
187                 if self.cfg.forbid_self_donation:
188                     for _ in range(5):
189                         donor = self.rng.choice(N, p=emig_weight)
190                         if donor != i:
191                             break
192                 if donor == i:
193                     continue
194             else:
195                 donor = self.rng.choice(N, p=emig_weight)

```

```

192         X_new[i, k] = X[donor, k]
193
194     return X_new
195
196 def _mutation_step(self, X: np.ndarray, mutation_rate: float) -> np
    .ndarray:
197     N, d = X.shape
198     mask = self.rng.random((N, d)) < mutation_rate
199     if not np.any(mask):
200         return X
201     noise = self.low + self.rng.random((N, d)) * self.span
202     X_mut = X.copy()
203     X_mut[mask] = noise[mask]
204     return X_mut
205
206 def _local_search(self, X: np.ndarray, fit: np.ndarray, gen: int)
    -> Tuple[np.ndarray, np.ndarray]:
207     N, d = X.shape
208     k = min(self.cfg.local_search_count, N)
209     if k <= 0 or self.cfg.local_search_steps <= 0:
210         return X, fit
211
212     step_frac = self._current_local_step(gen)
213
214     idx_sorted = np.argsort(fit)
215     target_idx = idx_sorted[:k]
216
217     X_loc = X.copy()
218     fit_loc = fit.copy()
219
220     for idx in target_idx:
221         x = X_loc[idx].copy()
222         f = float(fit_loc[idx])
223
224         for _ in range(self.cfg.local_search_steps):
225             noise = self.rng.normal(loc=0.0, scale=1.0, size=d) * (
                step_frac * self.span)
226             cand = x + noise
227             cand = np.clip(cand, self.low, self.high)
228
229             f_cand = float(self._evaluate(cand.reshape(1, -1))[0])
230             if f_cand < f:
231                 x = cand
232                 f = f_cand
233
234         X_loc[idx] = x
235         fit_loc[idx] = f
236

```

```

237         return X_loc, fit_loc
238
239     def _apply_elitism(
240         self,
241         X_old: np.ndarray,
242         fit_old: np.ndarray,
243         X_new: np.ndarray,
244         fit_new: np.ndarray,
245     ) -> Tuple[np.ndarray, np.ndarray]:
246         if self.cfg.elite_count <= 0:
247             return X_new, fit_new
248
249         N = X_old.shape[0]
250         elite_count = min(self.cfg.elite_count, N)
251
252         elite_idx = np.argsort(fit_old)[:elite_count]
253         worst_idx_new = np.argsort(fit_new)[-elite_count:]
254
255         X_res = X_new.copy()
256         fit_res = fit_new.copy()
257
258         X_res[worst_idx_new] = X_old[elite_idx]
259         fit_res[worst_idx_new] = fit_old[elite_idx]
260
261         return X_res, fit_res
262
263     def _restart_partially(self, X: np.ndarray, fit: np.ndarray) ->
264         Tuple[np.ndarray, np.ndarray]:
265         N, d = X.shape
266         n_restart = int(max(1, round(self.cfg.restart_fraction * N)))
267         max_replace = max(1, N - self.cfg.elite_count)
268         n_restart = min(n_restart, max_replace)
269
270         worst_idx = np.argsort(fit)[-n_restart:]
271
272         X_new = X.copy()
273         X_new[worst_idx] = self._init_population(n_restart, d)
274         fit_new = self._evaluate(X_new)
275         return X_new, fit_new
276
277 if __name__ == "__main__":
278     from test_functions import rastrigin
279
280     cfg = BB0ConfigModified(
281         pop_size=50,
282         dims=10,
283         bounds=[(-5.12, 5.12)] * 10,

```

```

284         max_generations=100,
285         mutation_start=0.05,
286         mutation_end=0.005,
287         elite_count=2,
288         stagnation_generations=20,
289         restart_fraction=0.3,
290         random_seed=42,
291     )
292
293     algo = ClassicBBO_Modified(cfg, objective=rastrigin)
294     res = algo.optimize()
295     print("Best f:", res["best_f"])

```

Листинг 17: Реализация метода роя частиц для сравнительных экспериментов (server/BBO/PSO/PSO.py)

```

1  """
2  Метод роя частиц PSO для задач минимизации
3
4  Особенности реализации:
5  - инерция w линейно уменьшается от w_start до w_end
6  - когнитивная c1 и социальная c2 составляющие
7  - ограничение скорости по модулю v_max = vmax_frac * диапазон
8  - обработка границ позиций через обрезку clip
9  - элитарность не используется, хранятся личные и глобальные лучшие
10 """
11 from __future__ import annotations
12 from dataclasses import dataclass
13 from typing import Callable, Optional, Sequence, Tuple, Dict, Any
14 import numpy as np
15
16
17 @dataclass
18 class PSOConfig:
19     swarm_size: int = 50
20     dims: int = 10
21     bounds: Optional[Sequence[Tuple[float, float]]] = None
22     max_generations: int = 200
23     w_start: float = 0.9
24     w_end: float = 0.4
25     c1: float = 2.0
26     c2: float = 2.0
27     vmax_frac: float = 0.2
28     random_seed: Optional[int] = None
29
30     def __post_init__(self):
31         assert self.swarm_size >= 2
32         if self.bounds is None:

```

```

33         self.bounds = [(-5.0, 5.0)] * self.dims
34     assert len(self.bounds) == self.dims
35     assert self.max_generations >= 1
36     assert self.vmax_frac > 0
37
38
39 class PSO:
40     def __init__(self, config: PSOConfig, objective: Callable[[np.
41         ndarray], float],
42         on_progress: Optional[Callable] = None):
43         self.cfg = config
44         self.objective = objective
45         self.on_progress = on_progress
46         self.rng = np.random.default_rng(config.random_seed)
47         self.low = np.array([a for a, _ in self.cfg.bounds], dtype=
48             float)
49         self.high = np.array([b for _, b in self.cfg.bounds], dtype=
50             float)
51         self.span = self.high - self.low
52         self.vmax = self.cfg.vmax_frac * self.span
53
54     def optimize(self) -> Dict[str, Any]:
55         N, n = self.cfg.swarm_size, self.cfg.dims
56         X = self.low + self.rng.random((N, n)) * self.span
57         V = -self.vmax + self.rng.random((N, n)) * (2.0 * self.vmax)
58
59         fit = self._evaluate(X)
60         pbest_pos = X.copy()
61         pbest_fit = fit.copy()
62
63         g_idx = int(np.argmin(pbest_fit))
64         gbest_pos = pbest_pos[g_idx].copy()
65         gbest_fit = float(pbest_fit[g_idx])
66
67         history_best = [gbest_fit]
68
69         for t in range(self.cfg.max_generations):
70             w = self._inertia_weight(t)
71
72             r1 = self.rng.random((N, n))
73             r2 = self.rng.random((N, n))
74
75             cognitive = self.cfg.c1 * r1 * (pbest_pos - X)
76             social = self.cfg.c2 * r2 * (gbest_pos - X)
77             V = w * V + cognitive + social
78
79             V = np.clip(V, -self.vmax, self.vmax)

```

```

78         X = X + V
79
80         X = np.clip(X, self.low, self.high)
81
82         fit = self._evaluate(X)
83         improved = fit < pbest_fit
84         if np.any(improved):
85             pbest_pos[improved] = X[improved]
86             pbest_fit[improved] = fit[improved]
87
88         g_idx = int(np.argmin(pbest_fit))
89         if pbest_fit[g_idx] < gbest_fit:
90             gbest_fit = float(pbest_fit[g_idx])
91             gbest_pos = pbest_pos[g_idx].copy()
92
93         history_best.append(gbest_fit)
94
95         if self.on_progress:
96             self.on_progress({
97                 "iteration": t,
98                 "max_iterations": self.cfg.max_generations,
99                 "best_f": gbest_fit,
100                 "best_x": gbest_pos.tolist(),
101                 "history_best_f_tail": history_best[-10:],
102                 "population": X.copy(),
103                 "fitness": fit.copy(),
104                 "velocities": V.copy(),
105             })
106
107         return {
108             "best_x": gbest_pos.copy(),
109             "best_f": float(gbest_fit),
110             "history_best_f": history_best,
111             "final_positions": X.copy(),
112             "final_fitness": fit.copy(),
113             "final_velocities": V.copy(),
114         }
115
116     def _evaluate(self, X: np.ndarray) -> np.ndarray:
117
118         try:
119             vals = self.objective(X)
120             vals = np.asarray(vals, dtype=float)
121             if vals.shape == (X.shape[0],):
122                 return vals
123         except Exception:
124             pass
125

```

```

126         return np.array([self.objective(ind) for ind in X], dtype=float
127                             )
128
129     def _inertia_weight(self, t: int) -> float:
130
131         T = max(1, self.cfg.max_generations - 1)
132         alpha = min(1.0, max(0.0, t / T))
133         return self.cfg.w_start + alpha * (self.cfg.w_end - self.cfg.
134             w_start)
135
136 if __name__ == "__main__":
137     from test_functions import sphere, rastrigin, ackley
138
139     n = 20
140     cfg = PSOConfig(
141         swarm_size=50,
142         dims=n,
143         bounds=[(-5.12, 5.12)] * n,
144         max_generations=200,
145         w_start=0.9,
146         w_end=0.4,
147         c1=2.0,
148         c2=2.0,
149         vmax_frac=0.2,
150         random_seed=42,
151     )
152
153     algo = PSO(cfg, objective=rastrigin)
154     res = algo.optimize()
155     print("Best f:", res["best_f"])
156     print("Best x first 5 dims:", res["best_x"][:5])

```

Листинг 18: Инициализация мобильного приложения, WebSocket-клиента и BLoC-провайдеров (mobile/lib/app.dart)

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'core/ws/ws_client.dart';
4 import 'features/connection/connection_screen.dart';
5 import 'features/connection/connection_cubit.dart';
6
7 class App extends StatelessWidget {
8     const App({super.key});
9
10    @override
11    Widget build(BuildContext context) {
12        return RepositoryProvider(

```

```

13     create: (_) => WsClient(),
14     child: BlocProvider(
15       create: (ctx) => ConnectionCubit(ctx.read<WsClient>()),
16       child: MaterialApp(
17         title: 'Оптимизатор BB0',
18         debugShowCheckedModeBanner: false,
19         theme: ThemeData(
20           colorSchemeSeed: Colors.indigo,
21           brightness: Brightness.light,
22           useMaterial3: true,
23         ),
24         home: const ConnectionScreen(),
25       ),
26     ),
27   );
28 }
29 }

```

Листинг 19: Модель конфигурации эксперимента и формирование JSON-задания (mobile/lib/core/models/job_config.dart)

```

1  class ObjectiveConfig {
2    String kind;
3    String? name;
4    String? expr;
5    bool minimize;
6
7    ObjectiveConfig({
8      this.kind = 'builtin',
9      this.name = 'sphere',
10     this.expr,
11     this.minimize = true,
12   });
13
14   Map<String, dynamic> toJson() {
15     final map = <String, dynamic>{
16       'kind': kind,
17       'minimize': minimize,
18     };
19     if (kind == 'builtin') {
20       map['name'] = name;
21     } else if (kind == 'expression') {
22       map['expr'] = expr;
23     }
24     return map;
25   }
26 }
27

```

```

28 class BoundsConfig {
29     String kind;
30     double low;
31     double high;
32     List<List<double>>? items;
33
34     BoundsConfig({
35         this.kind = 'uniform',
36         this.low = -5.12,
37         this.high = 5.12,
38         this.items,
39     });
40
41     Map<String, dynamic> toJson() {
42         if (kind == 'uniform') {
43             return {'kind': 'uniform', 'low': low, 'high': high};
44         }
45         return {'kind': 'per_dim', 'items': items};
46     }
47 }
48
49 class ProblemConfig {
50     int dims;
51     BoundsConfig bounds;
52
53     ProblemConfig({this.dims = 10, BoundsConfig? bounds})
54         : bounds = bounds ?? BoundsConfig();
55
56     Map<String, dynamic> toJson() => {
57         'dims': dims,
58         'bounds': bounds.toJson(),
59     };
60 }
61
62 class StreamingInclude {
63     bool bestX;
64     bool bestF;
65     bool population;
66     bool fitness;
67     bool velocities;
68
69     StreamingInclude({
70         this.bestX = true,
71         this.bestF = true,
72         this.population = false,
73         this.fitness = false,
74         this.velocities = false,
75     });

```

```

76
77 Map<String, dynamic> toJson() => {
78     'best_x': bestX,
79     'best_f': bestF,
80     'population': population,
81     'fitness': fitness,
82     'velocities': velocities,
83 };
84 }
85
86 class StreamingConfig {
87     String mode;
88     int everyK;
89     StreamingInclude include;
90
91     StreamingConfig({
92         this.mode = 'per_generation',
93         this.everyK = 1,
94         StreamingInclude? include,
95     }) : include = include ?? StreamingInclude();
96
97     Map<String, dynamic> toJson() => {
98         'mode': mode,
99         'every_k': everyK,
100         'include': include.toJson(),
101         'max_points': {'history_best_f': 5000},
102     };
103 }
104
105 class OutputConfig {
106     bool returnFinalPopulation;
107     bool returnFinalFitness;
108     bool returnFinalVelocities;
109
110     OutputConfig({
111         this.returnFinalPopulation = true,
112         this.returnFinalFitness = true,
113         this.returnFinalVelocities = false,
114     });
115
116     Map<String, dynamic> toJson() => {
117         'return_final_population': returnFinalPopulation,
118         'return_final_fitness': returnFinalFitness,
119         'return_final_velocities': returnFinalVelocities,
120     };
121 }
122
123 class JobConfig {

```

```

124 ObjectiveConfig objective;
125 ProblemConfig problem;
126 String methodName;
127 Map<String, dynamic> methodConfig;
128 StreamingConfig streaming;
129 OutputConfig output;
130 String priority;
131 int timeoutMs;
132
133 JobConfig({
134     ObjectiveConfig? objective,
135     ProblemConfig? problem,
136     this.methodName = 'bbo.classic',
137     Map<String, dynamic>? methodConfig,
138     StreamingConfig? streaming,
139     OutputConfig? output,
140     this.priority = 'normal',
141     this.timeoutMs = 600000,
142 }) : objective = objective ?? ObjectiveConfig(),
143     problem = problem ?? ProblemConfig(),
144     methodConfig = methodConfig ?? {},
145     streaming = streaming ?? StreamingConfig(),
146     output = output ?? OutputConfig();
147
148 Map<String, dynamic> toSubmitPayload() => {
149     'job': {
150         'priority': priority,
151         'timeout_ms': timeoutMs,
152         'tags': ['mobile'],
153     },
154     'objective': objective.toJson(),
155     'problem': problem.toJson(),
156     'method': {
157         'name': methodName,
158         'config': methodConfig,
159     },
160     'streaming': streaming.toJson(),
161     'output': output.toJson(),
162 };
163 }

```

Листинг 20: WebSocket-клиент мобильного приложения с поддержкой переподключения (mobile/lib/core/ws/ws_client.dart)

```

1 import 'dart:async';
2 import 'dart:convert';
3 import 'package:web_socket_channel/web_socket_channel.dart';
4 import '../protocol/envelope.dart';

```

```

5 import '../protocol/message_types.dart';
6 import '../protocol/models.dart';
7 import 'ws_event.dart';
8
9 class WsClient {
10   WebSocketChannel? _channel;
11   final _eventController = StreamController<WsEvent>.broadcast();
12   Timer? _pingTimer;
13   Timer? _pongTimeoutTimer;
14   Timer? _reconnectTimer;
15   Timer? _reconnectCountdownTimer;
16   String? _host;
17   int? _port;
18   bool _intentionalClose = false;
19   int _reconnectAttempts = 0;
20   bool _isReconnecting = false;
21   int _reconnectDelaySeconds = 0;
22   int _reconnectRemainingSeconds = 0;
23   bool _awaitingPong = false;
24   final List<Map<String, dynamic>> _pendingEnvelopes = [];
25   static const _maxReconnectDelay = 30;
26   static const _pingIntervalSeconds = 5;
27   static const _pongTimeoutSeconds = 10;
28
29   Stream<WsEvent> get events => _eventController.stream;
30   bool get isConnected => _channel != null;
31   bool get isReconnecting => _isReconnecting;
32   int get reconnectAttempt => _reconnectAttempts;
33   int get reconnectDelaySeconds => _reconnectDelaySeconds;
34   int get reconnectRemainingSeconds => _reconnectRemainingSeconds;
35
36   Future<void> connect(String host, int port) async {
37     _host = host;
38     _port = port;
39     _intentionalClose = false;
40     _reconnectAttempts = 0;
41     await _doConnect();
42   }
43
44   Future<void> _doConnect() async {
45     try {
46       final uri = Uri.parse('ws://$_host:$_port/ws/optimize');
47       _channel = WebSocketChannel.connect(uri);
48       await _channel!.ready;
49       _cancelReconnectTimers();
50       _reconnectAttempts = 0;
51       _eventController.add(WsConnected());
52

```

```

53     _channel!.stream.listen(
54         (data) => _onMessage(data as String),
55         onDone: _onDone,
56         onError: (e) => _onDone(),
57     );
58
59     _sendHello();
60 } catch (e) {
61     _channel = null;
62     _eventController.add(WsDisconnected(reason: e.toString()));
63     _scheduleReconnect();
64 }
65 }
66
67 void _sendHello() {
68     final envelope = buildEnvelope(
69         type: MessageTypes.hello,
70         payload: {
71             'client': {
72                 'name': 'mobile-app',
73                 'version': '1.0.0',
74                 'platform': 'android',
75                 'lang': 'dart',
76             },
77             'wants': {'progress_stream': true, 'chunking': true},
78         },
79         clientReqId: 'hello-1',
80     );
81     send(envelope);
82 }
83
84 void _onMessage(String raw) {
85     final envelope = parseEnvelope(raw);
86     final type = getType(envelope);
87     final payload = getPayload(envelope);
88
89     switch (type) {
90         case MessageTypes.hello:
91             _eventController.add(WsHelloReceived(ServerHello.fromPayload(
92                 payload)));
93             _flushPending();
94             _startPing();
95         case MessageTypes.ping:
96             _sendRaw(buildEnvelope(type: MessageTypes.pong, payload: {}));
97         case MessageTypes.jobAccepted:
98             _eventController.add(WsJobAccepted(JobAccepted.fromPayload(
99                 payload)));
100         case MessageTypes.jobQueued:

```

```

99         _eventController.add(WsJobQueued(JobQueued.fromPayload(payload)
100             ));
101     case MessageTypes.jobStarted:
102         _eventController.add(WsJobStarted(JobStarted.fromPayload(
103             payload)));
104     case MessageTypes.jobProgress:
105         _eventController.add(WsJobProgress(JobProgress.fromPayload(
106             payload)));
107     case MessageTypes.jobResult:
108         _eventController.add(WsJobResult(JobResult.fromPayload(payload)
109             ));
110     case MessageTypes.jobFinished:
111         _eventController.add(WsJobFinished(JobFinished.fromPayload(
112             payload)));
113     case MessageTypes.jobStatus:
114         _eventController.add(WsJobStatus(JobStatus.fromPayload(payload)
115             ));
116     case MessageTypes.error:
117         _eventController.add(WsError(ProtocolError.fromPayload(payload)
118             ));
119     case MessageTypes.pong:
120         _awaitingPong = false;
121         _pongTimeoutTimer?.cancel();
122         _eventController.add(WsPong());
123     }
124 }
125
126 void _onDone() {
127     if (_channel == null) return;
128     _channel = null;
129     _pingTimer?.cancel();
130     _pongTimeoutTimer?.cancel();
131     _awaitingPong = false;
132     if (!_intentionalClose) {
133         _eventController.add(WsDisconnected(reason: 'Connection lost'));
134         _scheduleReconnect();
135     } else {
136         _eventController.add(WsDisconnected());
137     }
138 }
139
140 void _scheduleReconnect() {
141     if (!_intentionalClose || _host == null) return;
142     _cancelReconnectTimers();
143     _reconnectAttempts++;
144     final delay = _reconnectDelay();
145     var remaining = delay;
146     _isReconnecting = true;

```

```

140     _reconnectDelaySeconds = delay;
141     _reconnectRemainingSeconds = remaining;
142     _eventController.add(
143         WsReconnectScheduled(
144             attempt: _reconnectAttempts,
145             delaySeconds: delay,
146             remainingSeconds: remaining,
147         ),
148     );
149     _reconnectCountdownTimer = Timer.periodic(const Duration(seconds:
150         1), (
151         timer,
152     ) {
153         remaining--;
154         if (remaining <= 0) {
155             timer.cancel();
156             return;
157         }
158         _reconnectRemainingSeconds = remaining;
159         _eventController.add(
160             WsReconnectTick(
161                 attempt: _reconnectAttempts,
162                 delaySeconds: delay,
163                 remainingSeconds: remaining,
164             ),
165         );
166     });
167     _reconnectTimer = Timer(Duration(seconds: delay), _doConnect);
168 }
169 int _reconnectDelay() {
170     final delay = 1 << (_reconnectAttempts - 1);
171     return delay > _maxReconnectDelay ? _maxReconnectDelay : delay;
172 }
173
174 void _startPing() {
175     _pingTimer?.cancel();
176     _pongTimeoutTimer?.cancel();
177     _pingTimer = Timer.periodic(const Duration(seconds:
178         _pingIntervalSeconds), (
179     - ,
180     ) {
181         _sendPing();
182     });
183     _sendPing();
184 }
185 void _startPongTimeout() {

```

```

186     _pongTimeoutTimer?.cancel();
187     _pongTimeoutTimer = Timer(const Duration(seconds:
188         _pongTimeoutSeconds), () {
189         _handleConnectionLost('Ping timeout');
190     });
191 }
192
193 void _sendPing() {
194     if (_channel == null || _awaitingPong) return;
195     _awaitingPong = true;
196     send(buildEnvelope(type: MessageTypes.ping, payload: {}));
197     _startPongTimeout();
198 }
199
200 void _handleConnectionLost(String reason) {
201     if (_channel == null || _intentionalClose) return;
202     _channel?.sink.close();
203     _channel = null;
204     _pingTimer?.cancel();
205     _pongTimeoutTimer?.cancel();
206     _awaitingPong = false;
207     _eventController.add(WsDisconnected(reason: reason));
208     _scheduleReconnect();
209 }
210
211 void send(Map<String, dynamic> envelope) {
212     if (_channel == null) {
213         _pendingEnvelopes.add(envelope);
214         return;
215     }
216     _sendRaw(envelope);
217 }
218
219 void _sendRaw(Map<String, dynamic> envelope) {
220     try {
221         _channel?.sink.add(jsonEncode(envelope));
222     } catch (_) {
223         _handleConnectionLost('Connection lost');
224     }
225 }
226
227 void _flushPending() {
228     if (_channel == null || _pendingEnvelopes.isEmpty) return;
229     final pending = List<Map<String, dynamic>>.from(_pendingEnvelopes);
230     _pendingEnvelopes.clear();
231     for (final envelope in pending) {
232         _channel!.sink.add(jsonEncode(envelope));
233     }

```

```

233     }
234
235     void _cancelReconnectTimers() {
236         _reconnectTimer?.cancel();
237         _reconnectTimer = null;
238         _reconnectCountdownTimer?.cancel();
239         _reconnectCountdownTimer = null;
240         _isReconnecting = false;
241         _reconnectDelaySeconds = 0;
242         _reconnectRemainingSeconds = 0;
243     }
244
245     void submitJob({
246         required String clientReqId,
247         required Map<String, dynamic> payload,
248     }) {
249         send(
250             buildEnvelope(
251                 type: MessageTypes.jobSubmit,
252                 payload: payload,
253                 clientReqId: clientReqId,
254             ),
255         );
256     }
257
258     void removePendingByClientReqId(String clientReqId) {
259         _pendingEnvelopes.removeWhere((envelope) {
260             final header = envelope['header'] as Map<String, dynamic>;
261             final trace = header?['trace'] as Map<String, dynamic>;
262             return trace?['client_req_id'] == clientReqId;
263         });
264     }
265
266     void cancelJob(String jobId, String clientReqId) {
267         send(
268             buildEnvelope(
269                 type: MessageTypes.cancel,
270                 payload: {'job_id': jobId, 'reason': 'user_cancel'},
271                 clientReqId: clientReqId,
272                 jobId: jobId,
273             ),
274         );
275     }
276
277     void requestStatus(String jobId, String clientReqId) {
278         send(
279             buildEnvelope(
280                 type: MessageTypes.jobStatusGet,

```

```

281         payload: {'job_id': jobId},
282         clientReqId: clientReqId,
283         jobId: jobId,
284     ),
285 );
286 }
287
288 Future<void> disconnect() async {
289     _intentionalClose = true;
290     _pingTimer?.cancel();
291     _pongTimeoutTimer?.cancel();
292     _awaitingPong = false;
293     _cancelReconnectTimers();
294     _pendingEnvelopes.clear();
295     await _channel?.sink.close();
296     _channel = null;
297 }
298
299 void dispose() {
300     _intentionalClose = true;
301     _pingTimer?.cancel();
302     _pongTimeoutTimer?.cancel();
303     _awaitingPong = false;
304     _cancelReconnectTimers();
305     _pendingEnvelopes.clear();
306     _channel?.sink.close();
307     _eventController.close();
308 }
309 }

```

Листинг 21: Логика настройки эксперимента и валидации параметров (mobile/lib/features/configuration/config_cubit.dart)

```

1  import 'package:equatable/equatable.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import '../core/models/job_config.dart';
4  import '../core/models/method_configs.dart';
5  import '../visualization/test_functions.dart';
6
7  class ConfigState extends Equatable {
8      final JobConfig config;
9      final String? validationError;
10     final int _version;
11
12     const ConfigState({
13         required this.config,
14         this.validationError,
15         int version = 0,

```

```

16    }) : _version = version;
17
18    ConfigState copyWith({
19        JobConfig? config,
20        String? validationError,
21        int? version,
22    }) {
23        return ConfigState(
24            config: config ?? this.config,
25            validationError: validationError,
26            version: version ?? _version,
27        );
28    }
29
30    int get version => _version;
31
32    @override
33    List<Object?> get props => [_version, validationError];
34 }
35
36 class ConfigCubit extends Cubit<ConfigState> {
37     ConfigCubit() : super(ConfigState(config: _defaultConfig()));
38
39     static JobConfig _defaultConfig() {
40         final config = JobConfig();
41         config.methodConfig = createDefaultConfig(config.methodName);
42         return config;
43     }
44
45     void setObjectiveKind(String kind) {
46         state.config.objective.kind = kind;
47         _emitUpdated();
48     }
49
50     void setObjectiveName(String name) {
51         state.config.objective.name = name;
52         final pd = defaultBoundsPerDim[name];
53         if (pd != null) {
54             state.config.problem.dims = pd.length;
55             state.config.problem.bounds.kind = 'per_dim';
56             state.config.problem.bounds.items = pd.map((p) => [p[0], p[1]]).
                    toList();
57         } else {
58             final b = defaultBoundsUniform[name];
59             if (b != null) {
60                 state.config.problem.bounds.kind = 'uniform';
61                 state.config.problem.bounds.low = b[0];
62                 state.config.problem.bounds.high = b[1];

```

```

63         state.config.problem.bounds.items = null;
64     }
65 }
66 _emitUpdated();
67 }
68
69 void setExpression(String expr) {
70     state.config.objective.expr = expr;
71     final requiredDims = _requiredDimsFromExpression(expr);
72     if (requiredDims != null && requiredDims > state.config.problem.
73         dims) {
74         _setDimsValue(requiredDims);
75     }
76     _emitUpdated();
77 }
78
79 void setMinimize(bool minimize) {
80     state.config.objective.minimize = minimize;
81     _emitUpdated();
82 }
83
84 void setDims(int dims) {
85     _setDimsValue(dims);
86     _emitUpdated();
87 }
88
89 void _setDimsValue(int dims) {
90     state.config.problem.dims = dims;
91     final b = state.config.problem.bounds;
92     if (b.kind == 'per_dim') {
93         final old = b.items ?? [];
94         b.items = List.generate(
95             dims,
96             (i) => i < old.length ? old[i] : [b.low, b.high],
97         );
98     }
99 }
100
101 void setBoundsKind(String kind) {
102     state.config.problem.bounds.kind = kind;
103     if (kind == 'per_dim' && state.config.problem.bounds.items == null)
104     {
105         state.config.problem.bounds.items = List.generate(
106             state.config.problem.dims,
107             (_) => [
108                 state.config.problem.bounds.low,
109                 state.config.problem.bounds.high,
110             ],

```

```

109         );
110     }
111     _emitUpdated();
112 }
113
114 void setBoundsUniform(double low, double high) {
115     state.config.problem.bounds.low = low;
116     state.config.problem.bounds.high = high;
117     _emitUpdated();
118 }
119
120 void setBoundsPerDim(int index, double low, double high) {
121     state.config.problem.bounds.items?[index] = [low, high];
122     _emitUpdated();
123 }
124
125 void setMethod(String method) {
126     state.config.methodName = method;
127     state.config.methodConfig = createDefaultConfig(method);
128     if (!isPsoMethod(method)) {
129         state.config.streaming.include.velocities = false;
130         state.config.output.returnFinalVelocities = false;
131     }
132     _emitUpdated();
133 }
134
135 void setMethodConfigValue(String key, dynamic value) {
136     state.config.methodConfig[key] = value;
137     _emitUpdated();
138 }
139
140 void setStreamingMode(String mode) {
141     state.config.streaming.mode = mode;
142     _emitUpdated();
143 }
144
145 void setEveryK(int k) {
146     state.config.streaming.everyK = k;
147     _emitUpdated();
148 }
149
150 void setStreamingInclude(String field, bool value) {
151     final inc = state.config.streaming.include;
152     switch (field) {
153         case 'best_x':
154             inc.bestX = value;
155         case 'best_f':
156             inc.bestF = value;

```

```

157         case 'population':
158             inc.population = value;
159         case 'fitness':
160             inc.fitness = value;
161         case 'velocities':
162             inc.velocities = value;
163     }
164     _emitUpdated();
165 }
166
167 void setOutputFlag(String field, bool value) {
168     final out = state.config.output;
169     switch (field) {
170         case 'return_final_population':
171             out.returnFinalPopulation = value;
172         case 'return_final_fitness':
173             out.returnFinalFitness = value;
174         case 'return_final_velocities':
175             out.returnFinalVelocities = value;
176     }
177     _emitUpdated();
178 }
179
180 void setPriority(String priority) {
181     state.config.priority = priority;
182     _emitUpdated();
183 }
184
185 void applyPreset(String preset) {
186     final config = state.config;
187     switch (preset) {
188         case 'rastrigin_bbo':
189             config.objective.kind = 'builtin';
190             config.objective.name = 'rastrigin';
191             config.objective.minimize = true;
192             config.problem.dims = 10;
193             config.problem.bounds = BoundsConfig(low: -5.12, high: 5.12);
194             config.methodName = 'bbo.classic';
195             config.methodConfig = createDefaultConfig('bbo.classic');
196         case 'sphere_pso':
197             config.objective.kind = 'builtin';
198             config.objective.name = 'sphere';
199             config.objective.minimize = true;
200             config.problem.dims = 10;
201             config.problem.bounds = BoundsConfig(low: -5.12, high: 5.12);
202             config.methodName = 'pso';
203             config.methodConfig = createDefaultConfig('pso');
204         case 'ackley_modified':

```

```

205         config.objective.kind = 'builtin';
206         config.objective.name = 'ackley';
207         config.objective.minimize = true;
208         config.problem.dims = 10;
209         config.problem.bounds = BoundsConfig(low: -5.0, high: 5.0);
210         config.methodName = 'bbo.classic.modified';
211         config.methodConfig = createDefaultConfig('bbo.classic.modified
212             ');
213     }
214     _emitUpdated();
215 }
216 String? validate() {
217     final c = state.config;
218     if (c.objective.kind == 'expression' &&
219         (c.objective.expr == null || c.objective.expr!.isEmpty)) {
220         return 'Необходимо ввести выражение';
221     }
222     if (c.objective.kind == 'expression' && c.objective.expr != null) {
223         final requiredDims = _requiredDimsFromExpression(c.objective.expr
224             !);
225         if (requiredDims != null && requiredDims > c.problem.dims) {
226             return 'Выражение использует x${requiredDims - 1}, поэтому разм
227                 ерность должна быть не меньше $requiredDims';
228         }
229     }
230     if (c.problem.dims < 1 || c.problem.dims > 100) {
231         return 'Размерность должна быть от 1 до 100';
232     }
233     if (c.objective.name == 'bukin6' && c.problem.dims != 2) {
234         return 'Bukin N.6 требует 2 измерения';
235     }
236     if (c.problem.bounds.kind == 'uniform' &&
237         c.problem.bounds.low >= c.problem.bounds.high) {
238         return 'Нижняя граница должна быть меньше верхней';
239     }
240     final mc = c.methodConfig;
241     final maxGen = mc['max_generations'] as int? ?? 200;
242     if (maxGen < 1 || maxGen > 10000) {
243         return 'Макс. поколений должно быть от 1 до 10 000';
244     }
245     if (isPsoMethod(c.methodName)) {
246         if ((mc['swarm_size'] as num?) != null && (mc['swarm_size'] as
247             num) < 2) {
248             return 'Размер роя должен быть >= 2';
249         }
250     } else {

```

```

248         if ((mc['pop_size'] as num?) != null && (mc['pop_size'] as num) <
249             2) {
250             return 'Размер популяции должен быть >= 2';
251         }
252         return null;
253     }
254
255     void _emitUpdated() {
256         emit(ConfigState(config: state.config, version: state.version + 1))
257         ;
258     }
259
260     int? _requiredDimsFromExpression(String expr) {
261         final matches = RegExp(r'\bx(\d+)\b').allMatches(expr);
262         int? maxIndex;
263         for (final match in matches) {
264             final index = int.tryParse(match.group(1)!);
265             if (index != null && (maxIndex == null || index > maxIndex)) {
266                 maxIndex = index;
267             }
268         }
269         return maxIndex == null ? null : maxIndex + 1;
270     }

```

Листинг 22: Обработка прогресса выполнения и накопление истории сходимости (mobile/lib/features/progress/progress_cubit.dart)

```

1  import 'dart:async';
2  import 'package:equatable/equatable.dart';
3  import 'package:flutter_bloc/flutter_bloc.dart';
4  import '../core/models/job_config.dart';
5  import '../core/protocol/models.dart';
6  import '../core/ws/ws_client.dart';
7  import '../core/ws/ws_event.dart';
8
9  enum JobPhase {
10     submitting,
11     queued,
12     started,
13     running,
14     succeeded,
15     failed,
16     cancelled,
17     timeout,
18 }
19

```

```

20 class ProgressState extends Equatable {
21     final JobPhase phase;
22     final String? jobId;
23     final int iteration;
24     final int maxIterations;
25     final double? bestF;
26     final List<double>? bestX;
27     final List<double> historyBestF;
28     final List<double?> liveHistoryBestF;
29     final List<double>? fitness;
30     final List<List<double>> historyBestX;
31     final int elapsedMs;
32     final int? queuePosition;
33     final int? queueEtaMs;
34     final JobResult? result;
35     final String? errorMessage;
36     final bool isReconnecting;
37     final int reconnectAttempt;
38     final int reconnectDelaySeconds;
39     final int reconnectRemainingSeconds;
40
41     const ProgressState({
42         this.phase = JobPhase.submitting,
43         this.jobId,
44         this.iteration = 0,
45         this.maxIterations = 1,
46         this.bestF,
47         this.bestX,
48         this.historyBestF = const [],
49         this.liveHistoryBestF = const [],
50         this.fitness,
51         this.historyBestX = const [],
52         this.elapsedMs = 0,
53         this.queuePosition,
54         this.queueEtaMs,
55         this.result,
56         this.errorMessage,
57         this.isReconnecting = false,
58         this.reconnectAttempt = 0,
59         this.reconnectDelaySeconds = 0,
60         this.reconnectRemainingSeconds = 0,
61     });
62
63     ProgressState copyWith({
64         JobPhase? phase,
65         String? jobId,
66         int? iteration,
67         int? maxIterations,

```

```

68     double? bestF,
69     List<double>? bestX,
70     List<double>? historyBestF,
71     List<double?>? liveHistoryBestF,
72     List<double>? fitness,
73     List<List<double>>? historyBestX,
74     int? elapsedMs,
75     int? queuePosition,
76     int? queueEtaMs,
77     JobResult? result,
78     String? errorMessage,
79     bool? isReconnecting,
80     int? reconnectAttempt,
81     int? reconnectDelaySeconds,
82     int? reconnectRemainingSeconds,
83 }) {
84     return ProgressState(
85         phase: phase ?? this.phase,
86         jobId: jobId ?? this.jobId,
87         iteration: iteration ?? this.iteration,
88         maxIterations: maxIterations ?? this.maxIterations,
89         bestF: bestF ?? this.bestF,
90         bestX: bestX ?? this.bestX,
91         historyBestF: historyBestF ?? this.historyBestF,
92         liveHistoryBestF: liveHistoryBestF ?? this.liveHistoryBestF,
93         fitness: fitness ?? this.fitness,
94         historyBestX: historyBestX ?? this.historyBestX,
95         elapsedMs: elapsedMs ?? this.elapsedMs,
96         queuePosition: queuePosition ?? this.queuePosition,
97         queueEtaMs: queueEtaMs ?? this.queueEtaMs,
98         result: result ?? this.result,
99         errorMessage: errorMessage ?? this.errorMessage,
100        isReconnecting: isReconnecting ?? this.isReconnecting,
101        reconnectAttempt: reconnectAttempt ?? this.reconnectAttempt,
102        reconnectDelaySeconds:
103            reconnectDelaySeconds ?? this.reconnectDelaySeconds,
104        reconnectRemainingSeconds:
105            reconnectRemainingSeconds ?? this.reconnectRemainingSeconds,
106    );
107 }
108
109 double get progress => maxIterations > 0 ? iteration / maxIterations
    : 0;
110
111 @override
112 List<Object?> get props => [
113     phase,
114     jobId,

```

```

115     iteration,
116     maxIterations,
117     bestF,
118     historyBestF.length,
119     liveHistoryBestF.length,
120     fitness,
121     historyBestX.length,
122     elapsedMs,
123     queuePosition,
124     result,
125     errorMessage,
126     isReconnecting,
127     reconnectAttempt,
128     reconnectDelaySeconds,
129     reconnectRemainingSeconds,
130 ];
131 }
132
133 class ProgressCubit extends Cubit<ProgressState> {
134     final WsClient _ws;
135     final JobConfig config;
136     final String _clientReqId;
137     StreamSubscription? _sub;
138     bool _hadConnectionGap = false;
139
140     ProgressCubit(this._ws, this.config, {required String clientReqId})
141       : _clientReqId = clientReqId,
142         super(
143           ProgressState(
144             isReconnecting: _ws.isReconnecting,
145             reconnectAttempt: _ws.reconnectAttempt,
146             reconnectDelaySeconds: _ws.reconnectDelaySeconds,
147             reconnectRemainingSeconds: _ws.reconnectRemainingSeconds,
148           ),
149         ) {
150       _sub = _ws.events.listen(_onEvent);
151       _ws.submitJob(clientReqId: clientReqId, payload: config.
         toSubmitPayload());
152     }
153
154     void _onEvent(WsEvent event) {
155       switch (event) {
156         case WsConnected():
157           emit(state.copyWith(isReconnecting: false));
158         case WsHelloReceived():
159           emit(state.copyWith(isReconnecting: false));
160           final jobId = state.jobId;
161           if (jobId != null && !_isTerminal(state.phase)) {

```

```

162         _ws.requestStatus(jobId, _clientReqId);
163     }
164     case WsReconnectScheduled(
165         :final attempt,
166         :final delaySeconds,
167         :final remainingSeconds,
168     ):
169         _hadConnectionGap = true;
170         emit(
171             state.copyWith(
172                 isReconnecting: true,
173                 reconnectAttempt: attempt,
174                 reconnectDelaySeconds: delaySeconds,
175                 reconnectRemainingSeconds: remainingSeconds,
176             ),
177         );
178     case WsReconnectTick(
179         :final attempt,
180         :final delaySeconds,
181         :final remainingSeconds,
182     ):
183         emit(
184             state.copyWith(
185                 isReconnecting: true,
186                 reconnectAttempt: attempt,
187                 reconnectDelaySeconds: delaySeconds,
188                 reconnectRemainingSeconds: remainingSeconds,
189             ),
190         );
191     case WsJobAccepted(:final data):
192         emit(
193             state.copyWith(
194                 jobId: data.jobId,
195                 phase: JobPhase.queued,
196                 queuePosition: data.queue?.position,
197                 queueEtaMs: data.queue?.etaMs,
198             ),
199         );
200     case WsJobQueued(:final data):
201         emit(
202             state.copyWith(
203                 queuePosition: data.queue.position,
204                 queueEtaMs: data.queue.etaMs,
205             ),
206         );
207     case WsJobStarted(:final data):
208         emit(state.copyWith(jobId: data.jobId, phase: JobPhase.started)
209             );

```

```

209     case WsJobProgress(:final data):
210         _emitProgress(data.progress, snapshots: data.snapshots);
211     case WsJobResult(:final data):
212         emit(
213             state.copyWith(
214                 result: data,
215                 bestF: data.result.bestF,
216                 bestX: data.result.bestX,
217                 historyBestF: data.result.historyBestF,
218             ),
219         );
220     case WsJobFinished(:final data):
221         final phase = _phaseFromServerState(data.finalState);
222         emit(state.copyWith(phase: phase));
223     case WsJobStatus(:final data):
224         final phase = _phaseFromServerState(data.state);
225         if (data.progress != null) {
226             _emitProgress(data.progress!, phase: phase);
227         } else {
228             emit(
229                 state.copyWith(
230                     jobId: data.jobId,
231                     phase: phase,
232                     queuePosition: data.queue?.position,
233                     queueEtaMs: data.queue?.etaMs,
234                 ),
235             );
236         }
237     case WsError(:final error):
238         emit(
239             state.copyWith(phase: JobPhase.failed, errorMessage: error.
240                 message),
241         );
242     default:
243         break;
244 }
245
246 void _emitProgress(
247     ProgressData progress, {
248     JobPhase phase = JobPhase.running,
249     ProgressSnapshots? snapshots,
250 }) {
251     final previousIteration = state.iteration;
252     final newHistory = List<double>.from(state.historyBestF);
253     final newLiveHistory = List<double?>.from(state.liveHistoryBestF);
254     if (progress.historyBestFTail != null) {
255         final tail = progress.historyBestFTail!;

```

```

256     final tailStartIteration = progress.iteration - tail.length + 1;
257     if (_hadConnectionGap && state.iteration > 0) {
258         final missedPoints = tailStartIteration - previousIteration -
259             1;
260         if (missedPoints > 0) {
261             newLiveHistory.addAll(List<double?>.filled(missedPoints, null
262                 ));
263         }
264     }
265     for (var i = 0; i < tail.length; i++) {
266         final iteration = tailStartIteration + i;
267         if (newHistory.isEmpty || iteration > previousIteration) {
268             final v = tail[i];
269             newHistory.add(v);
270             newLiveHistory.add(v);
271         }
272     }
273 } else if (progress.bestF != null &&
274     (newHistory.isEmpty || progress.iteration > previousIteration))
275     {
276     if (_hadConnectionGap && state.iteration > 0) {
277         final missedPoints = progress.iteration - previousIteration -
278             1;
279         if (missedPoints > 0) {
280             newLiveHistory.addAll(List<double?>.filled(missedPoints, null
281                 ));
282         }
283     }
284     newHistory.add(progress.bestF!);
285     newLiveHistory.add(progress.bestF!);
286 }
287 final fitness = snapshots?.fitness;
288 final newHistoryX = List<List<double>>.from(state.historyBestX);
289 if (progress.bestX != null) {
290     newHistoryX.add(List<double>.from(progress.bestX!));
291 }
292 _hadConnectionGap = false;
293 emit(
294     state.copyWith(
295         phase: phase,
296         iteration: progress.iteration,
297         maxIterations: progress.maxIterations,
298         bestF: progress.bestF,
299         bestX: progress.bestX,
300         historyBestF: newHistory,
301         liveHistoryBestF: newLiveHistory,
302         fitness: fitness,
303         historyBestX: newHistoryX,

```

```

299         elapsedMs: progress.elapsedMs,
300     ),
301 );
302 }
303
304 JobPhase _phaseFromServerState(String value) {
305     return switch (value) {
306         'queued' => JobPhase.queued,
307         'started' => JobPhase.started,
308         'running' => JobPhase.running,
309         'succeeded' => JobPhase.succeeded,
310         'failed' => JobPhase.failed,
311         'cancelled' => JobPhase.cancelled,
312         'timeout' => JobPhase.timeout,
313         _ => JobPhase.failed,
314     };
315 }
316
317 bool _isTerminal(JobPhase phase) {
318     return phase == JobPhase.succeeded ||
319         phase == JobPhase.failed ||
320         phase == JobPhase.cancelled ||
321         phase == JobPhase.timeout;
322 }
323
324 void cancelJob() {
325     final jobId = state.jobId;
326     if (jobId == null) {
327         _ws.removePendingByClientReqId(_clientReqId);
328         emit(
329             state.copyWith(
330                 phase: JobPhase.cancelled,
331                 errorMessage: 'Задача ещѣ не принята сервером',
332             ),
333         );
334         return;
335     }
336     _ws.cancelJob(jobId, 'cancel-$jobId');
337 }
338
339 @override
340 Future<void> close() {
341     _sub?.cancel();
342     return super.close();
343 }
344 }

```

Листинг 23: Локальное хранение результатов экспериментов
(mobile/lib/storage/database.dart)

```
1 import 'package:sqflite/sqflite.dart';
2 import 'package:path_provider/path_provider.dart';
3 import '../core/models/job_result.dart';
4
5 class AppDatabase {
6   static Database? _db;
7
8   static Future<Database> get database async {
9     _db ??= await _open();
10    return _db!;
11  }
12
13  static Future<Database> _open() async {
14    final dir = await getApplicationDocumentsDirectory();
15    final path = '${dir.path}/optimizer.db';
16    return openDatabase(
17      path,
18      version: 1,
19      onCreate: (db, version) async {
20        await db.execute('''
21          CREATE TABLE job_results (
22            id INTEGER PRIMARY KEY AUTOINCREMENT,
23            job_id TEXT NOT NULL,
24            method TEXT NOT NULL,
25            objective_name TEXT NOT NULL,
26            objective_kind TEXT NOT NULL,
27            dims INTEGER NOT NULL,
28            best_f REAL NOT NULL,
29            status TEXT NOT NULL,
30            result_json TEXT,
31            metrics_json TEXT,
32            config_json TEXT,
33            created_at TEXT NOT NULL
34          )
35        ''');
36      },
37    );
38  }
39
40  static Future<int> insertResult(SavedJobResult result) async {
41    final db = await database;
42    return db.insert('job_results', result.toDbMap());
43  }
44
45  static Future<List<SavedJobResult>> getAllResults() async {
46    final db = await database;
```

```

47     final maps =
48         await db.query('job_results', orderBy: 'created_at DESC');
49     return maps.map(SavedJobResult.fromDbMap).toList();
50 }
51
52 static Future<List<SavedJobResult>> getResultsByFilter({
53     String? method,
54     String? objectiveName,
55     String? status,
56 }) async {
57     final db = await database;
58     final where = <String>[];
59     final args = <dynamic>[];
60     if (method != null) {
61         where.add('method = ?');
62         args.add(method);
63     }
64     if (objectiveName != null) {
65         where.add('objective_name = ?');
66         args.add(objectiveName);
67     }
68     if (status != null) {
69         where.add('status = ?');
70         args.add(status);
71     }
72     final maps = await db.query(
73         'job_results',
74         where: where.isEmpty ? null : where.join(' AND '),
75         whereArgs: args.isEmpty ? null : args,
76         orderBy: 'created_at DESC',
77     );
78     return maps.map(SavedJobResult.fromDbMap).toList();
79 }
80
81 static Future<SavedJobResult?> getResultById(int id) async {
82     final db = await database;
83     final maps =
84         await db.query('job_results', where: 'id = ?', whereArgs: [id])
85         ;
86     if (maps.isEmpty) return null;
87     return SavedJobResult.fromDbMap(maps.first);
88 }
89
90 static Future<void> deleteResult(int id) async {
91     final db = await database;
92     await db.delete('job_results', where: 'id = ?', whereArgs: [id]);
93 }

```

```

94 static Future<void> deleteAll() async {
95     final db = await database;
96     await db.delete('job_results');
97 }
98 }

```

Листинг 24: Отрисовка 3D-поверхности, траектории поиска и найденного решения (mobile/lib/visualization/surface_3dPainter.dart)

```

1 import 'dart:math';
2 import 'package:flutter/material.dart';
3 import 'color_maps.dart';
4
5 typedef TestFunction3D = double Function(List<double> x);
6
7 class _Face {
8     final Offset a, b, c, d;
9     final double depth;
10    final Color color;
11
12    _Face(this.a, this.b, this.c, this.d, this.depth, this.color);
13 }
14
15 class Surface3DPainter extends CustomPainter {
16     final TestFunction3D function;
17     final double xMin, xMax, yMin, yMax;
18     final int resolution;
19     final double rotX;
20     final double rotZ;
21     final int dimX;
22     final int dimY;
23
24     final List<List<double>>? historyBestX;
25
26     final List<double>? bestPoint;
27
28     Surface3DPainter({
29         required this.function,
30         required this.xMin,
31         required this.xMax,
32         required this.yMin,
33         required this.yMax,
34         this.resolution = 50,
35         this.rotX = -0.6,
36         this.rotZ = 0.8,
37         this.dimX = 0,
38         this.dimY = 1,
39         this.historyBestX,

```

```

40     this.bestPoint,
41 });
42
43 @override
44 void paint(Canvas canvas, Size size) {
45     final sw = size.width;
46     final sh = size.height;
47     final cx = sw / 2;
48     final cy = sh / 2;
49     final scale = min(sw, sh) * 0.38;
50
51     final n = resolution;
52     final grid = List.generate(n + 1, (_) => List<double>.filled(n + 1,
53         0.0));
54     double fMin = double.infinity;
55     double fMax = double.negativeInfinity;
56
57     for (int i = 0; i <= n; i++) {
58         final xVal = xMin + (xMax - xMin) * i / n;
59         for (int j = 0; j <= n; j++) {
60             final yVal = yMin + (yMax - yMin) * j / n;
61             final point = _makePoint(xVal, yVal);
62             final f = function(point);
63             grid[i][j] = f;
64             if (f < fMin) fMin = f;
65             if (f > fMax) fMax = f;
66         }
67     }
68
69     final fRange = fMax - fMin;
70     if (fRange == 0) return;
71
72     final cosX = cos(rotX), sinX = sin(rotX);
73     final cosZ = cos(rotZ), sinZ = sin(rotZ);
74
75     Offset project(double px, double py, double pz) {
76         final nx = 2 * (px - xMin) / (xMax - xMin) - 1;
77         final ny = 2 * (py - yMin) / (yMax - yMin) - 1;
78         final nz = 2 * (pz - fMin) / fRange - 1;
79
80         final rx = nx * cosZ - ny * sinZ;
81         final ry = nx * sinZ + ny * cosZ;
82         final rz = nz;
83
84         final ry2 = ry * cosX - rz * sinX;
85
86         return Offset(cx + rx * scale, cy - ry2 * scale);
87     }

```

```

87
88     double depth(double px, double py, double pz) {
89         final nx = 2 * (px - xMin) / (xMax - xMin) - 1;
90         final ny = 2 * (py - yMin) / (yMax - yMin) - 1;
91         final nz = 2 * (pz - fMin) / fRange - 1;
92
93         final ry = nx * sinZ + ny * cosZ;
94
95         return ry * sinX + nz * cosX;
96     }
97
98     final faces = <_Face>[];
99
100    for (int i = 0; i < n; i++) {
101        final x0 = xMin + (xMax - xMin) * i / n;
102        final x1 = xMin + (xMax - xMin) * (i + 1) / n;
103        for (int j = 0; j < n; j++) {
104            final y0 = yMin + (yMax - yMin) * j / n;
105            final y1 = yMin + (yMax - yMin) * (j + 1) / n;
106
107            final f00 = grid[i][j];
108            final f10 = grid[i + 1][j];
109            final f11 = grid[i + 1][j + 1];
110            final f01 = grid[i][j + 1];
111
112            final a = project(x0, y0, f00);
113            final b = project(x1, y0, f10);
114            final c = project(x1, y1, f11);
115            final d = project(x0, y1, f01);
116
117            final avgF = (f00 + f10 + f11 + f01) / 4;
118            final t = _logScale(avgF, fMin, fRange);
119            final color = ColorMaps.viridis(t);
120
121            final avgDepth = (depth(x0, y0, f00) + depth(x1, y0, f10) +
122                depth(x1, y1, f11) + depth(x0, y1, f01)) /
123                4;
124
125            faces.add(_Face(a, b, c, d, avgDepth, color));
126        }
127    }
128
129    faces.sort((a, b) => a.depth.compareTo(b.depth));
130
131    final paint = Paint()..style = PaintingStyle.fill;
132    final edgePaint = Paint()
133        ..style = PaintingStyle.stroke
134        ..strokeWidth = 0.3

```

```

135         ..color = Colors.black26;
136
137     for (final face in faces) {
138         final path = Path()
139             ..moveTo(face.a.dx, face.a.dy)
140             ..lineTo(face.b.dx, face.b.dy)
141             ..lineTo(face.c.dx, face.c.dy)
142             ..lineTo(face.d.dx, face.d.dy)
143             ..close();
144
145         paint.color = face.color;
146         canvas.drawPath(path, paint);
147         canvas.drawPath(path, edgePaint);
148     }
149
150     _drawAxes(canvas, size, project, cx, cy, scale);
151
152     if (historyBestX != null && historyBestX!.isNotEmpty) {
153         final pathPaint = Paint()
154             ..color = Colors.red.withValues(alpha: 0.9)
155             ..strokeWidth = 2.0
156             ..style = PaintingStyle.stroke;
157
158         final dotPaint = Paint()
159             ..color = Colors.red
160             ..style = PaintingStyle.fill;
161
162         final path = _samplePath(historyBestX!, 60);
163
164         Offset? prev;
165         for (final pt in path) {
166             if (pt.length <= max(dimX, dimY)) continue;
167             final px = pt[dimX];
168             final py = pt[dimY];
169             if (px < xMin || px > xMax || py < yMin || py > yMax) continue;
170             final fVal = function(_makePoint(px, py));
171             final projected = project(px, py, fVal);
172
173             if (prev != null) {
174                 canvas.drawLine(prev, projected, pathPaint);
175             }
176             canvas.drawCircle(projected, 3.0, dotPaint);
177             prev = projected;
178         }
179     }
180
181     if (bestPoint != null && bestPoint!.length > max(dimX, dimY)) {
182         final bx = bestPoint![dimX];

```

```

183         final by = bestPoint![dimY];
184         if (bx >= xMin && bx <= xMax && by >= yMin && by <= yMax) {
185             final fVal = function(_makePoint(bx, by));
186             final projected = project(bx, by, fVal);
187             _drawStar(canvas, projected, 10, Colors.yellow, Colors.red);
188         }
189     }
190 }
191
192 List<double> _makePoint(double xVal, double yVal) {
193     final dims = max(dimX, dimY) + 1;
194     final point = List<double>.filled(max(dims, 2), 0.0);
195     if (bestPoint != null) {
196         for (int i = 0; i < min(bestPoint!.length, point.length); i++) {
197             point[i] = bestPoint![i];
198         }
199     }
200     point[dimX] = xVal;
201     point[dimY] = yVal;
202     return point;
203 }
204
205 double _logScale(double val, double fMin, double fRange) {
206     final shifted = val - fMin;
207     if (fRange <= 0) return 0;
208     return log(shifted + 1) / log(fRange + 1);
209 }
210
211 List<List<double>> _samplePath(List<List<double>> full, int maxPoints
    ) {
212     if (full.length <= maxPoints) return full;
213     final step = full.length / maxPoints;
214     return List.generate(maxPoints, (i) => full[(i * step).floor()]);
215 }
216
217 void _drawAxes(Canvas canvas, Size size, Offset Function(double,
    double, double) project,
218     double cx, double cy, double scale) {
219     final tp = TextPainter(textDirection: TextDirection.ltr);
220
221     void drawLabel(String text, Offset pos) {
222         tp.text = TextSpan(
223             text: text,
224             style: const TextStyle(color: Colors.black87, fontSize: 12,
225                 fontWeight: FontWeight.w600),
226         );
227         tp.layout();
228         tp.paint(canvas, pos - Offset(tp.width / 2, tp.height / 2));

```

```

228     }
229
230     final xEnd = project(xMax, yMin, 0);
231     drawLabel('x$dimX', Offset(xEnd.dx, xEnd.dy + 14));
232
233     final yEnd = project(xMin, yMax, 0);
234     drawLabel('y$dimY', Offset(yEnd.dx - 14, yEnd.dy));
235 }
236
237 void _drawStar(Canvas canvas, Offset center, double r, Color fill,
    Color stroke) {
238     final path = Path();
239     for (int i = 0; i < 10; i++) {
240         final radius = i.isEven ? r : r / 2.5;
241         final angle = -pi / 2 + i * pi / 5;
242         final point = Offset(center.dx + radius * cos(angle), center.dy +
            radius * sin(angle));
243         if (i == 0) {
244             path.moveTo(point.dx, point.dy);
245         } else {
246             path.lineTo(point.dx, point.dy);
247         }
248     }
249     path.close();
250     canvas.drawPath(path, Paint()..color = fill..style = PaintingStyle.
        fill);
251     canvas.drawPath(
252         path,
253         Paint()
254             ..color = stroke
255             ..style = PaintingStyle.stroke
256             ..strokeWidth = 1.5,
257     );
258 }
259
260 @override
261 bool shouldRepaint(covariant Surface3DPainter old) =>
262     old.rotX != rotX || old.rotZ != rotZ || old.resolution !=
        resolution;
263 }

```